

Topology and Remeshing in ParaDiS

Execution Flow and Detailed Code Guide

ParaDiS-Extended Development Notes

August 2026

Abstract

ParaDiS evolves a graph of dislocation nodes and segments. Force integration moves the graph geometrically; topology code changes its connectivity; and remeshing changes its discretization while preserving the physical network. This guide follows those operations from the outer time-step driver through node splitting, node merging, collision handling, distributed-operation replay, mesh coarsening, and mesh refinement. It is written against the repository state at commit `cca5ff2`.

The code excerpts are included directly from the source tree and retain their original line numbers. Each selected executable block is followed by a line-by-line explanation. Large, repetitive material-specific routines are described at their dispatch and mutation boundaries rather than copied five times. The source files remain authoritative when a future change moves a line or changes a condition.

Contents

1	Scope and reading convention	3
1.1	What topology and remesh mean in ParaDiS	3
1.2	How to read the listings	3
1.3	One-cycle mental model	3
2	Data model, controls, and invariants	4
2.1	Node state and topology flags	4
2.2	Public topology and remesh interfaces	7
2.3	Control-file parameters and derived thresholds	10
3	Where operations occur in one ParaDiS cycle	12
3.1	Collision-search preparation	15
4	The shared topology mutation layer	20
4.1	Ownership is the primary parallel invariant	20
4.2	Per-cycle exemptions and temporary mobility	22
4.3	<code>SplitNode()</code> : one primitive for separation and refinement	26
4.4	<code>MergeNode()</code> : common coarsen and collision primitive	34
4.5	Duplicate links and orphan nodes	40

5	Topology-event selection	44
5.1	Serial multi-node splitting	44
5.2	BCC binary-junction classifier	64
5.3	Parallel multi-node splitting	66
5.4	Cross slip and collision dispatch	68
6	MPI topology synchronization	74
6.1	Operation-log protocol	74
6.2	Sending the log	78
6.3	Replaying neighbouring operations	81
7	Remeshing	86
7.1	Dispatcher and postconditions	86
7.2	Force estimates used during topology changes	89
7.3	Mesh coarsening: selecting a removal	91
7.4	Mesh coarsening: committing nonoverlapping removals	98
7.5	Rule 2 refinement: midpoint insertion	105
7.6	Rule 3 refinement: fitted-arc insertion	116
7.7	Rule comparison	126
8	End-to-end transactions	126
8.1	Example: a long degree-two segment	126
8.2	Example: a short, collapsing degree-two node	126
8.3	Example: a multi-node separation	127
9	Validation, limitations, and developer checklist	127
9.1	Initialization checks that protect remesh	127
9.2	Implementation details to keep in mind	128
9.3	Invariants for topology modifications	129
9.4	Useful breakpoints and logging	129
9.5	Checklist for a new topology event	129

1 Scope and reading convention

1.1 What topology and remesh mean in ParaDiS

In this guide, *topology* means an operation that changes the graph: an arm is moved, a node is created or removed, a segment endpoint is relinked, or a Burgers vector/glide plane is changed. These events arise from multi-node separation, collision/annihilation, cross slip, segment splintering, HCP-specific splitting, and, for free surfaces, surface operations.

Remeshing is the later, dedicated rediscretization pass. It removes an eligible degree-two node to coarsen a short or collapsing local shape, or inserts a degree-two node to refine a long or highly curved segment. It uses the same `SplitNode()` and `MergeNode()` primitives as topology events, but it has its own selection criteria and MPI synchronization pass.

1.2 How to read the listings

Each listing shows exact source line numbers, not document-local line numbers. One explanation item may cover a contiguous group of lines when the lines together implement one expression, declaration, loop, or preprocessor guard. Braces, blank lines, and comments are explained with their surrounding code rather than restated mechanically. This gives the reader a practical line-by-line guide without pretending that an isolated brace has independent behavior.

File	Responsibility
src/ParadisStep.cc	Places topology and remesh in each accepted outer cycle.
src/Topology.cc	Shared mutation primitives, serial multi-node evaluation, ownership checks, duplicate-link cleanup.
src/Remesh.cc	Generic remesh dispatcher, force estimates, and common mesh coarsening.
src/RemeshRule_2.cc	Rule 2 refinement, which inserts at a segment midpoint.
src/RemeshRule_3.cc	Rule 3 refinement, which may place a degree-two refinement point on a fitted arc.
src/CommSendRemesh.cc	Pack, send, and replay graph mutations across MPI domains.
src/FixRemesh.cc	
src/RemoteOps.cc	
src/SplitMultiNodesParallel.cc	
src/SMN*.cc	Distributed force evaluation for parallel multi-node splitting.

1.3 One-cycle mental model

Integrate positions and velocities → initialise topology exemptions → identify/split multi-nodes → cross slip → collision and special material events → distribute/replay those changes → remove duplicate links → coarsen/refine the mesh → distribute/replay remesh changes → remove any orphan nodes.

The order is intentional. Topological physics events run first; remesh then cleans up the graph they leave behind. Two operation-list exchanges are normal in a parallel simulation: one after collision/topology work, and another after remeshing.

2 Data model, controls, and invariants

2.1 Node state and topology flags

```
24 /*
25  *      Define the various permitted nodal constraints.  The nodal constraint
26  *      is a single integer value in which each possible constraint is
27  *      represented by a single bit in the integer.  This allows a node
28  *      to be constrained in multiple ways.
29  *
30  *      WARNING: Currently, certain portions of ParaDiS (primarily the
31  *      code dealing with topological changes) do not support
32  *      pinning a node in only specific dimensions.  As a result,
33  *      those portions of the code will treat a node that is
34  *      pinned in ANY dimension as if it is pinned in ALL dimensions.
35  */
36 #define CONSTRAINT_ANY      0xffffffff /* -1 */
37 #define UNCONSTRAINED      0x00000000 /* 0 */
38 #define PINNED_X            0x00000001 /* 1 ONLY PARTIALLY SUPPORTED */
39 #define PINNED_Y            0x00000002 /* 2 ONLY PARTIALLY SUPPORTED */
40 #define PINNED_Z            0x00000004 /* 4 ONLY PARTIALLY SUPPORTED */
41 #define SURFACE_NODE        0x00000008 /* 8 */
42 #define CORNER_NODE         0x00000010 /* 16: Preserve a physical corner */
43                               /* during mesh coarsening. */
44
45 #define PINNED_NODE          7          /* Special constraint which */
46                               /* represents all of: */
47                               /* PINNED_X */
48                               /* PINNED_Y */
49                               /* PINNED_Z */
50
51 /*
52  *      The following constraints are used in post-processing tools only
53  */
54 #define MULTI_JUNCTION_NODE 0x00001000 /* 4096: Node has all 4 111 burgers */
55                               /* vectors associated with it */
56
57 #define LOOP_NODE            0x00002000 /* 8192: Node is in a loop with a */
58                               /* unique burgers vector */
59
60 #define CROSS_NODE           0x00004000 /* 16384: Node is a non multi- */
61                               /* junction node with more */
62                               /* than 2 neighbors */
63
64
65 /*****
66  *
67  *      Now define some macros for examining/manipulating node
68  *      constraint values...
69  *
70  *****/
71
72 /*
73  *      If the constraint value <cVal> has any constraint corresponding
74  *      to the bits set in the constraint list <cList>, return a value
75  *      of 1, otherwise return zero.
76  */
77 #define HAS_ANY_OF_CONSTRAINTS(cVal, cList) (((cVal) & (cList)) != 0)
```

```

78
79 /*
80 *      If the constraint value <cVal> has all the constraints corresponding
81 *      to the bits set in the constraint list <cList>, return a value
82 *      of 1, otherwise return zero.
83 */
84 #define HAS_ALL_OF_CONSTRAINTS(cVal, cList) (((cVal) & (cList)) == (cList))
85
86 /*
87 *      If the constraint value <cVal> has all the constraints corresponding
88 *      to the bits set in the constraint list <cList> and no other
89 *      constraints, return a value of 1, otherwise return zero.
90 */
91 #define HAS_ONLY_CONSTRAINTS(cVal, cList) ((cVal) == (cList))
92
93 /*
94 *      If the constraint value <cVal> has none of the constraints
95 *      corresponding to the bits set in the constraint list <cList>
96 *      return a value of 1, otherwise return zero.
97 */
98 #define HAS_NONE_OF_CONSTRAINTS(cVal, cList) (((cVal) & (cList)) == 0)
99
100 /*
101 *      If the constraint value <cVal> represents an unconstrained node
102 *      return a value of 1, otherwise return zero.
103 */
104 #define HAS_NO_CONSTRAINTS(cVal) (cVal == UNCONSTRAINED)
105
106 /*
107 *      Add all constraints specified in <cList> to <cVal>.
108 */
109 #define ADD_CONSTRAINTS(cVal, cList) (cVal |= (cList))
110
111 /*
112 *      Remove any constraints specified in <cList> from <cVal>.
113 */
114 #define REMOVE_CONSTRAINTS(cVal, cList) (cVal &= ~(cList))
115
116 /*
117 *      Set the constraint value <cVal> to the constraint list
118 *      specified in <cList>.
119 */
120 #define SET_CONSTRAINTS(cVal, cList) (cVal = (cList))
121
122
123 /*
124 *      Define the bit flags that can be set for the node.  Note: these
125 *      flags may be OR'ed together.
126 */
127 #define NODE_RESET_FORCES      0x01
128 #define NODE_OUTSIDE_SURF      0x02
129 #define NO_COLLISIONS          0x04
130 #define NO_MESH_COARSEN        0x08
131 #define NODE_CHK_DBL_LINK      0x10
132
133 /*
134 *      Used as bit flags to indicate the type of nodal data items
135 *      preserved by calls to PreserveNodalData().
136 */

```

```

137 #define NODE_POSITION 0x01
138 #define NODE_CURR_VEL 0x02
139 #define NODE_OLD_VEL 0x04
140
141 class Node_t
142 {
143     public:
144         int      flags;
145
146         real8     x , y , z ;           // nodal position
147         real8     fX, fY, fZ;           // nodal force: units=Pa*b^2)
148         real8     vX, vY, vZ;           // nodal velocity: units=burgMag/sec
149
150
151         real8     oldx , oldy , oldz ;   // for strain increment, Moono.Rhee
152         real8     oldvX , oldvY , oldvZ ; // nodal velocity at previous step
153         real8     currvX, currvY, currvZ; // nodal velocity at beginning of the
154                                           // current step. Only used during
155                                           // timestep integration while vX, vY
156                                           // and vZ are being determined.
157
158         Tag_t     myTag;                 // tag associated with this node
159
160         int       myIndex;               // index associated with this node (for gpu)
161
162         int       numNbrs;               // number of neighbors
163         Tag_t     *nbrTag;               // array of neighbor tags
164         int       *nbrIndex;             // array of node indices if neighbors (for gpu)
165
166         real8     *armfx, *armfy, *armfz; // arm specific force contribution
167         real8     *burgX, *burgY, *burgZ; // burgers vector
168         real8     *nx , *ny , *nz ;      // glide plane
169
170         real8     *sigbLoc;              // sig.b on arms (numNbr*3)
171         real8     *sigbRem;              // sig.b on arms (numNbr*3)
172
173         int       *armCoordIndex;        // Array of indices (1 per arm) into
174                                           // the mirror domain's arrays of
175                                           // coordinates (armX, armY, armZ)
176                                           // of nodes' neighbors. This array
177                                           // is only present and useful while
178                                           // task zero is downloading the data
179                                           // from the remote domain for
180                                           // generating output
181
182         int       *armNodeIndex;         // Array of node/arm indices into the
183                                           // current node list.
184
185         int       constraint;            // constraint = 1 : any surface node
186                                           // constraint = 7 : Frank-Read end
187                                           // points, fixed
188                                           // constraint = 9 : Jog
189                                           // constraint = 10 : Junction
190                                           // constraint = 20 : cross slip(default)
191                                           // 2-nodes non-deletable
192
193
194         int       cellIdx;               // cell node is currently sorted into
195         int       cell2Idx;              // cell2 node is currently sorted into

```

```

196     int    cell2QentIdx;           // Index of this node's entry in the
197                                     // home->cell2QentArray.
198
199     int    native;                // 1 = native node, 0 = ghost node
200
201     Node_t *next;                 // pointer to the next node in the queue
202                                     // (ghost or free)
203
204     Node_t *nextInCell;           // used to queue node onto the current
205                                     // containing cell

```

Line-by-line explanation.

Lines 24–50

Constraints are bit flags. Pin bits identify fixed coordinates, `SURFACE_NODE` identifies a free-surface node, and `CORNER_NODE` marks a physical degree-two corner that remesh should not normally collapse. The topology implementation currently treats any pin bit as a fully pinned node; the warning at lines 30–35 is operational.

Lines 72–120

The macros make bitfield tests readable. For example, `HAS_ANY_OF_CONSTRAINTS(node->constraint, PINNED_NODE)` is the normal test for a node that cannot be deleted or freely repositioned.

Lines 123–131

These are transient per-cycle flags. `NO_COLLISIONS` prevents a node touched by one event from being reused by another collision in the same cycle. `NO_MESH_COARSEN` similarly prevents overlapping coarsening. `NODE_RESET_FORCES` says cached force/velocity values are not reliable enough for geometry-sensitive work.

Lines 141–153

A node owns position, total force, velocity, and history. The `old*` and `currv*` fields support integration and plastic-strain bookkeeping, so topology must preserve or deliberately update them.

Lines 158–171

`myTag` is the globally meaningful identity. `nbrTag` identifies adjacent nodes. Per-arm force, Burgers-vector, and glide-plane arrays are the segment data stored at this endpoint.

Lines 185–205

`constraint` is persistent state. `cellIdx`, `cell2Idx`, and `cell2QentIdx` connect a node to spatial acceleration structures. `native` distinguishes local ownership from a ghost replica.

2.2 Public topology and remesh interfaces

```

16  #include "Typedefs.h"
17  #include "Node.h"
18  #include "Tag.h"
19
20
21  /*
22   *      Define the set of status codes that may be returned from
23   *      the MergeNode() function.
24   */
25  #define MERGE_SUCCESS          0x01 /* Merge succeeded          */
26  #define MERGE_NODE_ORPHANED    0x02 /* Merge succeeded, but left an */
27                                  /* orphaned node in a remote    */

```

```

28                                     /* domain */
29 #define MERGE_NO_REPOSITION          0x04 /* Merge succeeded, but the */
30                                     /* resulting node was not able */
31                                     /* to be repositioned. */
32 #define MERGE_NOT_PERMITTED          0x08 /* Merge failed because the */
33                                     /* requested merge would violate */
34                                     /* segment ownership rules */
35 #define MERGE_DOUBLE_LINK            0x10 /* Merge failed because it would */
36                                     /* have resulted in doubly */
37                                     /* connected nodes in another */
38                                     /* domain */
39 /*
40  *   Define the set of status codes that may be returned from
41  *   the SplitNode() function.
42  */
43 #define SPLIT_FAILED    0
44 #define SPLIT_SUCCESS   1
45
46 /*
47  *   Define any processing flags that can be provided to SplitNode() to
48  *   affect its behavior.
49  */
50 #define SPLIT_DUP_SURFACE_PROP 0x01
51
52
53 #define OPCLASS_SEPARATION  1
54 #define OPCLASS_COLLISION  2
55 #define OPCLASS_REMESH     3
56 /*
57  *   Prototypes for functions involved in altering topology
58  */
59 int   CheckCollisionConditions(Home_t *home, Node_t *node1, Node_t *node2);
60 int   EvaluateMobility(Home_t *home, Node_t *nodeA, MobArgs_t *mobArgs);
61 int   InitTopologyExemptions(Home_t *home);
62 void  MergeNode(Home_t *home, int opClass, Node_t *node1, Node_t *node2,
63               real8 *position, Node_t **mergedNode, int *status, int globalOp);
64 int   MergeNodePreCondition(Home_t *home, int opClass, Node_t *node1, Node_t *node2);
65 int   NodeTopologyExemptions(Home_t *home, Node_t *node);
66 int   RemoveDoubleLinks(Home_t *home, Node_t *node, int globalOp);
67 void  RemoveOrphanedNodes(Home_t *home);
68 void  SplitMultiNodes(Home_t *home);
69 int   BCC_binary_junction_node(Home_t *home, Node_t *node,
70                               real8 tjunc[3], int *planarJunc);
71 int   SplitNode(Home_t *home, int opClass, Node_t *node, real8 *pos1,
72               real8 *pos2, real8 *vel1, real8 *vel2, int armCount,
73               int *armList, int globalOp, Node_t **splitNode1,
74               Node_t **splitNode2, int flags);
75
76 /*
77  *   Some functions for preserving and restoring node info
78  *   while attempting multi-node or surface-node splits
79  */
80 void  BackupNode(Home_t *home, Node_t *origNode, Node_t *bkupNode);
81 void  FreeNodeArrays(Node_t *node);
82 void  GetForcesFromBkup(Home_t *home, Node_t *node, Node_t *bkupNodeList);
83 void  RestoreNode(Home_t *home, Node_t *origNode, Node_t *bkupNode);
84
85 /*
86  *   Define some functions we use when creating a new junction in

```



```

87 *      SplitMultiNode() to calculate the optimal length, direction
88 *      and forces for the new junction.
89 */
90 void AdjustJuncNodeForceAndVel(Home_t *home, Node_t *node,
91                               MobArgs_t *mobArgs);
92
93 real8 CalcJunctionLen(Home_t *home, Node_t *node, int armCount,
94                      int *armList, real8 dirx, real8 diry, real8 dirz,
95                      real8 splitDist, int isOpposite);
96
97 #ifdef FIX_PLASTIC_STRAIN
98 void UpdateNodePlasticStrain(Home_t *home, Node_t *node, real8 *oldPos, real8 *newPos);
99 #endif
100
101 #endif /* _TOPOLOGY_H */

```

```

20 enum {
21     REMESH_PENDING = 0,
22     REMESH_COMMITTED,
23     REMESH_REJECTED
24 };
25
26
27 /*
28 *      Structure to hold information regarding nodes selected to be
29 *      coarsened out during the first phase of MeshCoarsen().
30 */
31 typedef struct {
32     int status;
33     Tag_t nodeTag;
34     Tag_t nbr1Tag;
35     Tag_t nbr2Tag;
36 } CoarsenList_t;
37
38
39 /*
40 *      Structure to hold information regarding segments selected to be
41 *      refined out during the first phase of MeshRefine().
42 */
43 typedef struct {
44     int status;
45     int haveNewPos; /* only for remesh version 3 */
46     Tag_t node1Tag;
47     Tag_t node2Tag;
48     Tag_t newNodeTag;
49     real8 vector[3];
50     real8 f0Seg1[3];
51     real8 f1Seg1[3];
52     real8 f0Seg2[3];
53     real8 f1Seg2[3];
54     real8 newPos[3]; /* only for remesh version 3 */
55 } RefineList_t;
56
57
58 /*
59 *      Prototype the global functions related to rediscretization
60 */
61 void EstCoarsenForces(Home_t *home, Node_t *node1, Node_t *node2,

```

```

62     Node_t *node3, real8 f0Seg[3], real8 f1Seg[3]);
63 void EstRefinementForces(Home_t *home, Node_t *node1, Node_t *node2,
64     real8 newPos[3], real8 vec[3],
65     real8 f0Seg1[3], real8 f1Seg1[3],
66     real8 f0Seg2[3], real8 f1Seg2[3]);
67 void MeshCoarsen(Home_t *home);
68 int ProceedWithCoarsen(CoarsenList_t *list, int listEnts, Tag_t *nodeTag,
69     Tag_t *nbr1Tag, Tag_t *nbr2Tag);
70 void Remesh(Home_t *home);
71 void RemeshRule_2(Home_t *home);
72 void RemeshRule_3(Home_t *home);
73
74 #endif /* _REMESH_H */

```

Line-by-line explanation.

Topology.h, lines 16–31

Merge statuses are bitwise. A caller must test `MERGE_SUCCESS` rather than compare the full result: success may coexist with `MERGE_NO_REPOSITION` or `MERGE_NODE_ORPHANED`.

Topology.h, lines 36–55

`SplitNode()` returns a simple success/failure value. The operation classes identify separation, collision, and remesh invocations so common primitives can enforce suitable rules.

Topology.h, lines 59–100

The header separates high-level selection routines such as `SplitMultiNodes()` from low-level mutation routines such as `SplitNode()` and `MergeNode()`. A new feature should normally select an event at the high level and use these primitives.

Remesh.h, lines 20–24

A candidate event starts `REMESH_PENDING`; an executed event becomes `REMESH_COMMITTED`; overlapping or invalid candidates become `REMESH_REJECTED`.

Remesh.h, lines 30–59

Coarsen/refine records store `Tag_t`s instead of pointers because a prior event can delete or move a node. A refinement record also stores estimated force values for the two replacement segments. Rule 3 uses `haveNewPos` and `newPos` to carry its fitted-arc insertion point from selection to commit.

Remesh.h, lines 64–72

`MeshCoarsen()` is shared by rules 2 and 3. The rule functions differ principally in `MeshRefine()` placement logic.

2.3 Control-file parameters and derived thresholds

```

227 *      Discretization parameters and controls for topological changes
228 */
229     real8 minSeg;      /* min allowable segment length, before */
230                       /* removing a node */
231     real8 maxSeg;      /* max allowable segment length, before*/
232                       /* adding a node*/
233     real8 rann;        /* Annihilation distance (units of b) used in      */
234                       /* collision handling, rediscrretization, multi- */
235                       /* node splitting, etc. Defines closest distance */
236                       /* before dislocations are considered in contact */
237
238     int remeshRule;
239     int collisionMethod;

```

```

240     real8 remeshAreaMax; /* This is calculated from remeshAreaMin and */
241                          /* remeshAreaRatio and hence not user-provided*/
242     real8 remeshAreaMin; /* This values is based on the minSeg value */
243                          /* and hence not specified by the user. */
244     int splitMultiNodeFreq; /* Code will attempt to split multi-nodes */
245                          /* every cycle that is a multiple of this */
246                          /* value. */
247
248     real8 splitMultiNodeAlpha; /* factor used to control the split multi-node */
249                          /* vNoise term */
250
251     int split3node; /* Toggle BCC binary-junction */
252                  /* three-arm node splitting. */
253
254     int useParallelSplitMultiNode; /*Toggle indicating if the code */
255                          /* should use the parallel method */
256                          /* of multi-node splitting or not */
257
258     int useUniformJunctionLen; /* Toggle indicating whether to create */
259                          /* junctions of a uniform length in */
260                          /* SplitMultiNodes() or try to calc */
261                          /* the optimal junction length. */

```

```

579     // If not provided, set position error tolerance based on <rc>
580
581     if (param->rTol <= 0.0) {
582         param->rTol = 0.25 * param->rc;
583     }
584
585     // The deltaTT is set in the timestep integrator, but some
586     // mobility functions now use the deltaTT value, so it must
587     // be initialized before ParadisStep() is called since there
588     // is an initial mobility calculation done *before* timestep
589     // integration the first time into the function.
590
591     param->deltaTT = MIN(param->maxDT, param->nextDT);
592
593     if (param->deltaTT <= 0.0) {
594         param->deltaTT = param->maxDT;
595     }
596
597     // If the annihilation distance has not been provided, set
598     // it to a default based on <rc>
599
600     if (param->rann <= 0.0) {
601         param->rann = 2.0 * param->rTol;
602     }
603
604     // Minimum area criteria for remesh is dependent on maximum
605     // and minumum segment lengths and position error tolerance.
606
607     param->remeshAreaMin = 2.0 * param->rTol * param->maxSeg;
608
609     if (param->minSeg > 0.0) {
610         param->remeshAreaMin = MIN(param->remeshAreaMin,
611                                   (param->minSeg * param->minSeg *
612                                    sqrt(3.0) / 4));
612     }
613

```

```

614
615     // Maximum area criteria for remesh is dependent on minimum area,
616     // and maximum segment length.
617
618     param->remeshAreaMax = 0.5 * ((4.0 * param->remeshAreaMin) +
619                                   (0.25 * sqrt(3)) *
620                                   (param->maxSeg*param->maxSeg));
621
622     // If the user did not provide a minSeg length, calculate one
623     // based on the remesh minimum area criteria.
624
625     if (param->minSeg <= 0.0) {
626         param->minSeg = sqrt(param->remeshAreaMin * (4.0 / sqrt(3)));
627     }

```

Line-by-line explanation.

Param.h, lines 227–243

`minSeg` and `maxSeg` bound target discretization. `rann` is the annihilation/contact distance and is reused by collisions, multi-node separation, and remesh safety guards. The internal geometric thresholds are `remeshAreaMin` and `remeshAreaMax`.

Param.h, lines 244–261

`splitMultiNodeFreq` throttles multi-node work. `splitMultiNodeAlpha` supplies the noise penalty used when comparing trial split power. `split3node` enables the special BCC binary-junction case; the final controls select parallel evaluation and uniform versus calculated junction length.

Initialize.cc, lines 579–602

If absent, position tolerance becomes one quarter of core radius and `rann` becomes twice the tolerance. These defaults propagate into topology distances.

Initialize.cc, lines 604–620

Minimum area starts as $2rTol\ maxSeg$, is tightened by the equilateral-triangle area implied by `minSeg` when provided, and then produces maximum area. The formulas make curvature criteria consistent with length and numerical tolerances.

Initialize.cc, lines 622–627

When `minSeg` is absent, the code derives it from minimum area. A control file can therefore specify `maxSeg/rTol` and obtain a consistent lower length limit.

```

1  # Typical controls; values must suit material, units, and cell size.
2  maxSeg                = 200.0
3  minSeg                = 50.0
4  rann                  = 2.5
5  remeshRule            = 2
6  splitMultiNodeFreq    = 1
7  splitMultiNodeAlpha   = 1.0e-3
8  split3node            = 1
9  useParallelSplitMultiNode = 0
10 useUniformJunctionLen  = 0
11 collisionMethod        = 2

```

3 Where operations occur in one ParaDiS cycle

```

380     // nodes exempt from topological changes. These flags are used

```

```

381 // in both splitting multi-arm nodes and collisions, so this
382 // function should be invoked before either of those items are done.
383
384 InitTopologyExemptions(home);
385
386 // Now do all the topological changes from segment interactions
387 // (collisions, multinode splitting)... Clear the list of local
388 // operations that will be sent to the remote domains for processing,
389 // then split any multi-arm nodes that need splitting, cross slip
390 // nodes (as needed/allowed), handle all local collisions, then
391 // send remote nodes the list of ops needed to keep their data in sync.
392
393 ClearOpList(home);
394 SortNodesForCollision(home);
395
396 #ifdef FIX_PLASTIC_STRAIN
397 // Set the plastic strain correction to zero before handling
398 // the topological operations.
399
400 ResetPlasticStrain(home);
401 #endif
402
403 if (param->useParallelSplitMultiNode) { SplitMultiNodesParallel(home); }
404 else { SplitMultiNodes(home); }
405
406 #ifdef _ARLFEM
407 SplitSurfaceNodes(home);
408 #endif
409
410 // Call a generic cross-slip dispatch function that will invoke
411 // (if necessary) the cross-slip function appropriate to the
412 // type of material in use.
413
414 CrossSlip(home);
415
416 // Search for dislocation segments in close proximity to each other
417 // and if necessary handle any collision between them.
418
419 HandleCollisions(home);
420
421 // Check for any segments that should be decomposed (splintered)
422 // into multiple segments of a different burgers vector.
423
424 SplinterSegments(home);
425
426 // For HCP simulations, we may want to split segments with <c+a>
427 // burgers into two segments with a <c> and an <a> burgers vector
428 // respectively, and cross-slip one to a new plane...
429
430 HCP_CA_Split(home);
431
432 #ifdef ESHELBY
433 // Handle collisions between dislocations and particles
434
435 if (home->param->RefineSegCollision == 1)
436     HandleParticleCollisions(home);
437 #endif
438
439 #ifdef PARALLEL

```

```

440 #ifdef SYNC_TIMERS
441     TimerStart (home, POST_COLLISION_BARRIER);
442     MPI_Barrier(MPI_COMM_WORLD);
443     TimerStop  (home, POST_COLLISION_BARRIER);
444 #endif
445 #endif
446     TimerStart      (home, COL_SEND_REMESH);
447     CommSendRemesh(home);
448     TimerStop       (home, COL_SEND_REMESH);
449
450     TimerStart      (home, COL_FIX_REMESH );
451     FixRemesh       (home);
452     TimerStop       (home, COL_FIX_REMESH );
453
454     // For debugging purposes only, the following check has been added
455     // to force an immediate abort if we detect an unconstrained node
456     // at which the burgers vector is not conserved. This code can
457     // be removed once all the above topology modifying functions
458     // have been verified to play well together.
459
460     CheckBurgConservation(home, "After FixRemesh()");
461
462 #ifdef _ARLFEM
463     FixSurfaceTopology(home);
464 #endif
465
466     // Under certain circumstances, parallel topological changes can
467     // create double links between nodes; links which can not be detected
468     // until after FixRemesh() is called... so, a quick check has to be
469     // done to clean up these potential double-links here, or they will
470     // cause problems later on. Should only have to check nodes local
471     // to this domain.
472
473     for (int i=0; i < home->newNodeKeyPtr; i++)
474     {
475         Node_t *node = home->nodeKeys[i];
476
477         if (node)
478         {
479             RemoveDoubleLinks(home, node, 0);
480             node->flags &= ~NODE_CHK_DBL_LINK;
481         }
482     }
483
484     // If memory debugging is enabled, run a consistency check on all
485     // allocated memory blocks looking for corruption in any of the
486     // block headers or trailers.
487
488 #ifdef DEBUG_MEM
489     ParadisMemCheck();
490 #endif
491     // Invoke mesh coarsen/refine
492
493     Remesh(home);

```

Line-by-line explanation.

Lines 380–389

Output and the just-finished integration work occur before this excerpt. The first topology-specific action is `InitTopologyExemptions()`, which clears temporary exclusions and repairs stale corner marking.

Lines 391–395

`ClearOpList()` begins a fresh serialized change log. `SortNodesForCollision()` builds the fine `cell2` queues used to limit collision searches. It must happen before collision operations can delete nodes.

Lines 396–401

With `FIX_PLASTIC_STRAIN`, correction storage is reset before any operation changes swept area or a nodal position.

Lines 403–404

Exactly one multi-node implementation runs. The serial path evaluates trial splits locally; the parallel path distributes force contributions before committing local results.

Lines 406–414

Surface splitting is conditional on `_ARLFEM`; `CrossSlip()` is a material dispatcher. Both can change planes and positions and participate in topology synchronization.

Lines 416–436

Collisions are handled after multi-node/cross-slip events. Segment splintering, HCP splitting, and optional particle collisions then append further mutations to the same operation list.

Lines 439–452

The first `CommSendRemesh()/FixRemesh()` pair broadcasts physics-driven topology changes and replays neighboring domains' changes locally.

Lines 454–482

Burgers conservation is checked after replay. A final duplicate-link pass is needed because separately valid remote changes can jointly make two links connect the same pair of nodes.

Lines 484–493

After topology is stable, `Remesh()` performs the independent coarsen/refine pass and owns a second clear/send/fix sequence.

3.1 Collision-search preparation

```
57 static void InitCell2Values(Home_t *home,
58                             int *cell2PerCellX,
59                             int *cell2PerCellY,
60                             int *cell2PerCellZ,
61                             real8 *cell2SizeX,
62                             real8 *cell2SizeY,
63                             real8 *cell2SizeZ,
64                             real8 *cell2MinX,
65                             real8 *cell2MinY,
66                             real8 *cell2MinZ,
67                             int *numBodyCell2X,
68                             int *numBodyCell2Y,
69                             int *numBodyCell2Z)
70 {
71     real8    maxSeparation;
72     real8    Lx, Ly, Lz;
73     real8    probMinX, probMinY, probMinZ;
74     real8    probMaxX, probMaxY, probMaxZ;
75     real8    cellSizeX, cellSizeY, cellSizeZ;
```

```

76     Param_t *param;
77
78
79     param = home->param;
80
81     /*
82     *     Get the problem dimensions
83     */
84     probMinX = param->minSideX; probMaxX = param->maxSideX;
85     probMinY = param->minSideY; probMaxY = param->maxSideY;
86     probMinZ = param->minSideZ; probMaxZ = param->maxSideZ;
87
88     Lx = probMaxX - probMinX;
89     Ly = probMaxY - probMinY;
90     Lz = probMaxZ - probMinZ;
91
92     cellSizeX = Lx / param->nXcells;
93     cellSizeY = Ly / param->nYcells;
94     cellSizeZ = Lz / param->nZcells;
95
96     *cell2MinX = probMinX - cellSizeX;
97     *cell2MinY = probMinY - cellSizeY;
98     *cell2MinZ = probMinZ - cellSizeZ;
99
100    /*
101    *     Find the maximum separation between colliding node pairs
102    */
103    maxSeparation = (1.1 * param->maxSeg) + param->rann;
104
105    /*
106    *     Compute the size of cell2's, such that an integral number fall within
107    *     each cell, and the total number of cell2's in the minimum image problem
108    *     space
109    */
110    *cell2PerCellX = (int) floor(cellSizeX/maxSeparation);
111    *cell2PerCellY = (int) floor(cellSizeY/maxSeparation);
112    *cell2PerCellZ = (int) floor(cellSizeZ/maxSeparation);
113
114    *cell2PerCellX = MIN(*cell2PerCellX, MAXCELL2PERCELL);
115    *cell2PerCellY = MIN(*cell2PerCellY, MAXCELL2PERCELL);
116    *cell2PerCellZ = MIN(*cell2PerCellZ, MAXCELL2PERCELL);
117
118    *cell2PerCellX = MAX(*cell2PerCellX, 1);
119    *cell2PerCellY = MAX(*cell2PerCellY, 1);
120    *cell2PerCellZ = MAX(*cell2PerCellZ, 1);
121
122    *cell2SizeX = cellSizeX / *cell2PerCellX;
123    *cell2SizeY = cellSizeY / *cell2PerCellY;
124    *cell2SizeZ = cellSizeZ / *cell2PerCellZ;
125
126    *numBodyCell2X = *cell2PerCellX * param->nXcells;
127    *numBodyCell2Y = *cell2PerCellY * param->nYcells;
128    *numBodyCell2Z = *cell2PerCellZ * param->nZcells;
129
130    return;

```

```

265 *     Allocate and initialize the array of cell2 queue heads for this domain
266 *     and the single array containing the queues of node pointer/array index

```



```

267 *      pairs.
268 */
269     home->cell2 = (int *)realloc(home->cell2, numCell2s * sizeof(int));
270
271 #ifdef _OPENMP
272 #pragma omp parallel for
273 #endif
274     for (i = 0; i < numCell2s; i++) {
275         home->cell2[i] = -1;
276     }
277
278     home->cell2QentArray = (C2Qent_t *)realloc(home->cell2QentArray,
279                                                home->newNodeKeyPtr *
280                                                sizeof(C2Qent_t));
281 /*
282 *      Loop through all nodes. Queue each node onto one of the cell2's. A node
283 *      may fall into more than one cell2 if the domain contains the whole
284 *      problem in one or more directions. In this case, queue onto the lowest
285 *      numbered image
286 */
287 #ifdef _OPENMP
288 #pragma omp parallel for
289 #endif
290     for (i = 0; i < home->newNodeKeyPtr; i++) {
291         int     baseX, baseY, baseZ;
292         int     c2BaseIndex, c2Index;
293         int     cell2X, cell2Y, cell2Z;
294         int     cell2MinXus, cell2MinYus, cell2MinZus;
295         int     cell2Xplus, cell2Yplus, cell2Zplus;
296         real8   x, y, z;
297         Node_t *node;
298
299         home->cell2QentArray[i].node = (Node_t *)NULL;
300         home->cell2QentArray[i].next = -1;
301
302         if ((node = home->nodeKeys[i]) == (Node_t *)NULL) {
303             continue;
304         }
305
306         x = node->x;
307         y = node->y;
308         z = node->z;
309
310 /*
311 *      Get the image of the point closest to the center of the
312 *      current domain.
313 */
314         PBCPOSITION(param, centerX, centerY, centerZ, &x, &y, &z);
315
316         cell2X = (int) floor((x - cell2MinX) / cell2SizeX);
317         cell2Y = (int) floor((y - cell2MinY) / cell2SizeY);
318         cell2Z = (int) floor((z - cell2MinZ) / cell2SizeZ);
319
320 /*
321 *      If a native node falls outside native cell2 area
322 *      force it into the nearest native cell2.
323 */
324         if (cell2X < minNativeCell2X) {
325             cell2X = minNativeCell2X;

```

```

326     }
327     if (cell12X > maxNativeCell12X) {
328         cell12X = maxNativeCell12X;
329     }
330
331     if (cell12Y < minNativeCell12Y) {
332         cell12Y = minNativeCell12Y;
333     }
334     if (cell12Y > maxNativeCell12Y) {
335         cell12Y = maxNativeCell12Y;
336     }
337
338     if (cell12Z < minNativeCell12Z) {
339         cell12Z = minNativeCell12Z;
340     }
341     if (cell12Z > maxNativeCell12Z) {
342         cell12Z = maxNativeCell12Z;
343     }
344
345     /*
346     * Find minimum cell12 containing node in the domain range.
347     * This minimum cell will be adjusted to the domain range, rather
348     * than problem range, of cell12 indices.
349     *
350     * X direction
351     */
352     cell12MinXus = cell12X - numBodyCell12X;
353     cell12Xplus = cell12X + numBodyCell12X;
354
355     if (cell12MinXus >= minDomCell12X)
356         baseX = cell12MinXus - minDomCell12X;
357     else if (cell12X >= minDomCell12X && cell12X <= maxDomCell12X)
358         baseX = cell12X - minDomCell12X;
359     else if (cell12Xplus <= maxDomCell12X)
360         baseX = cell12Xplus - minDomCell12X;
361     else
362         continue; /* node moved outside of ghost cells; ignore */
363
364     /*
365     * Y direction
366     */
367     cell12MinYus = cell12Y - numBodyCell12Y;
368     cell12Yplus = cell12Y + numBodyCell12Y;
369
370     if (cell12MinYus >= minDomCell12Y)
371         baseY = cell12MinYus - minDomCell12Y;
372     else if (cell12Y >= minDomCell12Y && cell12Y <= maxDomCell12Y)
373         baseY = cell12Y - minDomCell12Y;
374     else if (cell12Yplus <= maxDomCell12Y)
375         baseY = cell12Yplus - minDomCell12Y;
376     else
377         continue; /* node moved outside of ghost cells; ignore */
378
379     /*
380     * Z direction
381     */
382     cell12MinZus = cell12Z - numBodyCell12Z;
383     cell12Zplus = cell12Z + numBodyCell12Z;
384

```

```

385         if (cell2MinZus >= minDomCell2Z)
386             baseZ = cell2MinZus - minDomCell2Z;
387         else if (cell2Z >= minDomCell2Z && cell2Z <= maxDomCell2Z)
388             baseZ = cell2Z - minDomCell2Z;
389         else if (cell2Zplus <= maxDomCell2Z)
390             baseZ = cell2Zplus - minDomCell2Z;
391         else
392             continue; /* node moved outside of ghost cells; ignore */
393
394     /*
395     *         Make cell2 index relative to the domain
396     */
397     cell2X -= minDomCell2X;
398     cell2Y -= minDomCell2Y;
399     cell2Z -= minDomCell2Z;
400
401     /*
402     *         Queue the node on the minimum cell2 that it falls into, but save the
403     *         ghost cell2 index in the node, since this is where it starts its
404     *         neighbor search from.
405     *
406     *         Use a 'critical' section to prevent multiple threads from updating
407     *         the cell2 stuff simultaneously
408     */
409     c2BaseIndex = EncodeCell2Idx(home, baseX, baseY, baseZ);
410     c2Index = EncodeCell2Idx(home, cell2X, cell2Y, cell2Z);
411
412     node->cell2Idx = c2Index;
413
414     #ifdef _OPENMP
415     #pragma omp critical (CRIT_SORT_FOR_COLLISION)
416     #endif
417     {
418         node->cell2QentIdx = nextQent;
419         home->cell2QentArray[nextQent].node = node;
420         home->cell2QentArray[nextQent].next = home->cell2[c2BaseIndex];
421         home->cell2[c2BaseIndex] = nextQent;
422
423         nextQent++;
424
425     } /* end "omp critical" section */

```

Line-by-line explanation.

Lines 57–103

The first call computes a permanent fine-grid size. `maxSeparation=1.1*maxSeg+rann` ensures candidates that could interact lie in the same or a neighboring `cell2`.

Lines 104–130

The code uses an integral number of fine cells per ordinary cell and caps it at 20 in each direction. It records the global number of body cells for later periodic-image mapping.

Lines 265–280

A head array represents queues; a compact second array represents queue entries. The latter has one potential entry per local node key.

Lines 282–343

Each node is folded to the image nearest the current domain centre, mapped to `cell2`, and clamped

into the native range when needed. This keeps periodic images from creating unbounded search locations.

Lines 394–425

The node's starting `cell2Idx` is stored, then a critical section prepends it to its queue and stores the queue-entry index. A deleted node can later have its queue pointer cleared without breaking traversal of the remaining queue.

4 The shared topology mutation layer

4.1 Ownership is the primary parallel invariant

```
8  *           In order to handle topology changes in parallel,
9  *           it is necessary to define ownership of both
10 *           nodes and segments, and adhere to some strict
11 *           rules as to what a domain may do to nodes/segments
12 *           it does own and may not do to nodes/segments it
13 *           does not own. These rules are as follows:
14 *
15 *           Ownership of a segment permits:
16 *               1) Change the node ID to which the
17 *                  segment is connected
18 *               2) Change the burger's vector, force,
19 *                  glide plane and all other segment
20 *                  specific qualities.
21 *               3) Deletion of the segment
22 *
23 *           Ownership of a node permits:
24 *               1) Change nodal position, force, velocity
25 *                  and all other node specific qualities.
26 *               2) Deletion of the node if (and only if)
27 *                  it has no attached segments.
28 *
29 *           Included functions:
30 *               AllocNodeArrays()
31 *               BackupNode()
32 *               BCC_binary_junction_node()
33 *               CheckCollisionConditions()
34 *               CopyNodeArrays()
```

Line-by-line explanation.

Lines 8–15

The comments establish the reason for strict ownership: all domains can see ghosts, but only a deterministic owner may mutate a shared graph entity.

Lines 17–23

Owning a segment permits changing its endpoint tag, per-segment Burgers vector, force, and glide plane, and permits deletion.

Lines 25–29

Owning a node permits changing its local state, but deleting it is only allowed once all arms are gone. That prevents one domain from silently removing a node still referenced by another.

Lines 31–34

The implementation collects helpers that enforce these rules. Callers should not manually edit

neighbor arrays.

```
2208 *      Function:      DomainOwnsSeg
2209 *      Description:   Determines if the specified domain owns
2210 *                    the segment beginning in the current domain
2211 *                    and terminating at the domain indicated by
2212 *                    <endTag>.
2213 *
2214 *                    Note: Ownership of segments crossing
2215 *                    domain boundaries alternates on
2216 *                    and even/odd cycles. Additionally,
2217 *                    the ownership is dependent on the
2218 *                    class of topological operation being
2219 *                    considered (i.e. during remesh operations
2220 *                    the ownership is the exact reverse of
2221 *                    ownership during collision handling.)
2222 *
2223 *      Arguments:
2224 *          opClass      Class of topological operation during which
2225 *                      this function is being invoked. Valid
2226 *                      values are:
2227 *                          OPCLASS_SEPARATION
2228 *                          OPCLASS_COLLISION
2229 *                          OPCLASS_REMESH
2230 *          thisDomain   domain containing the first endpoint of
2231 *                      the segment in question.
2232 *          endTag       pointer to the tag of the second endpoint
2233 *                      of the segment.
2234 *
2235 *      Returns:  1 if <thisDomain> owns the segment
2236 *               0 in all other cases.
2237 *
2238 *-----*/
2239 int DomainOwnsSeg(Home_t *home, int opClass, int thisDomain, Tag_t *endTag)
2240 {
2241     int ownsSeg = 1;
2242
2243     /*
2244     *      If both endpoints are in the same domain, the domain owns
2245     *      the segment.
2246     */
2247     if (thisDomain == endTag->domainID) {
2248         return(1);
2249     }
2250
2251     /*
2252     *      For collision handling and node separations, ownership
2253     *      of segments crossing domain boundaries is the lower
2254     *      numbered domain on even numbered cycles and the higher
2255     *      numbered domain for odd numbered cycles.
2256     *
2257     *      For remesh operations, ownership rules are the opposite
2258     *      of those used for collision handling.
2259     */
2260     switch (opClass) {
2261     case OPCLASS_SEPARATION:
2262     case OPCLASS_COLLISION:
2263         if (home->cycle & 0x01)
2264             ownsSeg = (thisDomain > endTag->domainID);
```

```

2265         else
2266             ownsSeg = (thisDomain < endTag->domainID);
2267         break;
2268     case OPCLASS_REMESH:
2269         if (home->cycle & 0x01)
2270             ownsSeg = (thisDomain < endTag->domainID);
2271         else
2272             ownsSeg = (thisDomain > endTag->domainID);
2273         break;
2274     default:
2275         Fatal("Invalid opClass %d in DomainOwnsSeg()", opClass);
2276         break;
2277 }
2278
2279 return(ownsSeg);
2280 }

```

Ownership schedule.

Lines 2208–2251

DomainOwnsSeg() assigns local ownership immediately when both endpoints share the same domain.

Lines 2252–2267

For separation/collision across domains, the lower domain owns a segment on even cycles and the higher domain owns it on odd cycles.

Lines 2268–2273

Remesh intentionally uses the inverse cross-domain rule. Alternation distributes work and lets an ownership-blocked operation become eligible on a later cycle instead of making one rank permanently responsible.

Lines 2275–2280

Invalid operation classes are treated as programmer errors rather than being given an unsafe default owner.

4.2 Per-cycle exemptions and temporary mobility

```

327 int InitTopologyExemptions(Home_t *home)
328 {
329     int i, numNodes, numGhostNodes;
330
331     numNodes = home->newNodeKeyPtr;
332     numGhostNodes = home->ghostNodeCount;
333
334     #ifdef _OPENMP
335     #pragma omp parallel for
336     #endif
337     for (i = 0; i < numNodes; i++) {
338         Node_t *localNode;
339
340         if ((localNode = home->nodeKeys[i]) == (Node_t *)NULL) {
341             continue;
342         }
343
344         localNode->flags &= ~(NO_COLLISIONS | NO_MESH_COARSEN);
345
346         if (HAS_ANY_OF_CONSTRAINTS(localNode->constraint, CORNER_NODE) &&

```

```

347         (localNode->numNbrs != 2)) {
348             REMOVE_CONSTRAINTS(localNode->constraint, CORNER_NODE);
349         }
350     }
351
352     #ifdef _OPENMP
353     #pragma omp parallel for
354     #endif
355     for (i = 0; i < numGhostNodes; i++) {
356         Node_t *ghostNode;
357
358         ghostNode = home->ghostNodeList[i];
359         ghostNode->flags &= ~(NO_COLLISIONS | NO_MESH_COARSEN);
360
361         if (HAS_ANY_OF_CONSTRAINTS(ghostNode->constraint, CORNER_NODE) &&
362             (ghostNode->numNbrs != 2)) {
363             REMOVE_CONSTRAINTS(ghostNode->constraint, CORNER_NODE);
364         }
365     }
366
367     return(0);
368 }
369
370
371 /*-----
372 *
373 *   Function:      EvaluateMobility
374 *   Description:   This function provides a way to invoke the proper
375 *                 mobility functions for specific nodes rather than
376 *                 looping through all the nodes native to the current
377 *                 domain. This function is used solely to evaluate
378 *                 temporary nodes created to determine if a many-armed
379 *                 node need be split.
380 *
381 *   Arguments:
382 *   in:            nodeA    Pointer to first node to evaluate
383 *   in/out:        mobArgs  Structure containing additional
384 *                           parameters for conveying information
385 *                           to/from the mobility function.
386 *
387 *   Returns:
388 *               0 if there were no errors
389 *               1 if the mobility function failed
390 *
391 *-----*/
392 int EvaluateMobility(Home_t *home, Node_t *nodeA, MobArgs_t *mobArgs)
393 {
394     int    arm, mobError = 0;
395     Node_t *nbrNode;
396     Param_t *param;
397
398     param = home->param;
399
400     Matrix33_Zero(mobArgs->invDragMatrix);
401     mobArgs->numGlideConstraints = 0;
402
403     /*
404     *   It's possible for this function to be called for a node
405     *   that has been orphaned. If this is the case, don't
406     *   bother calling the mobility function first because the
407     *   orphaned node should be deleted soon and second because

```

```

406 *      one or more of the mobility functions fail for nodes
407 *      that have no arms.
408 */
409     if (nodeA->numNbrs == 0)
410     {
411         nodeA->vX = 0;
412         nodeA->vY = 0;
413         nodeA->vZ = 0;
414         return(mobError);
415     }
416
417 /*
418 *      One or more of the mobility functions require knowledge access
419 *      to data structures for all neighbors of the node being evaluated.
420 *      If this information is not available in this domain, leave the
421 *      nodal velocity as is and don't attempt to recalculate it.
422 */
423     for (arm = 0; arm < nodeA->numNbrs; arm++) {
424         nbrNode = GetNodeFromTag(home, nodeA->nbrTag[arm]);
425         if (nbrNode == (Node_t *)NULL) {
426             return(mobError);
427         }
428     }
429
430 /*
431 *      Now just evaluate the mobility for the node
432 */
433     mobError = param->mobilityFunc(home, nodeA, mobArgs);
434
435 /*
436 *      If glide planes are being enforced, get the glide constraint list
437 *      and apply it to the velocity. Note: glide constraints and velocity
438 *      are both in the lab frame at this point.
439 */
440     real8 nodeVel[3];
441     nodeVel[0] = nodeA->vX;
442     nodeVel[1] = nodeA->vY;
443     nodeVel[2] = nodeA->vZ;
444
445     real8 (*glideConstraints)[3];
446     int *numGlideConstraints;
447     numGlideConstraints = &mobArgs->numGlideConstraints;
448     glideConstraints = mobArgs->glideConstraints;
449
450     if (param->enforceGlidePlanes) {
451         GetGlideConstraintList(nodeA, numGlideConstraints, glideConstraints);
452         ApplyConstraintsToVelocity(*numGlideConstraints, glideConstraints, nodeVel);
453     }
454
455     /* FIX THIS ? This does not do anything: (SA)
456     real8 oldvTest[3];
457     oldvTest[0] = nodeA->oldvX;
458     oldvTest[1] = nodeA->oldvY;
459     oldvTest[2] = nodeA->oldvZ;
460
461     if (param->enforceGlidePlanes) {
462         ApplyConstraintsToVelocity(*numGlideConstraints,
463                                   glideConstraints, oldvTest);
464     }

```



```

465      */
466
467
468  /*
469  *      Cap the nodal velocity to the speed of sound if needed
470  */
471      if (param->shearVelocity > 0.0)
472          ApplyShearVelocityCap(home, nodeVel);
473
474      nodeA->vX = nodeVel[0];
475      nodeA->vY = nodeVel[1];
476      nodeA->vZ = nodeVel[2];
477
478  #ifdef _ARLFEM
479  /*
480  *      The velocity of any surface node along the negative surface
481  *      normal direction should be zero to prevent the node moving into
482  *      the box. Make a call to adjust the velocity accordingly.
483  */
484      AdjustSurfaceNodeVel(home, node);
485  #endif
486
487
488  #ifdef NAN_CHECK
489      if ((isnan(nodeVel[0]) || isinf(nodeVel[0])) ||
490          (isnan(nodeVel[1]) || isinf(nodeVel[1])) ||
491          (isnan(nodeVel[2]) || isinf(nodeVel[2]))) {
492          PrintNode(nodeA);
493          Fatal("Mobility %s: (%d, %d) velocity error.",
494              param->mobilityLaw, nodeA->myTag.domainID, nodeA->myTag.index);
495      }
496  #endif
497      return(mobError);
498  }

```

Line-by-line explanation.

Lines 327–368

The routine clears NO_COLLISIONS and NO_MESH_COARSEN on native and ghost nodes. It removes CORNER_NODE if a node is no longer degree two, preventing stale protection of a later unrelated configuration.

Lines 390–415

EvaluateMobility() zeroes an orphan's velocity instead of calling a mobility law that may require arms.

Lines 417–428

Mobility is not evaluated until every neighbor can be resolved locally. This protects material laws that examine neighbor data.

Lines 430–476

The selected mobility law is called, then glide-plane constraints and the optional shear-velocity cap are applied to the returned velocity.

Lines 488–498

An optional NaN/Inf check turns bad trial mobility into a source-local fatal error rather than allowing corrupt state to propagate.

4.3 SplitNode(): one primitive for separation and refinement

```
2465 int SplitNode(Home_t *home, int opClass, Node_t *node, real8 *pos1,
2466               real8 *pos2, real8 *vel1, real8 *vel2, int armCount,
2467               int *armList, int globalOp, Node_t **splitNode1,
2468               Node_t **splitNode2, int flags)
2469 {
2470     int i, j, k, tarm, *tarmList, *tarmList2;
2471     real8 bx, by, bz;
2472     real8 cx, cy, cz;
2473     real8 nx, ny, nz;
2474     real8 segPlaneX=0.0, segPlaneY=0.0, segPlaneZ=0.0;
2475     real8 eps=1.0e-12;
2476     real8 ftmp[3];
2477     real8 burg[3], vel[3], dirVec[3], glidePlane[3];
2478     Node_t *newNode, *nbrNode;
2479     int ownsSelectedArms;
2480
2481     int thisDomain = home->myDomain;
2482     Param_t *param = home->param;
2483
2484     *splitNode1 = (Node_t *)NULL;
2485     *splitNode2 = (Node_t *)NULL;
2486
2487     int switchedArms = 0;
2488     /*
2489     * Do a few sanity checks...
2490     */
2491     if (node->myTag.domainID != thisDomain) {
2492         return(SPLIT_FAILED);
2493     }
2494
2495     if (armCount < 1) {
2496         Fatal("SplitNode: attempted to create new node with 0 arms");
2497     }
2498
2499     /*
2500     * If we are only moving a single segment from the original node to the
2501     * new node, it (likely) means we're simply bisecting a segment during
2502     * MeshRefine(). In that case, preserve the segment's glide plane
2503     * so when we add a new segment between the original node and new node
2504     * we can have the new segment properly inherit the glide plane.
2505     */
2506     if (armCount == 1) {
2507         segPlaneX = node->nx[armList[0]];
2508         segPlaneY = node->ny[armList[0]];
2509         segPlaneZ = node->nz[armList[0]];
2510     }
2511
2512     /*
2513     * The list of arms MUST be sorted in descending order (by index) to
2514     * avoid issues related to arm renumbering as arms are removed from the
2515     * existing node. Make a temporary copy of the arm list and sort
2516     * it appropriately here, just in case the caller didn't.
2517     */
2518     tarmList = (int *)calloc(1, sizeof(int) * armCount);
2519
2520     for (i = 0; i < armCount; i++) tarmList[i] = armList[i];
```

```

2521
2522     for (i = 0; i < armCount-1; i++) {
2523         for (j = 0; j < armCount - (i+1); j++) {
2524             if (tarmList[j] < tarmList[j+1]) {
2525                 tarm = tarmList[j+1];
2526                 tarmList[j+1] = tarmList[j];
2527                 tarmList[j] = tarm;
2528             }
2529         }
2530     }
2531
2532     /*
2533     *   All arms to be split from the original and moved to the new node
2534     *   MUST be owned by the current domain.
2535     */
2536     ownsSelectedArms = 1;
2537
2538     for (i = 0; i < armCount; i++) {
2539         ownsSelectedArms &= DomainOwnsSeg(home, opClass, thisDomain,
2540                                           &node->nbrTag[tarmList[i]]);
2541     }
2542
2543     /*
2544     *   We cannot do a split that would move the endpoint of a segment
2545     *   not owned by the current domain, so if the current domain does
2546     *   not own all arms that are to be moved to the new node, check
2547     *   if the arms to remain with <node> are all owned by the current
2548     *   domain. If they are, we can still do the split by switching
2549     *   the selected segments with the unselected segments.
2550     */
2551     if (!ownsSelectedArms) {
2552
2553         j = 0;
2554         k = 0;
2555
2556         tarmList2 = (int *)calloc(1, (node->numNbrs-armCount)*sizeof(int));
2557
2558         for (i = node->numNbrs-1; i >= 0; i--) {
2559             if ((j >= armCount) || (tarmList[j] != i))
2560                 tarmList2[k++] = i;
2561             else
2562                 j++;
2563         }
2564
2565         free(tarmList);
2566         tarmList = tarmList2;
2567         tarmList2 = (int *)NULL;
2568         armCount = node->numNbrs - armCount;
2569
2570         ownsSelectedArms = 1;
2571
2572         for (i = 0; i < armCount; i++) {
2573             ownsSelectedArms &= DomainOwnsSeg(home, opClass, thisDomain,
2574                                               &node->nbrTag[tarmList[i]]);
2575         }
2576
2577         if (!ownsSelectedArms) {
2578             free(tarmList);
2579             return(SPLIT_FAILED);

```

```

2580         }
2581
2582         switchedArms = 1;
2583     }

```

Line-by-line explanation.

Lines 2465–2487

The signature receives desired positions/velocities, arm indices to move, an operation class, and output pointers for the two post-split nodes. Outputs are first nulled so every failure is unambiguous.

Lines 2489–2497

Only a native node may be split by this domain. Moving zero arms is a fatal programming error because it would create an invalid empty node.

Lines 2499–2510

A one-arm split normally bisects a segment during remesh. The source arm's glide plane is saved so both child segments inherit it instead of receiving a newly guessed plane.

Lines 2512–2530

Arm removal rennumbers the array. The temporary arm list is sorted in descending order, guaranteeing that deleting a later index never invalidates an earlier index still to be processed.

Lines 2532–2541

The operation is permitted only if this domain owns each selected segment. Endpoint relinking is a segment mutation.

Lines 2543–2583

If requested arms are not all owned, the routine tries the complementary partition. `switchedArms` preserves logical `splitNode1/splitNode2` semantics; if neither partition is locally owned, the split fails without graph mutation.

```

2585  /*
2586  *      Add a new node.  Warning:  GetNewNativeNode() pops a node off
2587  *      the queue, but does not free the arm related arrays.  It does
2588  *      this on the assumption that the number of arms previously
2589  *      assigned to the node is likely to be the same that will
2590  *      be reattached to the node and that the code will overwrite
2591  *      the existing arm data.  The following code, however, assumes
2592  *      the new node has zero arms initially, so we MUST free those arrays
2593  *      and reset the arm count!
2594  */
2595     newNode = GetNewNativeNode(home);
2596     FreeNodeArms(newNode);
2597
2598     newNode->native = 1;
2599     SET_CONSTRAINTS(newNode->constraint, UNCONSTRAINED);
2600
2601     newNode->oldvX = 0.0;
2602     newNode->oldvY = 0.0;
2603     newNode->oldvZ = 0.0;
2604
2605     #ifdef _ARLFEM
2606     /*
2607     *      If necessary copy the surface constraint and properties
2608     *      from the original node to the new node.
2609     */
2610     if ((flags & SPLIT_DUP_SURFACE_PROP) &&
2611         HAS_ANY_OF_CONSTRAINTS(node->constraint, SURFACE_NODE))

```

```

2612     {
2613         ADD_CONSTRAINTS(newNode->constraint, SURFACE_NODE);
2614
2615         VECTOR_COPY(newNode->surfaceNorm, node->surfaceNorm);
2616     }
2617 #endif
2618
2619 /*
2620  *   Reset the velocities and positions for the two nodes involved
2621  *   in the split. If we split off the arms selected by the caller
2622  *   splitNode1 will be mapped to the original node. Otherwise, it
2623  *   will be mapped to the new node.
2624  */
2625     if (switchedArms) {
2626         *splitNode1 = newNode;
2627         *splitNode2 = node;
2628     } else {
2629         *splitNode1 = node;
2630         *splitNode2 = newNode;
2631     }
2632
2633     (*splitNode1)->x = pos1[X];
2634     (*splitNode1)->y = pos1[Y];
2635     (*splitNode1)->z = pos1[Z];
2636
2637     (*splitNode1)->vX = vel1[X];
2638     (*splitNode1)->vY = vel1[Y];
2639     (*splitNode1)->vZ = vel1[Z];
2640
2641     (*splitNode2)->x = pos2[X];
2642     (*splitNode2)->y = pos2[Y];
2643     (*splitNode2)->z = pos2[Z];
2644
2645     (*splitNode2)->vX = vel2[X];
2646     (*splitNode2)->vY = vel2[Y];
2647     (*splitNode2)->vZ = vel2[Z];
2648
2649     newNode->oldx = node->oldx;
2650     newNode->oldy = node->oldy;
2651     newNode->oldz = node->oldz;
2652
2653 /*
2654  *   If the node being split was pinned in any dimensions, then the
2655  *   node left at position <pos1> must be pinned in the same manner
2656  *   and the other node must be explicitly unpinned.
2657  *
2658  *   NOTE: For now, a node that is pinned in ANY dimension is
2659  *   treated here as if it is pinned in ALL dimensions.
2660  */
2661     if (HAS_ANY_OF_CONSTRAINTS(node->constraint, PINNED_NODE)) {
2662         ADD_CONSTRAINTS((*splitNode1)->constraint,
2663             node->constraint & PINNED_NODE);
2664         REMOVE_CONSTRAINTS((*splitNode2)->constraint, PINNED_NODE);
2665     }
2666
2667 #ifdef _ARLFEM
2668 /*
2669  *   If the node being split is a surface node, then the
2670  *   node left at <pos1> must retain the surface properties.

```

```

2671  *      Additionally, if the surface properties of the original
2672  *      node are NOT to be duplicated, we must explicitly zero
2673  *      out surface constraint in the node at <pos2>.
2674  */
2675      if (HAS_ANY_OF_CONSTRAINTS(node->constraint, SURFACE_NODE)) {
2676
2677          ADD_CONSTRAINTS((*splitNode1)->constraint, SURFACE_NODE);
2678          VECTOR_COPY((*splitNode1)->surfaceNorm, node->surfaceNorm);
2679
2680          if ((flags & SPLIT_DUP_SURFACE_PROP) == 0) {
2681              REMOVE_CONSTRAINTS((*splitNode2)->constraint, SURFACE_NODE);
2682          }
2683      }
2684  #endif
2685
2686  /*
2687  *      If this is a global operation that remote domains will have to
2688  *      know about, add the operation to the list that will be sent
2689  *      to the neighboring domains. NOTE: Certain split operations
2690  *      may have altered the coordinates of <origNode>, so also send
2691  *      out updated coordinates to the remote domains.
2692  */
2693      if (globalOp) {
2694          real8 coord[3], velocity[3];
2695
2696          coord[0] = newNode->x;
2697          coord[1] = newNode->y;
2698          coord[2] = newNode->z;
2699
2700          velocity[0] = newNode->vX;
2701          velocity[1] = newNode->vY;
2702          velocity[2] = newNode->vZ;
2703
2704          AddOpSplitNode(home, &node->myTag, &newNode->myTag,
2705                        coord, velocity);
2706
2707          coord[0] = node->x;
2708          coord[1] = node->y;
2709          coord[2] = node->z;
2710
2711          AddOpResetCoord(home, &node->myTag, coord);
2712      }

```

Line-by-line explanation.

Lines 2585–2603

`GetNewNativeNode()` obtains a reusable record. Existing arm arrays are freed because this primitive requires a zero-arm starting state. The new node becomes native, unconstrained, and has zero old velocity.

Lines 2605–2617

In a free-surface build, the optional flag duplicates surface constraint/normal data. Ordinary bulk remesh does not use this branch.

Lines 2619–2647

The routine maps the original/new records to `splitNode1/splitNode2` according to whether it complemented the selected arm set. It writes caller-chosen positions and velocities and copies old

position to the new node.

Lines 2653–2684

A pinned original remains on the `pos1` side; surface properties similarly remain there unless explicitly duplicated. This prevents a generic split from accidentally freeing an anchored endpoint.

Lines 2686–2712

A global operation serializes a `REM_OP_SPLIT_NODE` record plus a coordinate reset for the original node. The reset is needed because a caller may have moved that node.

```
2714 /*
2715  *   For each arm of the original node in armList, get the neighbor node,
2716  *   change the nbrNode->origNode linkage to nbrNode->newNode, and
2717  *   remove the origNode->nbrNode linkage. Note: the DEL_SEG_NONE flag
2718  *   is passed to ChangeArmBurg() to prevent the function from treating
2719  *   the segments as annihilated since they are simply having an endpoint
2720  *   shifted.
2721  */
2722 for (i = 0; i < armCount; i++) {
2723
2724     nbrNode = GetNeighborNode(home, node, tarmList[i]);
2725
2726     if (nbrNode == (Node_t *)NULL) {
2727         Fatal("Neighbor not found at %s line %d\n",
2728             __FILE__, __LINE__);
2729     }
2730
2731     ftmp[X] = node->armfx[tarmList[i]];
2732     ftmp[Y] = node->armfy[tarmList[i]];
2733     ftmp[Z] = node->armfz[tarmList[i]];
2734
2735     SubtractSegForce(node, nbrNode);
2736
2737     GetBurgersVectorNormal(node, nbrNode, &bx, &by, &bz, &nx, &ny, &nz);
2738     ChangeConnection(home, nbrNode, &node->myTag,
2739         &newNode->myTag, globalOp);
2740     InsertArm(home, newNode, &node->nbrTag[tarmList[i]],
2741         bx, by, bz, nx, ny, nz, globalOp);
2742     ChangeArmBurg(home, node, &nbrNode->myTag, 0, 0, 0, 0, 0, 0,
2743         globalOp, DEL_SEG_NONE);
2744     ResetSegForces(home, newNode, &nbrNode->myTag, ftmp[X],
2745         ftmp[Y], ftmp[Z], globalOp);
2746 }
2747
2748 free(tarmList);
2749
2750
2751 /*
2752  *   We will probably be re-evaluating forces on this new node. In
2753  *   order to do that, the new node must be assigned to the proper
2754  *   cell.
2755  */
2756 AssignNodeToCell(home, newNode);
2757
2758 /*
2759  *   If necessary, create a link between the original node and new node
2760  *   in order to conserve burgers vector. First sum up the burgers vectors
2761  *   for the arms of the new node...
2762  */
```

```

2763 *      One exception, though.  When we're dealing with free surfaces, if
2764 *      *both* nodes are on the surface, we do not want to add a new segment
2765 *      between the nodes...
2766 */
2767     bx = 0.0;
2768     by = 0.0;
2769     bz = 0.0;
2770
2771     if (HAS_NONE_OF_CONSTRAINTS((*splitNode1)->constraint, SURFACE_NODE) ||
2772         HAS_NONE_OF_CONSTRAINTS((*splitNode2)->constraint, SURFACE_NODE)) {
2773         for (i = 0; i < (*splitNode2)->numNbrs; i++) {
2774             bx += (*splitNode2)->burgX[i];
2775             by += (*splitNode2)->burgY[i];
2776             bz += (*splitNode2)->burgZ[i];
2777         }
2778     }
2779
2780 /*
2781 *      Under two circumstances we need to add an additional link between
2782 *      the two nodes:
2783 *      1) if the sum of burgers vectors on the new node is non-zero
2784 *      2) if the sum of burgers vectors on the new node is zero BUT
2785 *         the node is 'pinned'; Scenario is this:
2786 *         - the single arm of a pinned node is being split
2787 *         - the 'pinned' node is repositioned and unpinned
2788 *         - the new node is positioned at the location of
2789 *           the original node and pinned.
2790 *         - the original node maintains its single connection
2791 *
2792 *      At this point, the new node is pinned, but has no arms
2793 *      hence the sum of its burgers vectors is zero, so no
2794 *      link would be created between the original node and the
2795 *      new node. This results in the pinned node being orphaned
2796 *      and deleted, and the other node is no longer attached to
2797 *      a pinned node and this is bad.
2798 */
2799     if (fabs(bx*bx + by*by + bz*bz) > 0.001) {
2800
2801 /*
2802 *      If this was just a call to bisect a segment,
2803 *      the glide plane should just be inherited from
2804 *      the bisected segment. Otherwise, we need to
2805 *      calculate the glide plane of the new link.
2806 */
2807         if (((*splitNode1)->numNbrs == 1) ||
2808             ((*splitNode2)->numNbrs == 1)) {
2809             cx = segPlaneX;
2810             cy = segPlaneY;
2811             cz = segPlaneZ;
2812         } else {
2813             dirVec[X] = (*splitNode1)->x - (*splitNode2)->x;
2814             dirVec[Y] = (*splitNode1)->y - (*splitNode2)->y;
2815             dirVec[Z] = (*splitNode1)->z - (*splitNode2)->z;
2816
2817             ZImage(param, &dirVec[X], &dirVec[Y], &dirVec[Z]);
2818
2819             burg[X] = bx;
2820             burg[Y] = by;
2821             burg[Z] = bz;

```



```

2822
2823     NormalizeVec(dirVec);
2824     NormalizeVec(burg);
2825
2826     FindPreciseGlidePlane(home, burg, dirVec, glidePlane,
2827                           home->param->allowFuzzyGlidePlanes);
2828
2829     if (DotProduct(glidePlane,glidePlane) < eps) {
2830         vel[X] = ((*splitNode1)->vX + (*splitNode2)->vX) * 0.5;
2831         vel[Y] = ((*splitNode1)->vY + (*splitNode2)->vY) * 0.5;
2832         vel[Z] = ((*splitNode1)->vZ + (*splitNode2)->vZ) * 0.5;
2833
2834         NormalizeVec(vel);
2835         FindPreciseGlidePlane(home, vel, dirVec, glidePlane,
2836                               home->param->allowFuzzyGlidePlanes);
2837
2838         if (DotProduct(glidePlane,glidePlane) < eps) {
2839             PickScrewGlidePlane(home, burg, glidePlane);
2840         }
2841     }
2842
2843     NormalizeVec(glidePlane);
2844
2845     cx = glidePlane[X];
2846     cy = glidePlane[Y];
2847     cz = glidePlane[Z];
2848 }
2849
2850 /*
2851  *      Add links from the original node to the new node
2852  *      and vice-versa. Forces on this new segment will be
2853  *      added outside this function.
2854  */
2855     InsertArm(home, (*splitNode1), &(*splitNode2)->myTag, bx, by, bz, cx, cy, cz,
2856               globalOp);
2857     InsertArm(home, (*splitNode2), &(*splitNode1)->myTag, -bx, -by, -bz, cx, cy, cz,
2858               globalOp);
2859
2860     return(SPLIT_SUCCESS);

```

Line-by-line explanation.

Lines 2714–2746

For every selected arm, the code gets the neighbor, saves endpoint force, subtracts old contribution, relinks the neighbor to the new node, inserts the matching new arm, removes the old arm, and restores force. Helper calls keep both endpoints and the operation log consistent.

Lines 2748–2756

The temporary arm list is released and the new node is assigned to a spatial cell before later force work can find it.

Lines 2758–2778

The routine sums Burgers vectors attached to the new side. Unless both nodes are surface nodes, a nonzero sum requires a connecting segment for Burgers conservation.

Lines 2780–2799

The long comment explains a pinned-node corner case. The nonzero Burgers-sum test is the usual condition for adding the link.

Lines 2801–2848

A refinement split copies the original plane. General separation computes a plane from line direction and Burgers vector, uses velocity as a fallback, then chooses an allowed screw plane if undefined.

Lines 2850–2860

Opposite Burgers vectors are inserted at the two endpoints of the connecting segment. The routine leaves its force to the caller, which knows whether a saved or estimated force is appropriate.

4.4 MergeNode(): common coarsen and collision primitive

```
2879 int MergeNodePreCondition(Home_t *home, int opClass, Node_t *node1, Node_t *node2)
2880 {
2881     int node1Deletable, node2Deletable, thisDomain, armID;
2882     thisDomain = home->myDomain;
2883     node1Deletable = (node1->myTag.domainID == thisDomain);
2884     node1Deletable &= HAS_NONE_OF_CONSTRAINTS(node1->constraint,
2885                                                PINNED_NODE);
2886     node1Deletable &= (HAS_NO_CONSTRAINTS(node1->constraint) ||
2887                      HAS_ANY_OF_CONSTRAINTS(node2->constraint,
2888                                              SURFACE_NODE));
2889     node1Deletable &= ((node1->flags & NO_COLLISIONS) == 0);
2890     node1Deletable &= ((node1->flags & NO_MESH_COARSEN) == 0);
2891
2892     for (armID = 0; armID < node1->numNbrs; armID++) {
2893         node1Deletable &= DomainOwnsSeg(home, opClass, thisDomain,
2894                                         &node1->nbrTag[armID]);
2895     }
2896
2897     /*
2898     * If we can delete node1, use node2 as the target, but if node1
2899     * cannot be removed, use node2 if we can. If node2 cannot
2900     * be deleted either, return a failure to the caller.
2901     */
2902     if (node1Deletable) {
2903         //targetNode = node2;
2904         //deadNode = node1;
2905         return 0;
2906     } else {
2907         node2Deletable = (node2->myTag.domainID == thisDomain);
2908         node2Deletable &= HAS_NONE_OF_CONSTRAINTS(node2->constraint,
2909                                                    PINNED_NODE);
2910         node2Deletable &= (HAS_NO_CONSTRAINTS(node2->constraint) ||
2911                          HAS_ANY_OF_CONSTRAINTS(node1->constraint,
2912                                                  SURFACE_NODE));
2913         node2Deletable &= ((node2->flags & NO_COLLISIONS) == 0);
2914         node2Deletable &= ((node2->flags & NO_MESH_COARSEN) == 0);
2915
2916         for (armID = 0; armID < node2->numNbrs; armID++) {
2917             node2Deletable &= DomainOwnsSeg(home, opClass, thisDomain,
2918                                             &node2->nbrTag[armID]);
2919         }
2920
2921         if (!node2Deletable) {
2922             return -1;
```

```

2923     }
2924
2925     //targetNode = node1;
2926     //deadNode = node2;
2927     return 1;
2928 }

2976 void MergeNode(Home_t *home, int opClass, Node_t *node1, Node_t *node2,
2977               real8 *position, Node_t **mergedNode, int *status, int globalOp)
2978 {
2979     int i;
2980     int armID, armID1, armID2, thisDomain;
2981     int node1Deletable, node2Deletable;
2982     int targetIsLocal, nbrIsLocal, targetOwnsSeg;
2983     real8 bx, by, bz;
2984     real8 nx, ny, nz;
2985     real8 ftmp[3], oldPos[3];
2986     Node_t *nbrNode, *targetNode, *deadNode;
2987     Tag_t tmpTag, *nbr1Tag, *nbr2Tag;
2988
2989     thisDomain = home->myDomain;
2990
2991     targetNode = node1;
2992     deadNode = node1;
2993     *mergedNode = (Node_t *)NULL;
2994     *status = MERGE_NOT_PERMITTED;
2995
2996     /*
2997     * Check if we are permitted to delete node1. In order
2998     * for a node to be deletable, it must satisfy the following:
2999     *
3000     * 1) Node may not be a 'pinned' node
3001     * 2) Node must be owned by the current domain
3002     * 3) All arms of the node must be owned by the current domain
3003     * 4) Node may not be exempt from (additional) collisions this cycle
3004     * 5) Node may not be exempt from (additional) remeshes this cycle
3005     * 6) If node is a surface node, *both* nodes must be on the surface
3006     * for it to be deletable. (May need to add restrictions
3007     * that surface nodes be on the same surface)
3008     *
3009     * NOTE: For now, a node that is pinned in ANY dimension is treated
3010     * here as if it is pinned in ALL dimensions.
3011     */
3012     node1Deletable = (node1->myTag.domainID == thisDomain);
3013     node1Deletable &= HAS_NONE_OF_CONSTRAINTS(node1->constraint,
3014                                               PINNED_NODE);
3015     node1Deletable &= (HAS_NO_CONSTRAINTS(node1->constraint) ||
3016                       HAS_ANY_OF_CONSTRAINTS(node2->constraint,
3017                                               SURFACE_NODE));
3018     node1Deletable &= ((node1->flags & NO_COLLISIONS) == 0);
3019     node1Deletable &= ((node1->flags & NO_MESH_COARSEN) == 0);
3020
3021     for (armID = 0; armID < node1->numNbrs; armID++) {
3022         node1Deletable &= DomainOwnsSeg(home, opClass, thisDomain,
3023                                         &node1->nbrTag[armID]);
3024     }
3025
3026     /*

```

```

3027 *      If we can delete node1, use node2 as the target, but if node1
3028 *      cannot be removed, use node2 if we can. If node2 cannot
3029 *      be deleted either, return a failure to the caller.
3030 */
3031 if (node1Deletable) {
3032     targetNode = node2;
3033     deadNode = node1;
3034 } else {
3035     node2Deletable = (node2->myTag.domainID == thisDomain);
3036     node2Deletable &= HAS_NONE_OF_CONSTRAINTS(node2->constraint,
3037                                                PINNED_NODE);
3038     node2Deletable &= (HAS_NO_CONSTRAINTS(node2->constraint) ||
3039                       HAS_ANY_OF_CONSTRAINTS(node1->constraint,
3040                                                SURFACE_NODE));
3041     node2Deletable &= ((node2->flags & NO_COLLISIONS) == 0);
3042     node2Deletable &= ((node2->flags & NO_MESH_COARSEN) == 0);
3043
3044     for (armID = 0; armID < node2->numNbrs; armID++) {
3045         node2Deletable &= DomainOwnsSeg(home, opClass, thisDomain,
3046                                         &node2->nbrTag[armID]);
3047     }
3048
3049     if (!node2Deletable) {
3050         *status = MERGE_NOT_PERMITTED;
3051         *mergedNode = (Node_t *)NULL;
3052         return;
3053     }
3054
3055     targetNode = node1;
3056     deadNode = node2;
3057 }
3058
3059 /*
3060 *      Coarsening a node out *may* leave a double connection. That
3061 *      is only permitted if the current domain would own both
3062 *      links in the resulting double link and hence be able to
3063 *      collapse/annihilate the links.
3064 */
3065 targetIsLocal = (targetNode->myTag.domainID == thisDomain);
3066
3067 for (armID1 = 0; armID1 < targetNode->numNbrs; armID1++) {
3068     nbr1Tag = &targetNode->nbrTag[armID1];
3069     nbrIsLocal = (nbr1Tag->domainID == thisDomain);
3070     for (armID2 = 0; armID2 < deadNode->numNbrs; armID2++) {
3071         nbr2Tag = &deadNode->nbrTag[armID2];
3072
3073         /*
3074          *      If the target node and dead node both have links to a
3075          *      3rd node, check the ownership of the link between the
3076          *      target node and the 3rd node...
3077          */
3078         if ((nbr1Tag->domainID == nbr2Tag->domainID) &&
3079             (nbr1Tag->index == nbr2Tag->index )) {
3080
3081             if (targetIsLocal && nbrIsLocal) {
3082                 continue;
3083             }
3084
3085             targetOwnsSeg = DomainOwnsSeg(home, opClass,

```

```

3086                                     targetNode->myTag.domainID,
3087                                     nbr1Tag);
3088
3089         if ((targetIsLocal && targetOwnsSeg) ||
3090             (nbrIsLocal && !targetOwnsSeg)) {
3091             continue;
3092         }
3093     /*
3094     *           In the case of a simple triangle with a node in
3095     *           each domain, in theory we'd be safe coarsening out
3096     *           the node during remesh since no other operations
3097     *           would affect the remaining link... we might want
3098     *           to add in the following exception.
3099     */
3100     #if 0
3101         if ((opClass == OPCLASS_REMESH) &&
3102             (targetNode->numNbrs == 2) &&
3103             (nbrNode->numNbrs == 2)) {
3104             continue;
3105         }
3106     #endif
3107
3108     /*
3109     *           Link is remotely owned, so we cannot risk creating
3110     *           a double link here...
3111     */
3112     *mergedNode = (Node_t *)NULL;
3113     *status = MERGE_DOUBLE_LINK;
3114     return;
3115     }
3116     }
3117 }

```

Line-by-line explanation.

Lines 2879–2929

MergeNodePreCondition() is a fast version of the deletion eligibility test. It returns which supplied endpoint, if either, is a legal node to delete.

Lines 2976–2995

MergeNode() starts pessimistically: it returns no merged node and MERGE_NOT_PERMITTED until a legal deletion target is found.

Lines 2996–3024

node1Deletable requires local ownership, no pin, compatible surface condition, no per-cycle collision/coarsen exemption, and local ownership of every incident segment.

Lines 3027–3057

If node 1 cannot be deleted, the same test is applied to node 2. At most one legal choice is needed; if neither is deletable, nothing is changed.

Lines 3059–3117

A prospective double link is checked before mutation. If cleanup of that double link would be owned remotely, the merge is rejected. Never create a graph state that the current domain cannot repair.

```

3119 /*
3120 *           We've made it past all the checks that would have prevented the
3121 *           merge operation from being performed. The target and dead nodes
3122 *           have been selected, so if the target node is local to this

```

```

3123 *      domain, go ahead and reposition it.  If we can't reposition the
3124 *      node, the merge will still be done, but be sure to set the
3125 *      status flag for the caller indicating the node has not been
3126 *      repositioned.
3127 */
3128 *mergedNode = targetNode;
3129 *status = MERGE_SUCCESS;
3130
3131 if ((targetNode->myTag.domainID == thisDomain) &&
3132     (HAS_NO_CONSTRAINTS(targetNode->constraint) ||
3133      (HAS_ANY_OF_CONSTRAINTS(targetNode->constraint, SURFACE_NODE) &&
3134       HAS_ANY_OF_CONSTRAINTS(deadNode->constraint, SURFACE_NODE)))) {
3135
3136 #ifdef FIX_PLASTIC_STRAIN
3137 /*
3138 *      Update plastic strain as target node is being moved
3139 */
3140     if (opClass == OPCLASS_COLLISION || opClass == OPCLASS_REMESH) {
3141
3142         oldPos[X] = targetNode->x;
3143         oldPos[Y] = targetNode->y;
3144         oldPos[Z] = targetNode->z;
3145         UpdateNodePlasticStrain(home, targetNode, oldPos, position);
3146
3147         oldPos[X] = deadNode->x;
3148         oldPos[Y] = deadNode->y;
3149         oldPos[Z] = deadNode->z;
3150         UpdateNodePlasticStrain(home, deadNode, oldPos, position);
3151     }
3152 #endif
3153
3154         targetNode->x = position[X];
3155         targetNode->y = position[Y];
3156         targetNode->z = position[Z];
3157 /*
3158 *      If we're updating the location of the target node, we may
3159 *      have to communicate the new position to remote domains.
3160 */
3161     if (globalOp) {
3162         real8 coord[3];
3163
3164         coord[0] = targetNode->x;
3165         coord[1] = targetNode->y;
3166         coord[2] = targetNode->z;
3167
3168         AddOpResetCoord(home, &targetNode->myTag, coord);
3169     }
3170 } else {
3171     *status |= MERGE_NO_REPOSITION;
3172
3173 #ifdef FIX_PLASTIC_STRAIN
3174 /*
3175 *      Update plastic strain as dead node is implicitly
3176 *      being moved to the position of the target node
3177 */
3178     if (opClass == OPCLASS_COLLISION || opClass == OPCLASS_REMESH) {
3179
3180         oldPos[X] = deadNode->x;
3181         oldPos[Y] = deadNode->y;

```

```

3182         oldPos[Z] = deadNode->z;
3183         UpdateNodePlasticStrain(home, deadNode, oldPos, position);
3184     }
3185 #endif
3186 }
3187
3188 /*
3189  *   If there are any connections from the target/destination node
3190  *   to the dead node, use ChangeArmBurg() to get rid of those
3191  *   connections, then move all connections from the dead node to
3192  *   the target node, and add a new connection from the target node
3193  *   to each of the dead node's neighbors.
3194  */
3195 ChangeArmBurg(home, targetNode, &deadNode->myTag, 0.0, 0.0, 0.0,
3196              0.0, 0.0, 0.0, globalOp, DEL_SEG_NONE);
3197 ChangeArmBurg(home, deadNode, &targetNode->myTag, 0.0, 0.0, 0.0,
3198              0.0, 0.0, 0.0, globalOp, DEL_SEG_NONE);
3199
3200 /*
3201  *   Move all connections from the dead node to the target node
3202  *   and add a new connection from the target node to each of the
3203  *   dead node's neighbors.
3204  */
3205 for (i = (deadNode->numNbrs - 1); i >= 0; i--) {
3206     tmpTag.domainID = deadNode->nbrTag[i].domainID;
3207     tmpTag.index     = deadNode->nbrTag[i].index;
3208     nbrNode = GetNeighborNode(home, deadNode, i);
3209
3210     if (nbrNode == (Node_t *)NULL) {
3211         Fatal("Neighbor not found at %s line %d\n",
3212             __FILE__, __LINE__);
3213     }
3214
3215     ftmp[X] = deadNode->armfx[i];
3216     ftmp[Y] = deadNode->armfy[i];
3217     ftmp[Z] = deadNode->armfz[i];
3218
3219     GetBurgersVectorNormal(deadNode, nbrNode, &bx, &by, &bz, &nx, &ny, &nz);
3220     ChangeConnection(home, nbrNode, &deadNode->myTag,
3221                     &targetNode->myTag, globalOp);
3222     ChangeArmBurg(home, deadNode, &nbrNode->myTag, 0.0, 0.0, 0.0,
3223                 0.0, 0.0, 0.0, globalOp, DEL_SEG_NONE);
3224     InsertArm(home, targetNode, &tmpTag, bx, by, bz,
3225             nx, ny, nz, globalOp);
3226
3227     ResetSegForces(home, targetNode, &tmpTag, ftmp[X], ftmp[Y],
3228                 ftmp[Z], globalOp);
3229 }
3230
3231 /*
3232  *   Any constraints that may have been associated with the node
3233  *   being removed should be transferred to the target node.
3234  */
3235 ADD_CONSTRAINTS(targetNode->constraint, deadNode->constraint);
3236
3237 /*
3238  *   The <deadNode> should have no more arms, so we can safely
3239  *   remove it now.
3240  */

```

```

3241     RemoveNode(home, deadNode, globalOp);
3242
3243     /*
3244     *     Check for multiple links between the target node and any other
3245     *     single node. If found, the burgers vectors for the multiple links
3246     *     will either cancel each other out in which case both links are
3247     *     removed, or the links get combined into a single link.
3248     */
3249     if (RemoveDoubleLinks(home, targetNode, globalOp)) {
3250         *status |= MERGE_NODE_ORPHANED;
3251     }
3252
3253     /*
3254     *     It's possible that the target node no longer has any arms. If
3255     *     the neighbor is local, go ahead and remove it, otherwise, just
3256     *     set a status flag letting the caller know a remote node has been
3257     *     orphaned.
3258     */
3259     if (targetNode->numNbrs == 0) {
3260         if (targetNode->myTag.domainID == thisDomain) {
3261             RemoveNode(home, targetNode, globalOp);
3262             *mergedNode = (Node_t *)NULL;
3263         } else {
3264             *status |= MERGE_NODE_ORPHANED;
3265         }
3266     }
3267
3268     return;

```

Line-by-line explanation.

Lines 3119–3135

The chosen survivor is returned and marked successful. If it is local and position-compatible, it is moved to the requested merge position.

Lines 3136–3151

With plastic-strain correction enabled, collision or remesh merging accounts for swept area from both nodes before coordinates change.

Lines 3154–3186

The target coordinate is written and, for global events, logged for remote replay. If constrained or remote, the merge still proceeds but reports MERGE_NO_REPOSITION.

Lines 3188–3229

Any direct target/dead link is deleted first. Each remaining dead-node arm is transferred: the neighbor is relinked, the dead endpoint arm is deleted, the survivor receives the same Burgers vector/plane, and its force is restored.

Lines 3231–3241

All dead-node constraints are inherited by the survivor, then the now-arm-free dead node is removed.

Lines 3243–3268

Duplicate links are combined or annihilated. If that leaves an orphan, a local orphan is deleted immediately and a remote orphan is reported for later cleanup.

4.5 Duplicate links and orphan nodes

```

91 void RemoveOrphanedNodes(Home_t *home)

```



```

92 {
93     int i;
94     Node_t *node;
95
96     for (i = 0; i < home->newNodeKeyPtr; i++) {
97         node = home->nodeKeys[i];
98         if (node == (Node_t *)NULL) continue;
99         if (node->numNbrs == 0) {
100             #if 0
101                 printf("WARNING: Removing orphan node (%d,%d)\n",
102                     node->myTag.domainID, node->myTag.index);
103             #endif
104             RemoveNode(home, node, 0);
105         }
106     }
107
108     return;
109 }

```

```

523 int RemoveDoubleLinks(Home_t *home, Node_t *node, int globalOp)
524 {
525     int i, j, nbrIndex, nbrDomain, thisDomain;
526     int arm, needGlide;
527     int nodeOrphaned = 0;
528     real8 bx, by, bz;
529     real8 nx, ny, nz;
530     real8 bsum[3];
531     Node_t *nbrNode;
532
533     thisDomain = home->myDomain;
534
535     for (i = 0; i < (node->numNbrs - 1); i++) {
536
537         nbrDomain = node->nbrTag[i].domainID;
538         nbrIndex = node->nbrTag[i].index;
539
540         for (j = i + 1; j < node->numNbrs; j++) {
541
542             if ((node->nbrTag[j].domainID == nbrDomain) &&
543                 (node->nbrTag[j].index == nbrIndex)) {
544
545                 /*
546                  * Found redundant arms linking 2 nodes. If
547                  * the burgers vectors cancel, remove both links
548                  * otherwise, reset the burgers vector for the
549                  * first arm and remove the second.
550                  */
551
552                 bx = node->burgX[i]+node->burgX[j];
553                 by = node->burgY[i]+node->burgY[j];
554                 bz = node->burgZ[i]+node->burgZ[j];
555
556                 /*
557                  * Just a safety check to prevent tiny non-zero
558                  * components of burgers vector due to machine
559                  * precision or round-off issues.
560
561                 if (fabs(bx) < 1.0e-06) bx = 0.0;
562                 if (fabs(by) < 1.0e-06) by = 0.0;

```

```

562         if (fabs(bz) < 1.0e-06) bz = 0.0;
563
564         if (fabs((bx * bx) + (by * by) + (bz * bz)) < 1.0e-6) {
565
566             nbrNode = GetNeighborNode(home, node, j);
567
568             if (nbrNode == (Node_t *)NULL) {
569                 Fatal("Neighbor not found at %s line %d\n",
570                     __FILE__, __LINE__);
571             }
572
573             ChangeArmBurg(home, node, &nbrNode->myTag,
574                 0.0, 0.0, 0.0,
575                 0.0, 0.0, 0.0,
576                 globalOp, DEL_SEG_HALF);
577
578             ChangeArmBurg(home, nbrNode, &node->myTag,
579                 0.0, 0.0, 0.0,
580                 0.0, 0.0, 0.0,
581                 globalOp, DEL_SEG_HALF);
582
583             /*
584             *
585             * It's possible the neighbor node no longer has any
586             * arms. If the neighbor is local, go ahead and remove
587             * it, otherwise, just set a status flag letting the
588             * caller know a remote node has been orphaned.
589             */
590
591             if (nbrNode->numNbrs == 0) {
592                 if (nbrNode->myTag.domainID == thisDomain) {
593                     RemoveNode(home, nbrNode, globalOp);
594                 } else {
595                     nodeOrphaned = 1;
596                 }
597             }
598
599             /*
600             * We removed multiple links, so set i back by one so
601             * we don't skip over an arm in the outer loop.
602             */
603
604             i--;
605         } else {
606
607             /*
608             *
609             * The new segment resulting from this merge has to have
610             * its glide plane recalculated, but that is handled
611             * within ChangeArmBurg().
612             */
613
614             nx = 0.0;
615             ny = 0.0;
616             nz = 0.0;
617
618             nbrNode = GetNeighborNode(home, node, j);
619
620             if (nbrNode == (Node_t *)NULL) {
621                 Fatal("Neighbor not found at %s line %d\n",
622                     __FILE__, __LINE__);
623             }
624
625             /*
626             * We don't yet know what glide plane to use for this

```

```

621 *          new segment.  If both the old segments had the same
622 *          glide plane, use the same one.  If they had different
623 *          planes, look for another segment on either end-node
624 *          which has the same burgers vector as the new segment
625 *          and inherit the plane from that segment if possible.
626 *          Otherwise, glide plane will be calculated in
627 *          ChangeArmBurg().
628 */
629
630         needGlide = 1;
631
632         if (fabs((node->nx[i] * node->nx[j]) +
633                 (node->ny[i] * node->ny[j]) +
634                 (node->nz[i] * node->nz[j])) > 0.999) {
635             nx = node->nx[i];
636             ny = node->ny[i];
637             nz = node->nz[i];
638             needGlide = 0;
639         }
640
641         if (needGlide) {
642             for (arm = 0; arm < node->numNbrs; arm++) {
643                 bsum[X] = node->burgX[arm] + bx;
644                 bsum[Y] = node->burgY[arm] + by;
645                 bsum[Z] = node->burgZ[arm] + bz;
646                 if (DotProduct(bsum, bsum) < 1.0e-4) {
647                     nx = node->nx[arm];
648                     ny = node->ny[arm];
649                     nz = node->nz[arm];
650                     needGlide = 0;
651                     break;
652                 }
653             }
654
655             if (needGlide) {
656                 for (arm = 0; arm < nbrNode->numNbrs; arm++) {
657                     bsum[X] = nbrNode->burgX[arm] + bx;
658                     bsum[Y] = nbrNode->burgY[arm] + by;
659                     bsum[Z] = nbrNode->burgZ[arm] + bz;
660                     if (DotProduct(bsum, bsum) < 1.0e-4) {
661                         nx = nbrNode->nx[arm];
662                         ny = nbrNode->ny[arm];
663                         nz = nbrNode->nz[arm];
664                         needGlide = 0;
665                         break;
666                     }
667                 }
668             }
669
670             ChangeArmBurg(home, node, &nbrNode->myTag,
671                          bx, by, bz, nx, ny, nz,
672                          globalOp, DEL_SEG_HALF);
673
674             /*
675             *          It can happen that the values of (nx, ny, nz) are set to zero and
676             *          that ChangeArmBurg chose a plane at random for the node-nbr segment.
677             *          To ensure that the plane is the same for nbr - node, we recover it.
678             *          Bug found by R. Sills
679
680             int k;

```

```

680         for (k=0; k < node->numNbrs; k++) {
681
682             if ((node->nbrTag[k].domainID != nbrNode->myTag.domainID) ||
683                 (node->nbrTag[k].index != nbrNode->myTag.index)) {
684                 continue;
685             }
686
687             nx = node->nx[k];
688             ny = node->ny[k];
689             nz = node->nz[k];
690             break;
691         }
692
693         ChangeArmBurg(home, nbrNode, &node->myTag,
694                     -bx, -by, -bz, nx, ny, nz,
695                     globalOp, DEL_SEG_HALF);
696     }
697     /*
698     *      Redundant links removed (or merged), so break
699     *      out of the inner loop and continue the outer loop.
700     */
701     break;
702 } /* if redundant links found */
703 }
704 }
705 }
706
707     return(nodeOrphaned);
708 }

```

Line-by-line explanation.

Lines 91–109

RemoveOrphanedNodes() scans local node keys and deletes a node with zero arms. It deliberately does not message other domains; the subsequent ghost exchange publishes the corrected list.

Lines 523–572

RemoveDoubleLinks() finds repeated neighbor tags on one local node. It records relevant arm data before deleting or combining them.

Lines 573–593

If the Burgers vectors cancel, both duplicate arms are removed. This is topological annihilation of redundant opposite segments.

Lines 603–695

If they do not cancel, the vectors are combined and a valid glide plane is retained or recalculated. The routine updates forces and the remote-operation stream as requested.

Lines 696–708

A resulting local orphan is deleted. The return status lets callers record that an operation made an orphan, even when it is remote.

5 Topology-event selection

5.1 Serial multi-node splitting

```

1354 void SplitMultiNodes(Home_t *home)

```

```

1355 {
1356     int    i, j, k, q, nbrs, numSets;
1357     int    tmpArm;
1358     int    setIndex;
1359     int    armIndex, armIndex2;
1360     int    totalSets, colliding, origNumNbrs;
1361     int    splitStatus, mergeStatus;
1362     int    globalOp = 1;
1363     int    bkupNodeCount;
1364     int    segIndex, mobError, skipSplits = 0;
1365     int    *armList, *armList2;
1366     int    **armSets;
1367     int    localSplitVals[4], globalSplitVals[4];
1368     real8  powerMax, powerTest;
1369     real8  vd1, vd2, invvNorm;
1370     real8  dirx, diry, dirz;
1371     real8  minDist, splitDist, shortSeg;
1372     real8  vDiffx, vDiffy, vDiffz;
1373     real8  eps = 1.0e-12;
1374     real8  pos1[3], pos2[3], vel1[3], vel2[3], origPos[3], origVel[3];
1375     real8  vNoise;
1376     Node_t *node, *nbrNode, *origNode;
1377     Node_t *tmpNode, *newNode;
1378     Node_t *splitNode1, *splitNode2, *mergedNode;
1379     Node_t *bkupNodeList;
1380     Param_t *param;
1381     SegForce_t *segForces = (SegForce_t *)NULL;
1382     MobArgs_t  mobArgs1, mobArgs2;
1383
1384
1385     /*
1386     *    Only do multinode splits on cycles that are multiples
1387     *    of the <splitMultiNodeFreq> control file parameter.
1388     */
1389     if (home->cycle % home->param->splitMultiNodeFreq) {
1390         skipSplits = 1;
1391     }
1392
1393     TimerStart(home, SPLIT_MULTI_NODES);
1394
1395     param      = home->param;
1396     minDist    = param->rann * 2.0 + eps;
1397     shortSeg   = MIN(5.0, param->minSeg * 0.1);
1398     vNoise     = param->splitMultiNodeAlpha * param->rTol / param->deltaTT;
1399     splitDist  = minDist;
1400
1401     armList    = (int *)NULL;
1402     armList2   = (int *)NULL;
1403
1404     memset(localSplitVals, 0, sizeof(localSplitVals));
1405     memset(globalSplitVals, 0, sizeof(globalSplitVals));
1406
1407     #ifdef DEBUG_TOPOLOGY_DOMAIN
1408         dbgDom    = DEBUG_TOPOLOGY_DOMAIN;
1409     #else
1410         dbgDom    = -1;
1411     #endif
1412
1413     #ifdef ESHELBY

```

```

1414 /*
1415 *      In case we're simulating eshelby inclusions and using a different
1416 *      mobility for segments intersecting them, we need to re-initialize
1417 *      the segment/particle intersection list here. Any other portion of
1418 *      code using the list *should* have cleared it when the list was no
1419 *      longer needed, but just for safety's sake we do it here.
1420 */
1421 SegPartListClear(home);
1422 #endif
1423
1424 /*
1425 *      Loop through all native nodes. Nodes with at least four arms and
1426 *      eligible three-arm BCC binary junctions are split candidates.
1427 */
1428 for (i = 0; i < home->newNodeKeyPtr; i++) {
1429     int    binaryJunc = -1;
1430     int    repositionNode1    =0;
1431     int    repositionNode2    =0;
1432     int    repositionBothNodes=0;
1433     real8 minSplitDist;
1434     real8 distFactor;
1435     real8 splitPos1[3] = { 0.0, 0.0, 0.0 };
1436     real8 splitPos2[3] = { 0.0, 0.0, 0.0 };
1437
1438     node = home->nodeKeys[i];
1439     if (node ==(Node_t *)NULL) continue;
1440
1441     if (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) &&
1442         (node->numNbrs != 2)) {
1443         REMOVE_CONSTRAINTS(node->constraint, CORNER_NODE);
1444     }
1445
1446     if (node->numNbrs < 3) continue;
1447
1448     if (node->numNbrs == 3) {
1449         int    planarJunc;
1450         real8 tjunc[3];
1451
1452         if ((!param->split3node) ||
1453             (param->materialType != MAT_TYPE_BCC)) {
1454             continue;
1455         }
1456
1457         binaryJunc = BCC_binary_junction_node(home, node, tjunc,
1458                                             &planarJunc);
1459
1460         if ((binaryJunc < 0) || planarJunc) continue;
1461     }
1462
1463     if (HAS_ANY_OF_CONSTRAINTS(node->constraint, SURFACE_NODE)) {
1464         continue;
1465     }
1466
1467 /*
1468 *      It might happened through collision that a pinned node has
1469 *      more than one arm. 4-arms is a stretch but just in case.
1470 */
1471     if (HAS_ANY_OF_CONSTRAINTS(node->constraint, PINNED_NODE)) {
1472         continue;

```

```

1473         }
1474
1475         origNumNbrs = node->numNbrs;
1476
1477 #ifdef DEBUG_LOG_MULTI_NODE_SPLITS
1478         node->multiNodeLife++;
1479         localSplitVals[0]++;
1480         localSplitVals[1] += node->multiNodeLife;
1481 #endif
1482 /*
1483  *           If we don't want to evaluate the nodes for splitting this
1484  *           cycle, skip the node. NOTE: we could entirely skip the loop
1485  *           over nodes, but we want the above update to the multi-node
1486  *           lifetimes...
1487  */
1488         if (skipSplits) continue;
1489
1490         nbrs = node->numNbrs;
1491
1492 /*
1493  *           If we haven't already done so, allocate a temporary array
1494  *           in which to store segment forces while evaluating possible
1495  *           node splits. We'll allocate the array large enough to
1496  *           handle the split of a 15-node. If we need an array larger
1497  *           than that, we've probably got other problems.
1498  */
1499         if (segForces == (SegForce_t *)NULL) {
1500             segForces = (SegForce_t *)malloc(17 * sizeof(SegForce_t));
1501         }
1502
1503 /*
1504  *           If any of the node's segments is too short, don't do the
1505  *           split. This is an attempt to prevent creating nodes with
1506  *           incredibly short segments that oscillate at high velocities
1507  */
1508         if (HasShortSeg(home, node, shortSeg)) {
1509             continue;
1510         }

```

Line-by-line explanation.

Lines 1354–1382

The routine declares candidate partitions, backups, force records, mobility arguments, and the best power-release candidate.

Lines 1385–1400

The frequency guard lets diagnostics remain active while skipping costly tests on other cycles. Initial desired separation is $2r_{ann} + \epsilon$; `vNoise` penalizes candidates whose apparent advantage is within time-step/mobility noise.

Lines 1424–1461

Native nodes with at least four arms are candidates. A three-arm node is admitted only when `split3node` is enabled for BCC and `BCC_binary_junction_node()` identifies a non-planar binary junction.

Lines 1463–1510

Surface and pinned nodes are excluded. A short arm also excludes the candidate because splitting near an already tiny segment can create high-velocity numerical debris.

```

1512  /*
1513  *      Preserve the original state of the node being evaluated
1514  *      for separation as well as the state of all its neighbor
1515  *      nodes.
1516  */
1517      bkupNodeCount = node->numNbrs + 1;
1518      bkupNodeList = (Node_t *)malloc(bkupNodeCount * sizeof(Node_t));
1519
1520      BackupNode(home, node, &bkupNodeList[0]);
1521
1522      for (j = 0; j < node->numNbrs; j++) {
1523          nbrNode = GetNodeFromTag(home, node->nbrTag[j]);
1524          BackupNode(home, nbrNode, &bkupNodeList[j+1]);
1525      }
1526
1527  /*
1528  *      The determination of which (if any) arm-splitting
1529  *      possibilities to use when breaking apart a node is
1530  *      based on the maximum energy release rate. Use the
1531  *      original unsplit node as the baseline condition
1532  *      for the energy comparison.
1533  */
1534      powerMax = ((node->fX * node->vX) +
1535                  (node->fY * node->vY) +
1536                  (node->fZ * node->vZ));
1537
1538  /*
1539  *      Get the total number of ways in which this node can
1540  *      be split and build arrays of flags we'll use later
1541  *      for selecting which segments should be split off the
1542  *      node with each corresponding split.
1543  */
1544      SMNGetArmSets(nbrs, &totalSets, &armSets);
1545
1546  /*
1547  *      For each of the node-splitting possibilities, attempt
1548  *      the node split, evaluate the forces and velocities of
1549  *      the two nodes after the split, and determine which
1550  *      splitting possibility leads to the greatest energy
1551  *      release; that's the split that should ultimately be
1552  *      done.
1553  */
1554      colliding = 0;
1555      setIndex = -1;
1556      numSets = totalSets;
1557
1558      origPos[X] = node->x;
1559      origPos[Y] = node->y;
1560      origPos[Z] = node->z;
1561
1562      origVel[X] = node->vX;
1563      origVel[Y] = node->vY;
1564      origVel[Z] = node->vZ;
1565
1566      armList = (int *)calloc(1, (nbrs - 1) * sizeof(int));
1567      armList2 = (int *)calloc(1, (nbrs - 1) * sizeof(int));
1568
1569      for (j = 0; j < numSets; j++) {

```



```

1570         int tmpRepositionNode1, tmpRepositionNode2;
1571
1572         if ((nbrs == 3) && (armSets[j][binaryJunc] != 0)) {
1573             continue;
1574         }
1575
1576         /*
1577          *
1578          * Attempt to split the node. If the split fails,
1579          * just move on and check the next possible way of
1580          * splitting the node, but if the split succeeded
1581          * we need to evaluate the results then restore
1582          * the nodes to the original unsplit state.
1583          */
1584         for (armIndex = 0, k = nbrs - 1; k >= 0; k--) {
1585             if (armSets[j][k] == 1) {
1586                 armList[armIndex++] = k;
1587             }
1588         }
1589
1590         /*
1591          *
1592          * Note: for this test, both the node being split
1593          * and the resulting new node will initially retain the
1594          * location and velocity of the node being split.
1595          * This may lead to a zero length segment between the
1596          * two nodes, but the mobility function should be okay
1597          * with that. Also, this operation MUST NOT be
1598          * communicated to remote domains from processing!
1599          *
1600          * Note: on success, the arms selected to be split
1601          * will be connected to splitNode2 and all unselected
1602          * arms will be attached to splitNode1.
1603          */
1604         splitStatus = SplitNode(home, OPCLASS_SEPARATION,
1605                                node, origPos, origPos,
1606                                origVel, origVel,
1607                                armSets[j][nbrs],
1608                                armList, 0, &splitNode1,
1609                                &splitNode2, 0);
1610
1611         if (splitStatus != SPLIT_SUCCESS) continue;
1612
1613         /*
1614          *
1615          * For the initial split, the two nodes involved
1616          * both remain at the same location as the original
1617          * node. This means that the forces on all segments
1618          * attached to the nodes should be unchanged from
1619          * the corresponding segments attached to the original
1620          * node; this means we can avoid the expense of
1621          * calculating forces here, and just copy the
1622          * corresponding forces from the backup copies of
1623          * the original nodes...
1624          */
1625         GetForcesFromBkup(home, splitNode1, bkupNodeList);
1626         GetForcesFromBkup(home, splitNode2, bkupNodeList);
1627
1628         #ifdef ESHELBY
1629         /*
1630          *
1631          * If we're using special mobility for segments
1632          * intersecting eshelby inclusions, we need to

```

```

1629 *           generate the segment/particle intersection list
1630 *           (and clear it after calculating velocities)
1631 */
1632           FindNodePartIntersects(home, splitNode1);
1633           FindNodePartIntersects(home, splitNode2);
1634           SegPartListSort(home);
1635 #endif
1636
1637           mobError = EvaluateMobility(home, splitNode1, &mobArgs1);
1638           mobError |= EvaluateMobility(home, splitNode2, &mobArgs2);
1639
1640 #ifdef ESHELBY
1641           SegPartListClear(home);
1642 #endif

```

Line-by-line explanation.

Lines 1512–1525

The original node and every neighbor are deep-copied. Trial candidates temporarily mutate real data, so restore must recover arrays as well as scalar node fields.

Lines 1528–1536

Original $f \cdot v$ power is the baseline. A split is attractive only if post-split motion releases more power.

Lines 1539–1568

SMNGetArmSets() enumerates unique nontrivial partitions. Position/velocity arrays remember the best trial, while armList is built in descending arm-index order.

Lines 1569–1609

Each partition is applied with globalOp=0; the zero-length trial link is local and temporary. Selected arms attach to splitNode2. For BCC degree-three work, the binary-junction arm cannot be put in the selected subset.

Lines 1611–1642

Existing segment forces are copied from backups, avoiding a full force calculation at the zero-length trial. The two temporary nodes then receive mobility evaluation, with inclusion support when enabled.

```

1644 /*
1645 *           After the split is done, reposition the
1646 *           nodes and again calculate velocities. If
1647 *           the direction of velocity changes, don't
1648 *           do the split, but only if we didn't have
1649 *           any errors returned from the mobility function.
1650 */
1651           if (mobError == 0) {
1652
1653               tmpRepositionNode1 = 0;
1654               tmpRepositionNode2 = 0;
1655
1656 /*
1657 *           When the split creates a new junction segment, the
1658 *           segment is initially zero length but this segment
1659 *           will affect the force, velocity and direction of
1660 *           the two attached nodes. So, we need to try to
1661 *           determine the optimal junction direction and adjust
1662 *           the force/velocity of the two nodes appropriately.

```

```

1663  */
1664      AdjustJuncNodeForceAndVel(home, splitNode1, &mobArgs1);
1665      AdjustJuncNodeForceAndVel(home, splitNode2, &mobArgs2);
1666
1667      if (nbrs == 3) {
1668          Node_t *twoNode =
1669              (splitNode1->numNbrs == 2) ? splitNode1 :
1670              ((splitNode2->numNbrs == 2) ? splitNode2 :
1671              (Node_t *)NULL);
1672
1673          if (twoNode != (Node_t *)NULL) {
1674              twoNode->vX = 0.0;
1675              twoNode->vY = 0.0;
1676              twoNode->vZ = 0.0;
1677          }
1678      }
1679
1680      vd1 = (splitNode1->vX * splitNode1->vX) +
1681            (splitNode1->vY * splitNode1->vY) +
1682            (splitNode1->vZ * splitNode1->vZ);
1683
1684      vd2 = (splitNode2->vX * splitNode2->vX) +
1685            (splitNode2->vY * splitNode2->vY) +
1686            (splitNode2->vZ * splitNode2->vZ);
1687
1688  /*
1689  *      This configuration should only be considered as
1690  *      a viable option if the velocity of the nodes
1691  *      after the split are not near zero.
1692  */
1693      if ((fabs(vd1) > eps) || (fabs(vd2) > eps)) {
1694          real8 vel1Dotvel2;
1695          real8 vd1Sqrt, vd2Sqrt;
1696
1697          vel1Dotvel2 = (splitNode1->vX * splitNode2->vX) +
1698                      (splitNode1->vY * splitNode2->vY) +
1699                      (splitNode1->vZ * splitNode2->vZ);
1700
1701          vd1Sqrt = sqrt(vd1);
1702          vd2Sqrt = sqrt(vd2);
1703
1704          if (((nbrs != 3) || !param->enforceGlidePlanes) &&
1705              (vd1 > eps) && (vd2 > eps) &&
1706              (fabs(vel1Dotvel2/vd1Sqrt/vd2Sqrt+1.0) <
1707               0.01)) {
1708              /*
1709              *      In this case, the velocity of the two nodes
1710              *      is nearly equal and opposite, so we'll treat
1711              *      them as *exactly* equal and opposite and
1712              *      later reposition both nodes.
1713              */
1714              tmpRepositionNode1 = 1;
1715              tmpRepositionNode2 = 1;
1716
1717              if (vd1 > vd2) {
1718
1719                  splitNode2->vX = -splitNode1->vX;
1720                  splitNode2->vY = -splitNode1->vY;
1721                  splitNode2->vZ = -splitNode1->vZ;

```

```

1722
1723         vd2 = (splitNode2->vX * splitNode2->vX) +
1724             (splitNode2->vY * splitNode2->vY) +
1725             (splitNode2->vZ * splitNode2->vZ);
1726
1727     } else {
1728
1729         splitNode1->vX = -splitNode2->vX;
1730         splitNode1->vY = -splitNode2->vY;
1731         splitNode1->vZ = -splitNode2->vZ;
1732
1733         vd1 = (splitNode1->vX * splitNode1->vX) +
1734             (splitNode1->vY * splitNode1->vY) +
1735             (splitNode1->vZ * splitNode1->vZ);
1736
1737     }
1738 }
1739
1740 if (vd1 > vd2) {
1741
1742     tmpRepositionNode1 = 1;
1743     invvNorm = 1.0 / sqrt(vd1);
1744
1745     dirx = -splitNode1->vX * invvNorm;
1746     diry = -splitNode1->vY * invvNorm;
1747     dirz = -splitNode1->vZ * invvNorm;
1748
1749 } else {
1750
1751     tmpRepositionNode2 = 1;
1752     invvNorm = 1.0 / sqrt(vd2);
1753
1754     dirx = splitNode2->vX * invvNorm;
1755     diry = splitNode2->vY * invvNorm;
1756     dirz = splitNode2->vZ * invvNorm;
1757 }
1758
1759 /*
1760  * We initially want to separate the two nodes
1761  * by a distance of <splitDist>. If we're
1762  * moving both nodes, we'll move them each half
1763  * that distance, if we're only moving one node
1764  * we move it the full distance.
1765  */
1766
1767 repositionBothNodes = tmpRepositionNode1 *
1768                       tmpRepositionNode2;
1769
1770 if (repositionBothNodes) {
1771     minSplitDist = splitDist * 0.5;
1772 } else {
1773     minSplitDist = splitDist;
1774 }
1775
1776 distFactor = minSplitDist * tmpRepositionNode1;
1777
1778 splitNode1->x -= dirx * distFactor;
1779 splitNode1->y -= diry * distFactor;
1780 splitNode1->z -= dirz * distFactor;

```

```

1781 FoldBox(param, &splitNode1->x, &splitNode1->y,
1782         &splitNode1->z);
1783
1784 distFactor = minSplitDist * tmpRepositionNode2;
1785
1786 splitNode2->x += dirx * distFactor;
1787 splitNode2->y += diry * distFactor;
1788 splitNode2->z += dirz * distFactor;
1789
1790 FoldBox(param, &splitNode2->x, &splitNode2->y,
1791         &splitNode2->z);
1792
1793 /*
1794  * If eshelby inclusions are being simulated,
1795  * the segment/particle intersection list may be
1796  * generated during force calcs. We'll need to
1797  * sort the list after it's generated, and clear it
1798  * when we're done with it.
1799  */
1800 SetOneNodeForce(home, splitNode1);
1801 SetOneNodeForce(home, splitNode2);
1802
1803 /*
1804  * When the original node was split above, both new
1805  * nodes were left at the original position. If a
1806  * new segment was created connecting the nodes, it
1807  * initially zero length and therefore had no glide-
1808  * plane, but the mobility function can handle zero-
1809  * length segs. After calculating velocities, though
1810  * we've shifted the nodes making the connecting
1811  * segment (if any) non-zero length, so now we have
1812  * to explicitly set the glide plane of that seg
1813  * because some of the mobility functions depend on
1814  * the glide plane being set for non-zero length segs
1815  *
1816  * Added by Wei Cai, 2009/09/07
1817  */
1818 armIndex = GetArmID(splitNode1, splitNode2);
1819
1820 if (armIndex >= 0 ) {
1821     real8 burg[3], dirvec[3], nplane[3];
1822
1823     /*
1824      * Calculate glide plane normal as
1825      * n = cross ( burg(x,y,z), dir(x,y,z))
1826      */
1827     burg[X] = splitNode1->burgX[armIndex];
1828     burg[Y] = splitNode1->burgY[armIndex];
1829     burg[Z] = splitNode1->burgZ[armIndex];
1830
1831     dirvec[X] = dirx;
1832     dirvec[Y] = diry;
1833     dirvec[Z] = dirz;
1834
1835     FindPreciseGlidePlane(home, burg, dirvec,
1836                          nplane, home->param->allowFuzzyGlidePlanes
1837 );
1838 /*

```

```

1839  *           If the new segment is screw the glide
1840  *           plane is undefined. If that's the case, we
1841  *           need to randomly select a glide plane
1842  *           appropriate to the burgers vector.
1843  */
1844           if (DotProduct(nplane, nplane) < 1.0e-03) {
1845               PickScrewGlidePlane(home, burg, nplane);
1846           }
1847
1848           Normalize(&nplane[X], &nplane[Y], &nplane[Z]);
1849
1850           splitNode1->nx[armIndex] = nplane[X];
1851           splitNode1->ny[armIndex] = nplane[Y];
1852           splitNode1->nz[armIndex] = nplane[Z];
1853
1854           armIndex = GetArmID(splitNode2, splitNode1);
1855
1856           splitNode2->nx[armIndex] = nplane[X];
1857           splitNode2->ny[armIndex] = nplane[Y];
1858           splitNode2->nz[armIndex] = nplane[Z];
1859       }
1860
1861   #ifdef ESHELBY
1862
1863       SegPartListSort(home);
1864   #endif
1865
1866       mobError = EvaluateMobility(home, splitNode1, &mobArgs1);
1867       mobError |= EvaluateMobility(home, splitNode2, &mobArgs2);
1868
1869       if (nbrs == 3) {
1870           Node_t *twoNode =
1871               (splitNode1->numNbrs == 2) ? splitNode1 :
1872               ((splitNode2->numNbrs == 2) ? splitNode2 :
1873               (Node_t *)NULL);
1874
1875           if (twoNode != (Node_t *)NULL) {
1876               twoNode->vX = 0.0;
1877               twoNode->vY = 0.0;
1878               twoNode->vZ = 0.0;
1879           }
1880       }
1881
1882   #ifdef ESHELBY
1883
1884       SegPartListClear(home);
1885   #endif
1886
1887       vDiffx = splitNode2->vX - splitNode1->vX;
1888       vDiffy = splitNode2->vY - splitNode1->vY;
1889       vDiffz = splitNode2->vZ - splitNode1->vZ;
1890   } else {
1891       /*
1892       *           Velocities of the nodes after the split were too
1893       *           close to zero, so we won't do the split.
1894       */
1895       vDiffx = -1.0;
1896       vDiffy = -1.0;
1897       vDiffz = -1.0;

```

```

1898         dirx = 1.0;
1899         diry = 1.0;
1900         dirz = 1.0;
1901     }
1902
1903 } /* if (mobError == 0) */
1904
1905 if (mobError != 0) {
1906     /*
1907     *
1908     * If the mobility function was unable to converge on
1909     * velocities for either node in their new positions,
1910     * we can't really evaluate the node split properly.
1911     * Given that, regardless of what other split
1912     * possibilities were evaluated for this multi-node,
1913     * set flags to avoid splitting this node and skip
1914     * evaluation of any other possible splits for this
1915     * node this cycle.
1916     */
1917
1918     setIndex = -1;
1919     j = numSets;
1920
1921 } else if (((vDiffx*dirx) + (vDiffy*diry) +
1922           (vDiffz*dirz)) > 0) {
1923
1924     powerTest = ((splitNode1->fX * splitNode1->vX) +
1925                 (splitNode1->fY * splitNode1->vY) +
1926                 (splitNode1->fZ * splitNode1->vZ)) +
1927                 ((splitNode2->fX * splitNode2->vX) +
1928                 (splitNode2->fY * splitNode2->vY) +
1929                 (splitNode2->fZ * splitNode2->vZ)) -
1930                 vNoise * (
1931                     sqrt(splitNode1->fX * splitNode1->fX +
1932                           splitNode1->fY * splitNode1->fY +
1933                           splitNode1->fZ * splitNode1->fZ) +
1934                     sqrt(splitNode2->fX * splitNode2->fX +
1935                           splitNode2->fY * splitNode2->fY +
1936                           splitNode2->fZ * splitNode2->fZ));
1937
1938     /*
1939     *
1940     * If this potential node split would result in the
1941     * highest energy release, save enough info to
1942     * perform this split later.
1943     */
1944
1945     if (((powerTest - powerMax) > eps) &&
1946         (nbrs != 3) || (powerTest > 1.0)) {
1947
1948         vel1[X] = splitNode1->vX;
1949         vel1[Y] = splitNode1->vY;
1950         vel1[Z] = splitNode1->vZ;
1951
1952         vel2[X] = splitNode2->vX;
1953         vel2[Y] = splitNode2->vY;
1954         vel2[Z] = splitNode2->vZ;
1955
1956         splitPos1[X] = splitNode1->x;
1957         splitPos1[Y] = splitNode1->y;
1958         splitPos1[Z] = splitNode1->z;
1959
1960         splitPos2[X] = splitNode2->x;

```

```

1957         splitPos2[Y] = splitNode2->y;
1958         splitPos2[Z] = splitNode2->z;
1959
1960         repositionNode1 = tmpRepositionNode1;
1961         repositionNode2 = tmpRepositionNode2;
1962
1963         powerMax = powerTest;
1964         setIndex = j;
1965
1966     /*
1967     *
1968     *         Save the forces on all segments of the two nodes
1969     *         resulting from this split.  If this configuration
1970     *         is the one chosen for the final split, we can
1971     *         then use these preserved force values rather
1972     *         than go to the expense of recomputing full forces
1973     *         on the nodes yet once again.
1974     */
1975
1976         segIndex = 0;
1977
1978         SaveSegForces(home, splitNode1,
1979                     segForces, &segIndex);
1980
1981         SaveSegForces(home, splitNode2,
1982                     segForces, &segIndex);
1983
1984     /*
1985     *
1986     *         For 4-nodes only check if the split would
1987     *         likely result in an immediate collision
1988     *         reversing the effects of the split.
1989     *         If this is the case, we'll skip the split
1990     *         to avoid flicker that might affect the
1991     *         timestep.  We don't want to form many-armed
1992     *         nodes, though, so if we've got more than 4
1993     *         arms, don't use this criteria to restrict
1994     *         a node split.
1995     */
1996
1997         if (origNumNbrs == 4) {
1998             colliding = CheckCollisionConditions(home,
1999                                     splitNode2, splitNode1);
2000         } else {
2001             colliding = 0;
2002         }
2003     }
2004 }

```

Line-by-line explanation.

Lines 1644–1665

A successful mobility evaluation is adjusted for new junction force/velocity. The initially zero-length connecting segment becomes important once nodes separate.

Lines 1667–1687

The BCC degree-three case freezes its resulting degree-two corner during trial scoring. Squared speeds identify which node or nodes should move.

Lines 1688–1791

If post-split velocities separate endpoints, the faster side determines a trial direction. Equal-and-

opposite motion moves both nodes half the initial distance; otherwise one moves the full distance. FoldBox() maintains periodic coordinates.

Lines 1800–1888

Forces are recalculated after geometry changes. The new nonzero connector receives a valid glide plane before the second mobility call. Relative velocity is then saved for the power test.

Lines 1905–1920

Mobility failure terminates candidate evaluation for this node. A candidate is physically separating only when relative velocity has a positive component along selected separation direction.

Lines 1922–2001

Trial power is the sum of post-split $f \cdot v$ terms minus force-scaled noise. Only a strict improvement becomes best. Its segment forces/positions are saved; a four-arm split that would immediately collide is marked for rejection.

```
2003  /*
2004  *           Merge the new node back into the original node
2005  *           then restore the original node and it's neighbors
2006  *           to the pristine unsplit state. (Need to do a
2007  *           restore to insure the ordering of arms since
2008  *           the arrays of splitting possibilities use the
2009  *           original node's arm indices, plus we get back
2010  *           all the original force/velocity/etc values.)
2011  *
2012  *           Assumes that MergeNode() will merge <newNode>
2013  *           into <origNode>!
2014  */
2015
2016     if (node == splitNode1) {
2017         origNode = splitNode1;
2018         newNode  = splitNode2;
2019     } else {
2020         newNode  = splitNode1;
2021         origNode = splitNode2;
2022     }
2023
2024  /*
2025  *           This is a kludge to handle pinned multinodes.
2026  *           if the first node is pinned in any dimensions,
2027  *           move the second node to the original position
2028  *           and pin it in the same manner as the original
2029  *           node unpin the first node so the merge will take
2030  *           place as expected.
2031  *
2032  *           NOTE: For now, a node that is pinned in ANY dimension
2033  *           is treated here as if it is pinned in ALL
2034  *           dimensions.
2035  */
2036
2037     if (HAS_ANY_OF_CONSTRAINTS(newNode->constraint,
2038                                PINNED_NODE)) {
2039         origNode->x = origPos[X];
2040         origNode->y = origPos[Y];
2041         origNode->z = origPos[Z];
2042         ADD_CONSTRAINTS(origNode->constraint,
2043                        newNode->constraint & PINNED_NODE);
2044         REMOVE_CONSTRAINTS(newNode->constraint, PINNED_NODE);
2045     }
2046
2047     MergeNode(home, OPCLASS_SEPARATION, newNode, origNode,
```

```

2046             origPos, &mergedNode, &mergeStatus, 0);
2047
2048  /*
2049  *           If the merge failed, something's screwy! Abort.
2050  */
2051     if ((mergeStatus & MERGE_SUCCESS) == 0) {
2052         Fatal("Unable to merge (%d,%d) into (%d,%d)"
2053             "during separation test",
2054             newNode->myTag.domainID,
2055             newNode->myTag.index,
2056             origNode->myTag.domainID,
2057             origNode->myTag.index);
2058     }
2059
2060  /*
2061  *           Restore nodes to the original state
2062  */
2063     for (k = 0; k < bkupNodeCount; k++) {
2064         tmpNode = GetNodeFromTag(home,
2065                                 bkupNodeList[k].myTag);
2066         RestoreNode(home, tmpNode, &bkupNodeList[k]);
2067     }
2068
2069     #if 0
2070     if (mobError) {
2071         printf(" +++ SplitMultiNode: Unable to evaluate "
2072             "mobility for (%d,%d) -- node not split\n",
2073             node->myTag.domainID, node->myTag.index);
2074     }
2075     #endif
2076
2077     } /* for (j = 0; j < numSets;...) */
2078
2079  /*
2080  *           If it is necessary (and possible) to split the node,
2081  *           armSet[setIndex] points to the array indicating which
2082  *           arms are to be split off and which are to remain with
2083  *           the original node.
2084  *
2085  *           However, if configuration resulting after the split
2086  *           would result in an imminent collision which might
2087  *           possibly restore the original configuration, we won't
2088  *           do the split.
2089  */
2090     if (colliding) {
2091         setIndex = -1;
2092     #if 0
2093         printf("SplitMultiNodes(): Skip (%d,%d) split "
2094             "due to subsequent collision!\n",
2095             node->myTag.domainID, node->myTag.index);
2096     #endif
2097     }
2098
2099     if (setIndex >= 0) {
2100         int armSets2;
2101         real8 normsplitted;
2102         real8 invnormsplitted, dsPx, dsPy, dsPz;
2103         real8 junctionLen1, junctionLen2;
2104

```

```

2105     junctionLen1 = 0.0;
2106     junctionLen2 = 0.0;
2107
2108     /*
2109     *       Set up the list of arms to be passed to SplitNode.
2110     *       Note: the arm list MUST be in reverse order (i.e.
2111     *       highest arm index to lowest) These operations are
2112     *       always considered *global* so other domains will be
2113     *       notified of these ops.
2114     *
2115     *       In some cases we also need the list (armList2) of
2116     *       arms NOT being split off, so create that here as well.
2117     */
2118     armIndex = 0;
2119     armIndex2 = 0;
2120     armSets2 = nbrs - armSets[setIndex][nbrs];
2121
2122     for (k = nbrs - 1; k >= 0; k--) {
2123         if (armSets[setIndex][k] == 1) {
2124             armList[armIndex++] = k;
2125         } else {
2126             armList2[armIndex2++] = k;
2127         }
2128     }
2129
2130
2131     /*
2132     *       Get direction from saved data
2133     *       By definition, dir is (x2-x1)/|x2-x1|
2134     */
2135     dsPx = splitPos2[X] - splitPos1[X];
2136     dsPy = splitPos2[Y] - splitPos1[Y];
2137     dsPz = splitPos2[Z] - splitPos1[Z];
2138
2139     ZImage(param, &dsPx, &dsPy, &dsPz);
2140
2141     normsplitted = dsPx*dsPx + dsPy*dsPy + dsPz*dsPz;
2142     invnormsplitted = 1.0 / sqrt(normsplitted);
2143
2144     dirx = dsPx * invnormsplitted;
2145     diry = dsPy * invnormsplitted;
2146     dirz = dsPz * invnormsplitted;
2147
2148     /*
2149     *       If we are restricted to creating junctions of a
2150     *       uniform length, move 1 node the uniform distance or
2151     *       both nodes half that distance. Otherwise, use a line
2152     *       tension approximation to get the optimal distance to
2153     *       move the nodes along the junction direction.
2154     *
2155     *       By convention, splitNode2 has the armList, here pos2 has
2156     *       the arms list. pos1 has the other arms.
2157     */
2158     repositionBothNodes = repositionNode1 * repositionNode2;
2159
2160     if (repositionBothNodes) {
2161         minSplitDist = splitDist * 0.5;
2162     } else {
2163         minSplitDist = splitDist;

```

```

2164     }
2165
2166     distFactor = 0;
2167
2168     if (repositionNode1) {
2169
2170         if (param->useUniformJunctionLen) {
2171
2172             distFactor = minSplitDist;
2173
2174         } else {
2175
2176             junctionLen1 =
2177                 CalcJunctionLen(home, node,
2178                                armSets2, armList2,
2179                                -dirx, -diry, -dirz,
2180                                minSplitDist,
2181                                repositionBothNodes);
2182
2183             distFactor = MAX(minSplitDist, junctionLen1);
2184         }
2185     }
2186
2187     pos1[X] = node->x - dirx * distFactor;
2188     pos1[Y] = node->y - diry * distFactor;
2189     pos1[Z] = node->z - dirz * distFactor;
2190
2191     FoldBox(param, &pos1[X], &pos1[Y], &pos1[Z]);
2192
2193     distFactor = 0;
2194
2195     if (repositionNode2) {
2196
2197         if (param->useUniformJunctionLen) {
2198
2199             distFactor = minSplitDist;
2200
2201         } else {
2202             junctionLen2 =
2203                 CalcJunctionLen(home, node,
2204                                armSets[setIndex][nbrs],
2205                                armList, dirx, diry, dirz,
2206                                minSplitDist,
2207                                repositionBothNodes);
2208
2209             distFactor = MAX(minSplitDist, junctionLen2);
2210         }
2211     }
2212
2213     pos2[X] = node->x + dirx * distFactor;
2214     pos2[Y] = node->y + diry * distFactor;
2215     pos2[Z] = node->z + dirz * distFactor;
2216
2217     FoldBox(param, &pos2[X], &pos2[Y], &pos2[Z]);
2218
2219     if (nbrs == 3) {
2220         VECTOR_COPY(pos1, splitPos1);
2221         VECTOR_COPY(pos2, splitPos2);
2222     }

```

```

2223
2224 /*
2225 *      Note: after the split, the arms selected to be split
2226 *      will be connected to splitNode2 which will be
2227 *      positioned at pos2 with velocity vel2. Node
2228 *      splitNode1 will be attached to the arms not selected
2229 *      to be moved, and will have the position and velocity
2230 *      indicated by pos1 and vel1.
2231 */
2232 #ifdef DEBUG_LOG_MULTI_NODE_SPLITS
2233     localSplitVals[2]++;
2234     localSplitVals[3] += node->multiNodeLife;
2235 #endif
2236     splitStatus = SplitNode(home, OPCLASS_SEPARATION,
2237                             node, pos1, pos2, vel1, vel2,
2238                             armSets[setIndex][nbrs],
2239                             armList, globalOp,
2240                             &splitNode1, &splitNode2, 0);
2241 /*
2242 *      If the split failed, clean up then go back and
2243 *      continue looking for more multinodes to split.
2244 */
2245     if (splitStatus != SPLIT_SUCCESS) {
2246         free(armList);
2247         armList = (int *)NULL;
2248         for (k = 0; k < numSets; k++) free(armSets[k]);
2249         free(armSets);
2250         free(armList2);
2251         continue;
2252     }
2253
2254     if (nbrs == 3) {
2255         if (splitNode1->numNbrs == 2) {
2256             ADD_CONSTRAINTS(splitNode1->constraint, CORNER_NODE);
2257             if (globalOp) {
2258                 AddOpResetConstraint(home, &splitNode1->myTag,
2259                                     splitNode1->constraint);
2260             }
2261         }
2262         if (splitNode2->numNbrs == 2) {
2263             ADD_CONSTRAINTS(splitNode2->constraint, CORNER_NODE);
2264             if (globalOp) {
2265                 AddOpResetConstraint(home, &splitNode2->myTag,
2266                                     splitNode2->constraint);
2267             }
2268         }
2269     }
2270
2271 #ifdef FIX_PLASTIC_STRAIN
2272 /*
2273 *      Update plastic strain as the node is being split
2274 */
2275     if (splitStatus == SPLIT_SUCCESS) {
2276         real8 newPos[3];
2277
2278         newPos[X] = splitNode1->x;
2279         newPos[Y] = splitNode1->y;
2280         newPos[Z] = splitNode1->z;
2281         UpdateNodePlasticStrain(home, splitNode1, origPos, newPos);

```

```

2282
2283         newPos[X] = splitNode2->x;
2284         newPos[Y] = splitNode2->y;
2285         newPos[Z] = splitNode2->z;
2286         UpdateNodePlasticStrain(home, splitNode2, origPos, newPos);
2287     }
2288 #endif
2289
2290 #ifdef DEBUG_LOG_MULTI_NODE_SPLITS
2291     splitNode1->multiNodeLife = 0;
2292     splitNode2->multiNodeLife = 0;
2293 #endif
2294
2295 /*
2296  *
2297  * The forces for all segments attached to the
2298  * two nodes were preserved during the evaluation
2299  * step above, use those preserved values to reset
2300  * the forces on these nodes rather than explicitly
2301  * recompute the forces.
2302  *
2303  * WARNING! Make sure GetSavedSegForces() is called
2304  * for the two nodes in the same order SaveSegForces()
2305  * was called for the two nodes.
2306  */
2307
2308     segIndex = 0;
2309
2310     GetSavedSegForces(home, splitNode1, segForces, &segIndex);
2311     GetSavedSegForces(home, splitNode2, segForces, &segIndex);
2312
2313 /*
2314  *
2315  * Nodal velocities passed into SplitNode() should be okay,
2316  * but we do need to call ResetSegForces2() to distribute
2317  * the new forces out to the remote domains.
2318  */
2319
2320     if (node == splitNode1) {
2321         origNode = splitNode1;
2322         newNode = splitNode2;
2323     } else {
2324         newNode = splitNode1;
2325         origNode = splitNode2;
2326     }
2327
2328     for (q = 0; q < origNode->numNbrs; q++) {
2329         nbrNode = GetNodeFromTag(home, origNode->nbrTag[q]);
2330         tmpArm = GetArmID(nbrNode, origNode);
2331         ResetSegForces2(home, origNode, &nbrNode->myTag,
2332             origNode->armfx[q],
2333             origNode->armfy[q],
2334             origNode->armfz[q],
2335             nbrNode->armfx[tmpArm],
2336             nbrNode->armfy[tmpArm],
2337             nbrNode->armfz[tmpArm], globalOp);
2338     }
2339
2340     for (q = 0; q < newNode->numNbrs; q++) {
2341         nbrNode = GetNodeFromTag(home, newNode->nbrTag[q]);
2342         tmpArm = GetArmID(nbrNode, newNode);
2343         ResetSegForces2(home, newNode, &nbrNode->myTag,
2344             newNode->armfx[q],
2345             newNode->armfy[q],

```

```

2341         newNode->armfz[q],
2342         nbrNode->armfx[tmpArm],
2343         nbrNode->armfy[tmpArm],
2344         nbrNode->armfz[tmpArm], globalOp);
2345     }
2346
2347
2348
2349     /*
2350     *
2351     *       Mark both nodes involved in the split as 'exempt'
2352     *       from subsequent collisions this time step.
2353     */
2354     origNode->flags |= NO_COLLISIONS;
2355     newNode->flags  |= NO_COLLISIONS;
2356
2357     /*
2358     *       When debugging, dump some info on topological
2359     *       changes taking place and the nodes involved
2360     */
2361     #ifndef DEBUG_TOPOLOGY_CHANGES
2362     if ((dbgDom < 0) || (dbgDom == home->myDomain)) {
2363         printf("   Separate: (%d,%d) from (%d,%d)\n",
2364             newNode->myTag.domainID,
2365             newNode->myTag.index,
2366             origNode->myTag.domainID,
2367             origNode->myTag.index);
2368         PrintNode(origNode);
2369         PrintNode(newNode);
2370     }
2371     #endif
2372
2373     } /* if (setIndex >= 0) */
2374
2375     for (k = 0; k < numSets; k++) {
2376         free(armSets[k]);
2377     }
2378
2379     free(armSets);
2380     free(armList);
2381     free(armList2);
2382
2383     armList = (int *)NULL;
2384     armList2 = (int *)NULL;
2385
2386     /*
2387     *       Need to free the arrays and node structures used
2388     *       above for backup/restore or we end up with a memory
2389     *       leak.
2390     */
2391     for (k = 0; k < bkupNodeCount; k++) {
2392         FreeNodeArrays(&bkupNodeList[k]);
2393     }
2394
2395     free(bkupNodeList);
2396     bkupNodeList = (Node_t *)NULL;
2397
2398     } /* end loop over nodes */

```

Line-by-line explanation.

Lines 2003–2067

Every temporary split is merged back locally and the original node/neighbors are restored from deep backups. This is a true rollback, so original arm indices remain valid across trials.

Lines 2079–2097

A candidate is discarded if predicted immediate collision would undo the new configuration.

Lines 2099–2217

The winning partition is rebuilt in descending order. Its saved direction determines final positions. The code either uses uniform separation or calls `CalcJunctionLen()` for a line-tension-based length.

Lines 2219–2269

Final global `SplitNode()` creates graph mutation. A degree-two node from the BCC three-arm path receives `CORNER_NODE`, and its constraint reset is appended to the MPI log.

Lines 2271–2345

Plastic strain is corrected when enabled. Saved forces are restored to the winning configuration and pushed to every affected endpoint with `ResetSegForces2()`.

Lines 2349–2396

The two participants become collision-exempt for the rest of the cycle. Candidate and backup arrays are then freed.

5.2 BCC binary-junction classifier

```
1231 /*-----
1232 *
1233 *   Function:   BCC_binary_junction_node
1234 *   Description: Return the arm index of a three-arm BCC binary
1235 *                junction, or -1 if the node is not one. Also identify
1236 *                whether the junction line lies in the intersection of
1237 *                its parent glide planes.
1238 *
1239 *-----*/
1240 int BCC_binary_junction_node(Home_t *home, Node_t *node,
1241                             real8 tjunc[3], int *planarJunc)
1242 {
1243     int    armID;
1244     int    binaryJunc;
1245     int    numArmEdge, numArmGlide, numArmJunc;
1246     real8  ndir[3][3];
1247     real8  eps, angleTol;
1248     Param_t *param;
1249
1250     binaryJunc = -1;
1251     numArmEdge = 0;
1252     numArmGlide = 0;
1253     numArmJunc = 0;
1254
1255     VECTOR_ZERO(tjunc);
1256     *planarJunc = 0;
1257
1258     if ((node == (Node_t *)NULL) || (node->numNbrs != 3)) {
1259         return(-1);
1260     }
1261
1262     param = home->param;
```



```

1263     eps = 1.0e-12;
1264     angleTol = 5.0 * M_PI / 180.0;
1265
1266     for (armID = 0; armID < node->numNbrs; armID++) {
1267         real8 bMag, lineMag2;
1268         real8 burg[3], lineDir[3], normal[3];
1269         Node_t *nbrNode;
1270
1271         nbrNode = GetNeighborNode(home, node, armID);
1272
1273         if (nbrNode == (Node_t *)NULL) {
1274             continue;
1275         }
1276
1277         lineDir[X] = nbrNode->x - node->x;
1278         lineDir[Y] = nbrNode->y - node->y;
1279         lineDir[Z] = nbrNode->z - node->z;
1280
1281         ZImage(param, &lineDir[X], &lineDir[Y], &lineDir[Z]);
1282         lineMag2 = DotProduct(lineDir, lineDir);
1283
1284         if (lineMag2 < eps) {
1285             continue;
1286         }
1287
1288         NormalizeVec(lineDir);
1289
1290         burg[X] = node->burgX[armID];
1291         burg[Y] = node->burgY[armID];
1292         burg[Z] = node->burgZ[armID];
1293         bMag = Normal(burg);
1294
1295         if (bMag > (1.0 + eps)) {
1296             VECTOR_COPY(tjunc, lineDir);
1297             binaryJunc = armID;
1298             numArmJunc++;
1299         } else {
1300             CrossVector(burg, lineDir, normal);
1301
1302             if (fabs(DotProduct(normal, normal)) > 1.0e-2) {
1303                 VECTOR_COPY(ndir[numArmEdge], normal);
1304                 numArmEdge++;
1305             }
1306
1307             numArmGlide++;
1308         }
1309     }
1310
1311     if ((numArmJunc == 1) && (numArmGlide == 2)) {
1312         if (numArmEdge == 2) {
1313             real8 collinearTol, tdir[3];
1314
1315             collinearTol = 1.0 - cos(angleTol);
1316             CrossVector(ndir[0], ndir[1], tdir);
1317             NormalizeVec(tdir);
1318
1319             *planarJunc =
1320                 (fabs(fabs(DotProduct(tdir, tjunc)) - 1.0) <
1321                  collinearTol);

```

```

1322     } else if (numArmEdge == 1) {
1323         *planarJunc =
1324             (fabs(DotProduct(ndir[0], tjunc)) < sin(angleTol));
1325     } else {
1326         *planarJunc = 1;
1327     }
1328 } else {
1329     binaryJunc = -1;
1330 }
1331
1332 return(binaryJunc);
1333 }

```

Line-by-line explanation.

Lines 1231–1264

The classifier returns an arm index or -1 . It requires exactly three arms, initializes a candidate junction direction, and uses a five-degree geometric tolerance.

Lines 1266–1308

Each nonzero-length arm direction is calculated in the nearest periodic image. An arm with Burgers magnitude above one is treated as the junction arm; the other two supply edge/glide-plane normals.

Lines 1311–1329

A valid binary junction has one junction arm and two glissile arms. Cross products of parent normals decide whether the junction line is planar. Planar cases are excluded by the caller; the extension splits the non-planar case and preserves its physical corner.

Lines 1332–1333

The junction arm index lets the caller reject partitions that would move it to the degree-two side.

5.3 Parallel multi-node splitting

```

15 void SplitMultiNodesParallel(Home_t *home)
16 {
17     int      i, bufID;
18     int      gmnCount, lmnCount;
19     int      sendDomCount;
20     int      *sendDoms;
21     int      *sendBufLens;
22     int      recvDomCount;
23     int      *recvDoms;
24     real8    **sendBufs;
25     SMN_Info_t *gmnInfo, *lmnInfo;
26
27
28     /*
29     *    Only do multinode splits on cycles that are multiples
30     *    of the <splitMultiNodeFreq> control file parameter.
31     */
32     if (home->cycle % home->param->splitMultiNodeFreq) {
33         return;
34     }
35
36     TimerStart(home, SPLIT_MULTI_NODES);
37
38     /*

```

```

39  *      Get the lists of ghost and local multinodes for which this
40  *      domain needs to do some force calcs, split each of those nodes
41  *      and get the partial forces
42  */
43      gmnCount = 0;
44      gmnInfo = (SMN_Info_t *)NULL;
45
46      lmnCount = 0;
47      lmnInfo = (SMN_Info_t *)NULL;
48
49      BuildGhostMultiNodeList(home, &gmnCount, &gmnInfo);
50      SMNEvalSplitForces(home, gmnCount, gmnInfo);
51
52      BuildLocalMultiNodeList(home, &lmnCount, &lmnInfo);
53      SMNEvalSplitForces(home, lmnCount, lmnInfo);
54
55  /*
56  *      Pack the force data up for the appropriate remote domains
57  */
58      SMNPackForceBuffers(home, gmnCount, gmnInfo, &sendDomCount,
59                          &sendDoms, &sendBufLens, &sendBufs);
60
61  /*
62  *      Determine which remote domains should be sending force data
63  *      to this one, send force data for ghost multinodes to the
64  *      domains owning them, and receive force from any domain that may
65  *      have calculated partial forces for a multinode local to this domain.
66  */
67      BuildRecvDomList(home, lmnCount, lmnInfo, &recvDomCount,
68                      &recvDoms);
69
70      SMNForceComm(home, sendDomCount, sendDoms, sendBufLens, sendBufs,
71                  recvDomCount, recvDoms, lmnCount, lmnInfo);
72
73  /*
74  *      Free various buffers allocated for the communication step
75  */
76      if (gmnCount > 0) {
77
78          for (i = 0; i < gmnCount; i++) {
79              free(gmnInfo[i].splitInfo);
80          }
81
82          free(gmnInfo);
83      }
84
85      if (sendDomCount > 0) {
86
87          free(sendDoms);
88          free(sendBufLens);
89
90          for (bufID = 0; bufID < sendDomCount; bufID++) {
91              free(sendBufs[bufID]);
92          }
93
94          free(sendBufs);
95      }
96
97      if (recvDomCount > 0) {

```

```

98         free(recvDoms);
99     }
100
101     /*
102     *   We should have full forces for all possible splits of
103     *   each local multi-node, so now do the final evaluation to
104     *   determine if/how each multi-node should be split.
105     */
106     SMNEvalMultiNodeSplits(home, lmnCount, lmnInfo);
107
108     /*
109     *   Free up the multinode lists...
110     */
111     if (lmnCount > 0) {
112
113         for (i = 0; i < lmnCount; i++) {
114             free(lmnInfo[i].splitInfo);
115         }
116
117         free(lmnInfo);
118     }
119
120     TimerStop(home, SPLIT_MULTI_NODES);
121
122     return;
123 }

```

Line-by-line explanation.

Lines 17–34

The same cadence policy applies before allocating parallel-work data.

Lines 36–53

The domain builds ghost and native multi-node lists. It evaluates partial forces for every candidate partition because remote contributions are needed to score local candidates accurately.

Lines 55–71

Ghost-candidate force data is packed for owning domains. The code determines which domains should send contributions for native candidates, then exchanges buffers.

Lines 73–99

All transient communication buffers are freed before the final decision stage because their contents have been accumulated into native multi-node records.

Lines 101–106

`SMNEvalMultiNodeSplits()` now has complete forces for local candidates and commits only locally owned winners. The later generic operation exchange propagates resulting graph changes.

Lines 108–123

Candidate storage is released and the shared timer stops.

The records in `include/SMN.h`, lines 43–71 retain the number of arms and per-arm endpoint forces for each trial. Parallel mode changes where forces are evaluated, not the physical acceptance criterion or the `SplitNode()` graph operation.

5.4 Cross slip and collision dispatch

```

636 void CrossSlip(Home_t *home)

```

```

637 {
638     int enabled = home->param->enableCrossSlip;
639     int matType = home->param->materialType;
640     int bccIndx = home->param->crossSlipBCCIndex;
641
642     if (enabled)
643     {
644         if (matType==MAT_TYPE_BCC)
645         {
646             if (bccIndx==0) { CrossSlipBCC (home); }
647             if (bccIndx==1) { CrossSlipBCCRandom(home); }
648
649         }
650
651         else if (matType==MAT_TYPE_FCC          ) { CrossSlipFCC(home); }
652         else if (matType==MAT_TYPE_HCP          ) { CrossSlipHCP(home); }
653         else if (matType==MAT_TYPE_RHOMBOHEDRAL_VA ) { CrossSlipRhombohedralVa(home); }
654     }
655
656     return;

```

```

942 void HandleCollisions(Home_t *home)
943 {
944     switch(home->param->collisionMethod)
945     {
946         case 4: { PredictiveCollisionsLG (home); break; }
947         case 3: { RetroactiveCollisions (home); break; }
948         case 2: { PredictiveCollisions (home); break; }
949         case 1:
950         default: { ProximityCollisions (home); break; }
951     }
952
953     return;
954 }

```

Line-by-line explanation.

CrossSlip.cc, lines 636–640

The dispatcher reads run-time enable state, material type, and BCC model selector.

CrossSlip.cc, lines 642–654

When enabled, exactly one material-specific routine runs. These routines can reset glide planes and reposition nodes; they use the current topology operation list.

Collision.cc, lines 942–951

collisionMethod selects predictive-LG (4), retroactive (3), predictive (2), or proximity (1/default) search/response logic. The default parameter value is 2.

Collision.cc, lines 953–954

The dispatcher has no geometry; it isolates policy selection from callers.

```

776 void PredictiveCollisions(Home_t *home)
777 {
778     int i, j, k, q, arm12, arm21, arm34, arm43;
779     int thisDomain, splitStatus, mergeStatus;
780     int armAB, armBA;
781     int globalOp = 1, didCollision;
782     int close2node1, close2node2, close2node3, close2node4;

```

```

783     int      splitSeg1, splitSeg2;
784     int      cell2Index, nbrCell2Index, nextIndex;
785     int      cell2X, cell2Y, cell2Z, cx, cy, cz;
786     int      collisionConditionIsMet, adjustCollisionPoint=0;
787     real8     mindist2, dist2, ddist2dt, L1, L2, half;
788     real8     cTime, cPoint[3];
789     real8     x1, y1, z1, vx1, vy1, vz1;
790     real8     x2, y2, z2, vx2, vy2, vz2;
791     real8     x3, y3, z3, vx3, vy3, vz3;
792     real8     x4, y4, z4, vx4, vy4, vz4;
793     real8     newx, newy, newz;
794     real8     newvx, newvy, newvz;
795     real8     pnew[3], burg1[3];
796     real8     oldfp0[3], oldfp1[3];
797     real8     f0seg1[3], f1seg1[3], f0seg2[3], f1seg2[3];
798     real8     nodeVel[3], newNodeVel[3];
799     real8     newPos[3], newVel[3];
800     real8     vec1[3], vec2[3];
801     real8     p1[3], p2[3], p3[3], p4[3];
802     real8     v1[3], v2[3], v3[3], v4[3];
803     Tag_t     oldTag1, oldTag2;
804     Node_t     *node1, *node2, *node3, *node4, *tmpNbr;
805     Node_t     *mergenode1, *mergenode2, *targetNode;
806     Node_t     *splitNode1, *splitNode2;
807     Param_t    *param;
808     MobArgs_t  mobArgs;
809 #ifdef _ARLFEM
810     int      resetSurfaceProperties;
811     real8     femSurfaceNorm[3];
812 #endif
813
814     thisDomain = home->myDomain;
815     param      = home->param;
816
817     mindist2 = param->rann * param->rann;
818     half      = 0.5;
819
820     int localCollisionCnt = 0;
821
822 #ifdef DEBUG_TOPOLOGY_DOMAIN
823     dbgDom = DEBUG_TOPOLOGY_DOMAIN;
824 #else
825     dbgDom = -1;
826 #endif
827
828     TimerStart(home, COLLISION_HANDLING);
829
830 /*
831  * Start looping through native nodes looking for segments to collide...
832  */
833     for (i = 0; i < home->newNodeKeyPtr; i++)
834     {
835         if ((node1 = home->nodeKeys[i]) == (Node_t *)NULL) continue;
836         if (node1->flags & NO_COLLISIONS) continue;
837
838         didCollision = 0;
839
840 /*
841  * Loop through all cell2s neighboring the node. Only

```

```

842 *      nodes in these neighboring cell2s are candidates for
843 *      collisions.
844 *
845 *      NOTE: All nodes are assigned cell2 membership prior to
846 *      entering collision handling. However, nodes added during
847 *      node separation and collision handling are not given
848 *      cell2 membership for this cycle. In most cases this
849 *      is not a problem since those nodes have usually been
850 *      exempted from subsequent collisions this cycle. There
851 *      are certain circumstances, though, in which the exemptions
852 *      have not been set. If we encounter one such node that
853 *      has no cell2 membership, skip it for this cycle, or bad
854 *      things happen.
855 */
856     cell2Index = node1->cell2Idx;
857     if (cell2Index < 0) {
858         continue;
859     }
860
861     DecodeCell2Idx(home, cell2Index, &cell2X, &cell2Y, &cell2Z);
862
863     for (cx = cell2X - 1; cx <= cell2X + 1; cx++) {
864         for (cy = cell2Y - 1; cy <= cell2Y + 1; cy++) {
865             for (cz = cell2Z - 1; cz <= cell2Z + 1; cz++) {
866
867                 nbrCell2Index = EncodeCell2Idx(home, cx, cy, cz);
868
869             /*
870              *      Loop though all nodes in the neighbor cell2
871              */
872             nextIndex = home->cell2[nbrCell2Index];
873
874             while (nextIndex >= 0) {
875
876                 if (didCollision) break;
877
878                 node3 = home->cell2QentArray[nextIndex].node;
879                 nextIndex = home->cell2QentArray[nextIndex].next;
880
881                 if (node3 == (Node_t *)NULL) continue;
882                 if (node3->flags & NO_COLLISIONS) continue;
883
884                 if (CollisionNodeOrder(home, &node1->myTag,
885                                         &node3->myTag) >= 0) {
886                     continue;
887                 }
888
889             /*
890              *      Loop over all arms of node1. Skip any arms that
891              *      terminate at node3 (those hinge arms will be dealt
892              *      with later)
893              */
894             for (arm12 = 0; arm12 < node1->numNbrs; arm12++) {
895
896                 if (didCollision) break;
897
898                 node2 = GetNodeFromTag(home, node1->nbrTag[arm12]);
899
900                 if (node2 == (Node_t *)NULL) continue;

```

```

901         if (node2->flags & NO_COLLISIONS) continue;
902
903         if (CollisionNodeOrder(home, &node1->myTag,
904                                &node2->myTag) > 0) {
905             continue;
906         }
907
908         if ((node2->myTag.domainID == node3->myTag.domainID) &&
909             (node2->myTag.index == node3->myTag.index )) {
910             continue;
911         }
912
913         /*
914          *      Segment node1/node2 may only be used in a collision
915          *      if the segment is owned by the current domain.
916          */
917         if (!DomainOwnsSeg(home, OPCLASS_COLLISION,
918                            thisDomain, &node2->myTag)) {
919             continue;
920         }
921
922         /*
923          *      Loop over all arms of node3. Skip any segments that
924          *      terminate at node2 (those hinge arms will be dealt
925          *      with later), and skip any segments not owned by
926          *      node3 -- we'll deal with those segments when we
927          *      hit the node owning the segment
928          *
929          */
930         for (arm34 = 0; arm34 < node3->numNbrs; arm34++) {
931
932             if (didCollision) break;
933
934             node4 = GetNodeFromTag(home, node3->nbrTag[arm34]);
935
936             if (node4 == (Node_t *)NULL) continue;
937             if (node4->flags & NO_COLLISIONS) continue;
938
939             if ((node4->myTag.domainID==node2->myTag.domainID)&&
940                 (node4->myTag.index ==node2->myTag.index)) {
941                 continue;
942             }
943
944             if (CollisionNodeOrder(home, &node3->myTag,
945                                    &node4->myTag) > 0) {
946                 continue;
947             }
948
949             /*
950              *      At this point, segment node3/node4 is owned by
951              *      node3. If node3 is not native to this domain,
952              *      the segment may not be used in a collision since
953              *      the domain doing to collision must own both
954              *      segments.
955              */
956             if (node3->myTag.domainID != thisDomain) {
957                 continue;
958             }
959

```



```

960     x1 = node1->x; y1 = node1->y; z1 = node1->z;
961     x2 = node2->x; y2 = node2->y; z2 = node2->z;
962     x3 = node3->x; y3 = node3->y; z3 = node3->z;
963     x4 = node4->x; y4 = node4->y; z4 = node4->z;
964
965     vx1 = node1->vX; vy1 = node1->vY; vz1 = node1->vZ;
966     vx2 = node2->vX; vy2 = node2->vY; vz2 = node2->vZ;
967     vx3 = node3->vX; vy3 = node3->vY; vz3 = node3->vZ;
968     vx4 = node4->vX; vy4 = node4->vY; vz4 = node4->vZ;
969
970     PBCPOSITION(param, x1, y1, z1, &x2, &y2, &z2);
971     PBCPOSITION(param, x1, y1, z1, &x3, &y3, &z3);
972     PBCPOSITION(param, x3, y3, z3, &x4, &y4, &z4);
973
974     p1[X] = x1; p1[Y] = y1; p1[Z] = z1;
975     p2[X] = x2; p2[Y] = y2; p2[Z] = z2;
976     p3[X] = x3; p3[Y] = y3; p3[Z] = z3;
977     p4[X] = x4; p4[Y] = y4; p4[Z] = z4;
978
979     v1[X] = vx1; v1[Y] = vy1; v1[Z] = vz1;
980     v2[X] = vx2; v2[Y] = vy2; v2[Z] = vz2;
981     v3[X] = vx3; v3[Y] = vy3; v3[Z] = vz3;
982     v4[X] = vx4; v4[Y] = vy4; v4[Z] = vz4;
983
984     /*
985     *
986     *      It is possible to have a zero-length segment
987     *      (created by a previous collision). If we find
988     *      such a segment, do not try to use it in any
989     *      subsequent collisions.
990     */
991
992     vec1[X] = x2 - x1;
993     vec1[Y] = y2 - y1;
994     vec1[Z] = z2 - z1;
995
996     if (DotProduct(vec1, vec1) < 1.0e-20) {
997         continue;
998     }
999
1000     vec2[X] = x4 - x3;
1001     vec2[Y] = y4 - y3;
1002     vec2[Z] = z4 - z3;
1003
1004     if (DotProduct(vec2, vec2) < 1.0e-20) {
1005         continue;
1006     }
1007
1008     /*
1009     *
1010     *      Find the minimum distance between the two segments
1011     *      and determine if they should be collided.
1012     */
1013
1014     GetMinDist(x1, y1, z1, vx1, vy1, vz1,
1015                x2, y2, z2, vx2, vy2, vz2,
1016                x3, y3, z3, vx3, vy3, vz3,
1017                x4, y4, z4, vx4, vy4, vz4,
1018                &dist2, &ddist2dt, &L1, &L2);
1019
1020     /*
1021     *
1022     *      First check of segments already intersect. If
1023     *      not, find out if they will collide in the future.

```

```

1019  *
1020  *           Note: If the separation between the segments is
1021  *           less than 1b, treat them as if they are already
1022  *           intersecting
1023  */
1024
1025         if (dist2 < 1.0) {
1026             cPoint[X] = x1 + vec1[X] * L1;
1027             cPoint[Y] = y1 + vec1[Y] * L1;
1028             cPoint[Z] = z1 + vec1[Z] * L1;
1029             collisionConditionIsMet = 1;
1030         } else if (dist2 < mindist2) {
1031
1032             /*
1033              *           FIX ME!  If segments are to far away but
1034              *           moving fast, they can pass right through
1035              *           each other with no collision
1036
1037              *           Only do a rigorous treatment of points
1038              *           within the distance filter.  Find the
1039              *           collision point, collision time and the
1040              *           points on the segments where the two segments
1041              *           will be colliding.
1042              */

```

Line-by-line explanation of predictive-collision selection.

Lines 776–828

The implementation declares four endpoints, positions, velocities, candidate split/merge records, and rann^2 threshold.

Lines 830–887

Eligible native nodes visit their own and neighboring `cell2` queues. `NO_COLLISIONS`, absent membership, and stable `CollisionNodeOrder()` eliminate invalid or duplicate work.

Lines 889–958

Nested arm loops form segment pairs. Current domain must own both segments before it can change endpoints. Shared-hinge and remote-owned candidates are skipped.

Lines 960–1004

Four endpoints are mapped into consistent periodic images and zero-length segments are rejected before distance mathematics.

Lines 1006–1028

`GetMinDist()` calculates closest approach, approach rate, and fractional positions. Intersections closer than one Burgers-vector unit are treated as current collisions.

Lines 1029–1040

Nearby nonintersecting segments enter predictive time/point calculation. The remainder refines affected segments when needed, then calls `MergeNode()` at a permitted point, updates forces, and marks participants exempt. It uses the same split/merge primitives, not a separate graph-edit system.

6 MPI topology synchronization

6.1 Operation-log protocol

```

14  #define REM_OP_INCR_LEN 4096

```

```

15
16 typedef enum {
17     REM_OP_CHANGE_CONN,
18     REM_OP_INSERT_ARM,
19     REM_OP_REMOVE_NODE,
20     REM_OP_CHANGE_BURG,
21     REM_OP_SPLIT_NODE,
22     REM_OP_RESET_COORD,
23     REM_OP_RESET_SEG_FORCE1,
24     REM_OP_RESET_SEG_FORCE2,
25     REM_OP_MARK_FORCE_OBSOLETE,
26     REM_OP_RESET_PLANE,
27     REM_OP_RESET_CONSTRAINT
28 } OpType_t;
29
30 typedef struct {
31     int    opType;
32     Tag_t tag1;
33     Tag_t tag2;
34     Tag_t tag3;
35 } RemOpChangeConn_t;
36
37 typedef struct {
38     int    opType;
39     Tag_t tag1;
40     Tag_t tag2;
41     real8 burg[3];
42     real8 plane[3];
43 } RemOpInsertArm_t;
44
45 typedef struct {
46     int    opType;
47     Tag_t tag1;
48 } RemOpRemoveNode_t;
49
50 typedef struct {
51     int    opType;
52     Tag_t tag1;
53     Tag_t tag2;
54     real8 newBurg[3];
55     real8 newPlane[3];
56 } RemOpChangeBurg_t;
57
58 typedef struct {
59     int    opType;
60     Tag_t tag1;
61     Tag_t tag2;
62     real8 newCoord[3];
63     real8 newVelocity[3];
64 } RemOpSplitNode_t;
65
66 typedef struct {
67     int    opType;
68     Tag_t tag1;
69     real8 newCoord[3];
70 } RemOpResetCoord_t;
71
72 typedef struct {
73     int    opType;

```

```

74     Tag_t tag1;
75     int    constraint;
76 } RemOpResetConstraint_t;
77
78 typedef struct {
79     int    opType;
80     Tag_t tag1;
81     Tag_t tag2;
82     real8 f1[3];
83 } RemOpResetSegForce1_t;
84
85 typedef struct {
86     int    opType;
87     Tag_t tag1;
88     Tag_t tag2;
89     real8 f1[3];
90     real8 f2[3];
91 } RemOpResetSegForce2_t;
92
93 typedef struct {
94     int    opType;
95     Tag_t tag1;
96 } RemOpMarkForceObsolete_t;
97
98 typedef struct {
99     int    opType;
100    Tag_t tag1;
101    Tag_t tag2;
102    real8 newPlane[3];
103 } RemOpResetPlane_t;

```

```

45 void ClearOpList (Home_t *home)
46 {
47     memset(home->opListBuf, 0, home->opListUsedLen);
48     home->opListUsedLen = 0;
49
50     return;
51 }
52
53 /*-----
54  *
55  * Function:      FreeOpList
56  * Description:   Deallocates the memory dedicated to the operation
57  *               list sent to remote domains after topological
58  *               changes.
59  *
60  *-----*/
61 void FreeOpList (Home_t *home)
62 {
63
64     if (home->opListBuf != (char *)NULL) {
65         free(home->opListBuf);
66         home->opListBuf = (char *)NULL;
67     }
68
69     home->opListAllocLen = 0;
70     home->opListUsedLen  = 0;
71

```

```

72     return;
73 }
74
75 /*-----
76 *
77 *   Function:      InitOpList
78 *   Description:   Allocates an initial block of memory for the
79 *                  list of operations this domain will send to
80 *                  remote domains for processing during the
81 *                  various stages of topological changes.
82 *
83 *                  This function should only need to be called
84 *                  one time during initialization of the application.
85 *                  Adding operations to the list as well as altering
86 *                  the list size will be handled dynamically as
87 *                  necessary
88 *
89 *-----*/
90 void InitOpList (Home_t *home)
91 {
92
93     home->opListBuf      = (char *)calloc(1, REM_OP_INCR_LEN);
94     home->opListAllocLen = REM_OP_INCR_LEN;
95     home->opListUsedLen  = 0;
96
97     return;
98 }
99
100 /*-----
101 *
102 *   Function:      ExtendOpList
103 *   Description:   Increases the buffer size dedicated to the
104 *                  list of operations sent to remote domains
105 *                  after topological changes.
106 *
107 *-----*/
108 void ExtendOpList (Home_t *home)
109 {
110     home->opListAllocLen = home->opListAllocLen + REM_OP_INCR_LEN;
111     home->opListBuf = (char *)realloc(home->opListBuf,
112                                       home->opListAllocLen);
113     return;
114 }

```

```

670 void AddOpSplitNode(Home_t *home, Tag_t *tag1, Tag_t *tag2, real8 newCoord[3],
671                    real8 newVelocity[3])
672 {
673     /*
674     *   For threaded simulations, we must insure only 1 thread
675     *   updates the Op list at a time.
676     */
677     #ifdef _OPENMP
678     #pragma omp critical (CRIT_ADD_OP)
679     #endif
680     {
681         int      opDataLen = sizeof(RemOpSplitNode_t);
682         int      neededLen = home->opListUsedLen + opDataLen;
683         RemOpSplitNode_t opData;

```

```

684
685     if (home->opListAllocLen < neededLen) {
686         ExtendOpList(home);
687     }
688
689     opData.opType = REM_OP_SPLIT_NODE;
690
691     opData.tag1.domainID = tag1->domainID;
692     opData.tag1.index    = tag1->index;
693
694     opData.tag2.domainID = tag2->domainID;
695     opData.tag2.index    = tag2->index;
696
697     for (int i = 0; i < 3; ++i) {
698         opData.newCoord[i]    = newCoord[i];
699         opData.newVelocity[i] = newVelocity[i];
700     }
701
702     memcpy(&home->opListBuf[home->opListUsedLen], &opData, opDataLen);
703
704     home->opListUsedLen += opDataLen;
705
706 } // end omp critical section
707
708 return;

```

Line-by-line explanation.

OpList.h, lines 14–28

OpType_t enumerates every serialized mutation: connection changes, arm insertion/deletion through Burgers change, node split/removal, coordinate/force/plane/constraint reset. All ranks must run a binary with matching record layouts.

OpList.h, lines 30–103

Each record begins with opType, followed by tags and scalar/vector payload to replay exactly one mutation. Tags, rather than pointers, make a byte stream meaningful on a different rank.

RemoteOps.cc, lines 45–51

Clearing an operation list zeroes used bytes and resets used length. It is called once before physics topology work and again before remesh.

RemoteOps.cc, lines 90–114

The list begins as a 4096-byte buffer and grows in fixed increments. Producers need not predict count.

RemoteOps.cc, lines 670–708

A split producer enters an OpenMP critical section, ensures capacity, fills the fixed record (old tag, new tag, coordinates, velocity), copies it to the byte buffer, and advances opListUsedLen. Other AddOp*() helpers use this append pattern.

6.2 Sending the log

```

21 void CommSendRemesh(Home_t *home)
22 {
23     int          isrc, domainIdx, idst, outMsgLen;
24     int          numRemoteDomains;
25     int          *sendBufLen=0;

```

```

26     char          **sendBuf=0;
27     RemoteDomain_t *remDom=0;
28
29
30     numRemoteDomains = home->remoteDomainCount;
31     outMsgLen = home->opListUsedLen;
32
33 #ifdef PARALLEL
34 /*
35  *     pre-issue receives of message length from each neighbor
36  */
37     for (isrc = 0; isrc < home->remoteDomainCount; isrc++) {
38         domainIdx = home->remoteDomains[isrc];
39         remDom = home->remoteDomainKeys[domainIdx];
40
41         MPI_Irecv(&remDom->inBufLen, 1, MPI_INT, domainIdx, MSG_REMESH_LEN,
42                 MPI_COMM_WORLD, &home->inRequests[isrc]);
43     }
44
45 /*
46  *     We're sending the same buffer to all remote domains, but
47  *     MPI spec says we can't re-use the same buffer for multiple
48  *     asynch sends at the same time, so need to make multiple
49  *     copies of the buffer. (Use the existing buffer as the
50  *     one to be sent to the first remote domain)
51  */
52     if (numRemoteDomains > 0) {
53         int i;
54
55         sendBuf = (char **)malloc(numRemoteDomains * sizeof(char *));
56         sendBufLen = (int *)malloc(numRemoteDomains * sizeof(int));
57
58         sendBuf[0] = (char *)home->opListBuf;
59         sendBufLen[0] = outMsgLen;
60
61 /*
62  *     For first remote domain the existing buffer (opList) is used,
63  *     so just need to set up remaining send buffers here.
64  */
65         for (i = 1; i < numRemoteDomains; i++) {
66             sendBuf[i] = (char *)malloc(outMsgLen);
67             sendBufLen[i] = outMsgLen;
68             memcpy(sendBuf[i], home->opListBuf, outMsgLen);
69         }
70     }
71
72 /*
73  *     Send the message length to the receiving neighbors
74  */
75
76     for (idst = 0; idst < numRemoteDomains; idst++) {
77         domainIdx = home->remoteDomains[idst];
78         remDom = home->remoteDomainKeys[domainIdx];
79
80         MPI_Isend(&sendBufLen[idst], 1, MPI_INT, domainIdx, MSG_REMESH_LEN,
81                 MPI_COMM_WORLD, &home->outRequests[idst]);
82     } /* end for (idst = 0; ...) */
83
84 /*

```

```

85  *      Wait for the length sends/receives to complete
86  */
87      MPI_Waitall(numRemoteDomains, home->outRequests, home->outStatus);
88      MPI_Waitall(numRemoteDomains, home->inRequests, home->inStatus);
89
90  /*
91  *      Allocate input buffers and pre-issue receives
92  */
93      for (isrc = 0; isrc < numRemoteDomains; isrc++) {
94          domainIdx = home->remoteDomains[isrc];
95          remDom = home->remoteDomainKeys[domainIdx];
96
97          remDom->inBuf = (char *) malloc(remDom->inBufLen);
98          MPI_Irecv(remDom->inBuf, remDom->inBufLen, MPI_BYTE, domainIdx,
99                  MSG_REMESH, MPI_COMM_WORLD, &home->inRequests[isrc]);
100      }
101
102  /*
103  *      Send the data
104  */
105      for (idst = 0; idst < home->remoteDomainCount; idst++) {
106          domainIdx = home->remoteDomains[idst];
107          remDom = home->remoteDomainKeys[domainIdx];
108
109          MPI_Isend(sendBuf[idst], sendBufLen[idst], MPI_BYTE, domainIdx,
110                  MSG_REMESH, MPI_COMM_WORLD, &home->outRequests[idst]);
111      } /* end for (idst = 0; ...) */
112
113  /*
114  *      Wait for all traffic to complete
115  */
116      MPI_Waitall(numRemoteDomains, home->outRequests, home->outStatus);
117      MPI_Waitall(numRemoteDomains, home->inRequests, home->inStatus);
118
119  /*
120  *      Free any temporary buffers.  Note: first sendBuf points to the
121  *      actual <opList> which gets cleared elsewhere in the code, so
122  *      don't free that one here.
123  */
124      if (numRemoteDomains > 0) {
125          int i;
126
127          for (i = 1; i < numRemoteDomains; i++) {
128              free(sendBuf[i]);
129          }
130
131          free(sendBuf);
132          free(sendBufLen);
133      }
134  #endif
135      return;
136  }

```

Line-by-line explanation.

Lines 23–32

The routine finds remote neighbor count and current byte length of the local operation log.

Lines 33–43

In a parallel build, nonblocking receives for every neighbor's message length are posted first. This avoids send/send deadlock.

Lines 45–70

MPI cannot safely use one buffer for multiple nonblocking sends. The code retains the original for one receiver and clones it for every other remote domain.

Lines 72–100

Message lengths are exchanged and incoming byte buffers are allocated before full record streams are received.

Lines 102–117

Actual streams are sent/received asynchronously; all requests complete before return.

Lines 119–135

Temporary outgoing clones are released. The primary `home->opListBuf` remains owned by `Home_t`; serial builds make the network part a no-op.

6.3 Replaying neighbouring operations

```
25 void FixRemesh(Home_t *home)
26 {
27     int      idom;
28     Param_t *param;
29
30     /*
31     * Loop through the neighbor domains
32     */
33     param = home->param;
34
35     for (idom = 0; idom < home->remoteDomainCount; idom++) {
36         int      offSet, domIdx, recvOpListLen;
37         char      *recvOpListBuf;
38         Node_t     *node1, *node2;
39         RemoteDomain_t *remDom;
40
41         domIdx = home->remoteDomains[idom];
42         remDom = home->remoteDomainKeys[domIdx];
43
44         recvOpListBuf = (char *)remDom->inBuf;
45         recvOpListLen = remDom->inBufLen;
46
47         /*
48         * Loop through the operations from this neighbor
49         */
50         offSet = 0;
51
52         while (offSet < recvOpListLen) {
53             int opType;
54
55             memcpy(&opType, &recvOpListBuf[offSet], sizeof(int));
56
57             if (opType == REM_OP_CHANGE_CONN) {
58                 int opLen = sizeof(RemOpChangeConn_t);
59                 RemOpChangeConn_t opBuf;
60
61                 /*
```

```

62  *           Change the node1->node2 link so it becomes
63  *           a node1->node3 link instead.
64  */
65      memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
66      offSet += opLen;
67
68      node1 = GetNodeFromTag(home, opBuf.tag1);
69
70      if (node1 != NULL) {
71          ChangeConnection (home, node1, &opBuf.tag2, &opBuf.tag3, 0);
72      }
73
74  } else if (opType == REM_OP_INSERT_ARM) {
75      int          opLen = sizeof(RemOpInsertArm_t);
76      RemOpInsertArm_t opBuf;
77
78  /*
79  *           Add an arm from node1 to node2
80  */
81      memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
82      offSet += opLen;
83
84      node1 = GetNodeFromTag(home, opBuf.tag1);
85
86      if (node1 != NULL) {
87          InsertArm(home, node1, &opBuf.tag2,
88                  opBuf.burg[0], opBuf.burg[1], opBuf.burg[2],
89                  opBuf.plane[0], opBuf.plane[1], opBuf.plane[2], 0);
90      }
91
92  } else if (opType == REM_OP_SPLIT_NODE) {
93      int          opLen = sizeof(RemOpSplitNode_t);
94      real8        cellSize[3], cellIndex[3];
95      RemOpSplitNode_t opBuf;
96
97  /*
98  *           Create a ghost node with the necessary domain
99  *           and index, and set the coordinates and velocity
100  *           appropriately. Note: the node initially has
101  *           no arms, they will be added later.
102  */
103      memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
104      offSet += opLen;
105
106      node2 = GetNewGhostNode(home,
107                             opBuf.tag2.domainID,
108                             opBuf.tag2.index);
109      FreeNodeArms(node2);
110
111      SET_CONSTRAINTS(node2->constraint, UNCONSTRAINED);
112
113      node2->x = opBuf.newCoord[0];
114      node2->y = opBuf.newCoord[1];
115      node2->z = opBuf.newCoord[2];
116
117      node2->oldx = opBuf.newCoord[0];
118      node2->oldy = opBuf.newCoord[1];
119      node2->oldz = opBuf.newCoord[2];
120

```

```

121         node2->vX = opBuf.newVelocity[0];
122         node2->vY = opBuf.newVelocity[1];
123         node2->vZ = opBuf.newVelocity[2];
124
125     /*
126     *           Insure the node is assigned to the proper cell
127     */
128
129     cellSize[X] = param->Lx / param->nXcells;
130     cellSize[Y] = param->Ly / param->nYcells;
131     cellSize[Z] = param->Lz / param->nZcells;
132
133     cellIndex[X] = (int)(node2->x-param->minSideX) /
134                    (int)cellSize[X];
135
136     cellIndex[Y] = (int)(node2->y-param->minSideY) /
137                    (int)cellSize[Y];
138
139     cellIndex[Z] = (int)(node2->z-param->minSideZ) /
140                    (int)cellSize[Z];
141
142     cellIndex[X] = MAX(0, cellIndex[X]);
143     cellIndex[Y] = MAX(0, cellIndex[Y]);
144     cellIndex[Z] = MAX(0, cellIndex[Z]);
145
146     cellIndex[X] = MIN(param->nXcells, cellIndex[X]);
147     cellIndex[Y] = MIN(param->nYcells, cellIndex[Y]);
148     cellIndex[Z] = MIN(param->nZcells, cellIndex[Z]);
149
150     cellIndex[X]++;
151     cellIndex[Y]++;
152     cellIndex[Z]++;
153
154     node2->cellIdx = EncodeCellIdx(home,
155                                   cellIndex[X],
156                                   cellIndex[Y],
157                                   cellIndex[Z]);

```

```

158     } else if (opType == REM_OP_MARK_FORCE_OBSOLETE) {
159         int opLen = sizeof(RemOpMarkForceObsolete_t);
160         RemOpMarkForceObsolete_t opBuf;
161
162     /*
163     *           Mark a node so that new force and velocity
164     *           values will be computed for the node. Probably
165     *           because topology changes in another domain have
166     *           affected this node.
167     */
168
169     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
170     offSet += opLen;
171
172     node1 = GetNodeFromTag(home, opBuf.tag1);
173
174     if (node1 != (Node_t *)NULL) {
175         node1->flags |= NODE_RESET_FORCES;
176     }
177
178     } else if (opType == REM_OP_RESET_SEG_FORCE1) {
179         int opLen = sizeof(RemOpResetSegForce1_t);

```

```

179         RemOpResetSegForce1_t opBuf;
180
181     /*
182     *
183     */
184     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
185     offSet += opLen;
186
187     node1 = GetNodeFromTag(home, opBuf.tag1);
188
189     if (node1 != (Node_t *)NULL) {
190
191         ResetSegForces(home, node1, &opBuf.tag2,
192             opBuf.f1[0], opBuf.f1[1], opBuf.f1[2], 0);
193     }
194
195 } else if (opType == REM_OP_RESET_SEG_FORCE2) {
196     int opLen = sizeof(RemOpResetSegForce2_t);
197     RemOpResetSegForce2_t opBuf;
198
199     /*
200     *
201     */
202     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
203     offSet += opLen;
204
205     node1 = GetNodeFromTag(home, opBuf.tag1);
206
207     if (node1 != (Node_t *)NULL) {
208
209         ResetSegForces2(home, node1, &opBuf.tag2,
210             opBuf.f1[0], opBuf.f1[1], opBuf.f1[2],
211             opBuf.f2[0], opBuf.f2[1], opBuf.f2[2], 0);
212     }
213
214 } else if (opType == REM_OP_RESET_COORD) {
215     int opLen = sizeof(RemOpResetCoord_t);
216     RemOpResetCoord_t opBuf;
217
218     /*
219     *
220     */
221     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
222     offSet += opLen;
223
224     node1 = GetNodeFromTag(home, opBuf.tag1);
225
226     if (node1 != (Node_t *)NULL) {
227
228         node1->x = opBuf.newCoord[0];
229         node1->y = opBuf.newCoord[1];
230         node1->z = opBuf.newCoord[2];
231     }
232
233 } else if (opType == REM_OP_RESET_CONSTRAINT) {
234     int opLen = sizeof(RemOpResetConstraint_t);
235     RemOpResetConstraint_t opBuf;
236
237     /*

```

```

238 *           Reset the constraints of the specified node.
239 */
240     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
241     offSet += opLen;
242
243     node1 = GetNodeFromTag(home, opBuf.tag1);
244
245     if (node1 != (Node_t *)NULL) {
246         SET_CONSTRAINTS(node1->constraint, opBuf.constraint);
247     }
248
249 } else if (opType == REM_OP_REMOVE_NODE) {
250     int             opLen = sizeof(RemOpRemoveNode_t);
251     RemOpRemoveNode_t opBuf;
252
253 /*
254 *           Remove the specified node
255 */
256     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
257     offSet += opLen;
258
259     node1 = GetNodeFromTag(home, opBuf.tag1);
260
261     if (node1 != (Node_t *)NULL) {
262         RemoveNode(home, node1, 0);
263     }
264
265 } else if (opType == REM_OP_CHANGE_BURG) {
266     int             opLen = sizeof(RemOpChangeBurg_t);
267     RemOpChangeBurg_t opBuf;
268
269 /*
270 *           Change the burgers vector of a node's arm
271 *           to the given burgers vector. If the new
272 *           burgers vector is zero, the arm will be
273 *           removed.
274 */
275     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
276     offSet += opLen;
277
278     node1 = GetNodeFromTag(home, opBuf.tag1);
279
280     if (node1 != NULL) {
281
282         ChangeArmBurg(home, node1, &opBuf.tag2, opBuf.newBurg[0],
283                     opBuf.newBurg[1], opBuf.newBurg[2],
284                     opBuf.newPlane[0], opBuf.newPlane[1],
285                     opBuf.newPlane[2], 0, DEL_SEG_NONE);
286     }
287
288 } else if (opType == REM_OP_RESET_PLANE) {
289     int             opLen = sizeof(RemOpResetPlane_t);
290     RemOpResetPlane_t opBuf;
291
292 /*
293 *           Reset the glide plane for a specific segment.
294 */
295     memcpy(&opBuf, &recvOpListBuf[offSet], opLen);
296     offSet += opLen;

```

```

297         ResetGlidePlane(home, opBuf.newPlane, &opBuf.tag1,
298                         &opBuf.tag2, 0);
299
300     } else {
301         Fatal("FixRemesh : invalid Remesh operation");
302     }
303 }
304
305 } // end while (offSet < recvOpListLen)
306
307 /*
308  *   We're done with this domain's opList; release it
309  */
310     free(remDom->inBuf);
311 }
312
313 return;

```

Line-by-line explanation.

Lines 25–55

For every remote domain, `FixRemesh()` reads its received buffer from offset zero until the advertised length is consumed. The first integer chooses a record format.

Lines 57–90

Connection-change and insert-arm records are copied from the stream, the offset advances by `sizeof(record)`, and corresponding local endpoints are changed with `globalOp=0`. Replay must not append a new operation recursively.

Lines 92–157

A split record creates a ghost node at the specified tag, coordinates, and velocity, frees stale arms, and assigns a spatial cell. Later records add arms/connections. Producer ordering therefore matters: split precedes records that refer to its new tag.

Lines 158–212

Force-obsolete and force-reset records preserve force consistency. They are not cosmetic; later geometric selection may refuse a segment when forces are stale.

Lines 214–247

Coordinate and constraint records change persistent state. Constraint replay is how `CORNER_NODE` survives domain boundaries.

Lines 249–299

Node removal, Burgers/arm changes, and glide-plane reset records invoke the same low-level helpers used for local work, with logging disabled.

Lines 301–313

An unknown opcode is fatal because safe byte-offset recovery is impossible. Once a domain's stream is replayed its buffer is released.

7 Remeshing

7.1 Dispatcher and postconditions

```

1172 void Remesh(Home_t *home)
1173 {
1174     int    i;

```

```

1175     Node_t *node;
1176     Param_t *param;
1177
1178     param = home->param;
1179
1180     if (param->remeshRule < 0) return;
1181
1182     #ifdef PARALLEL
1183     #ifdef SYNC_TIMERS
1184         TimerStart(home, REMESH_START_BARRIER);
1185         MPI_Barrier(MPI_COMM_WORLD);
1186         TimerStop(home, REMESH_START_BARRIER);
1187     #endif
1188     #endif
1189
1190     TimerStart(home, REMESH);
1191     ClearOpList(home);
1192     InitTopologyExemptions(home);
1193
1194     switch(param->remeshRule) {
1195     case 2:
1196         RemeshRule_2(home);
1197         break;
1198     case 3:
1199         RemeshRule_3(home);
1200         break;
1201     default:
1202         Fatal("Remesh: undefined remesh rule %d", param->remeshRule);
1203         break;
1204     }
1205     TimerStop(home, REMESH);
1206
1207     #ifdef _ARLFEM
1208     /*
1209     *     If free-surfaces are in use rather than PBC, it is possible that
1210     *     some of the remesh operations have resulted in formation of
1211     *     segments between two nodes on the same surface. Any such
1212     *     segments must be cut here... This may orphan nodes, but those
1213     *     should be taken care of by the RemoveOrphanedNodes() call further
1214     *     on.
1215     */
1216     CutSurfaceSegments(home);
1217     #endif /* ifdef _ARLFEM */
1218
1219     /*
1220     *     Send to the neighboring domains, a list of all local
1221     *     operations (add/delete nodes, relinking of arms,
1222     *     etc) that may affect nodes in remote domains, and process
1223     *     the remesh operations from the neighboring domains
1224     */
1225     TimerStart(home, SEND_REMESH);
1226     CommSendRemesh(home);
1227     TimerStop(home, SEND_REMESH);
1228
1229     TimerStart(home, FIX_REMESH);
1230     FixRemesh(home);
1231     TimerStop(home, FIX_REMESH);
1232
1233     /*

```

```

1234 *      Under certain circumstances, parallel topological changes
1235 *      can create double links between nodes; links which can not be
1236 *      detected until after FixRemesh() is called... so, a quick
1237 *      check has to be done to clean up these potential double-
1238 *      links here, or they will cause problems later on. Should
1239 *      only have to check nodes local to this domain.
1240 */
1241     for (i = 0; i < home->newNodeKeyPtr; i++) {
1242         if ((node = home->nodeKeys[i]) == (Node_t *)NULL) continue;
1243         (void)RemoveDoubleLinks(home, node, 0);
1244         node->flags &= ~NODE_CHK_DBL_LINK;
1245     }
1246
1247 /*
1248 *      It is possible that remesh/coarsen has left an orphaned node.
1249 *      We need to get rid of any such nodes before the exchange of
1250 *      node information in CommSendGhosts().
1251 */
1252     RemoveOrphanedNodes(home);
1253
1254 #ifdef PARALLEL
1255 #ifdef SYNC_TIMERS
1256     TimerStart(home, REMESH_END_BARRIER);
1257     MPI_Barrier(MPI_COMM_WORLD);
1258     TimerStop(home, REMESH_END_BARRIER);
1259 #endif
1260 #endif
1261
1262 /*
1263 *      If memory debugging is enabled, run a consistency check on all
1264 *      allocated memory blocks looking for corruption in any of the
1265 *      block headers or trailers.
1266 */
1267 #ifdef DEBUG_MEM
1268     ParadisMemCheck();
1269 #endif
1270
1271     return;
1272 }

```

Line-by-line explanation.

Lines 1174–1181

A negative `remeshRule` disables this pass. Otherwise the dispatcher obtains parameters and may synchronize timer measurement at the start.

Lines 1190–1204

It starts the remesh timer, clears a fresh operation log, resets per-cycle exemptions, and dispatches rule 2 or rule 3. Any other nonnegative rule is fatal rather than silently selecting an algorithm.

Lines 1207–1217

Free-surface builds remove a segment that joins two nodes on the same surface.

Lines 1219–1231

The second operation-log exchange propagates all remesh splits/merges and replays remote changes.

Lines 1233–1245

Independent remote changes may create a duplicate link. Every local node is checked after replay.

Lines 1247–1252

Orphaned nodes are removed before the next ghost-node exchange. This is the final graph-cleanliness postcondition.

Lines 1254–1271

Optional timing/memory checks close the pass.

7.2 Force estimates used during topology changes

```
132 void EstRefinementForces(Home_t *home, Node_t *node1, Node_t *node2,
133                          real8 newPos[3], real8 vec[3],
134                          real8 f0Seg1[3], real8 f1Seg1[3],
135                          real8 f0Seg2[3], real8 f1Seg2[3])
136 {
137     int    arm12, arm21;
138     real8  p1[3], p2[3], oldfp1[3], oldfp2[3], burg[3];
139
140     arm12 = GetArmID(node1, node2);
141     arm21 = GetArmID(node2, node1);
142
143     burg[X] = node1->burgX[arm12];
144     burg[Y] = node1->burgY[arm12];
145     burg[Z] = node1->burgZ[arm12];
146
147     oldfp1[X] = node1->armfx[arm12];
148     oldfp1[Y] = node1->armfy[arm12];
149     oldfp1[Z] = node1->armfz[arm12];
150
151     oldfp2[X] = node2->armfx[arm21];
152     oldfp2[Y] = node2->armfy[arm21];
153     oldfp2[Z] = node2->armfz[arm21];
154
155     p1[X] = node1->x;
156     p1[Y] = node1->y;
157     p1[Z] = node1->z;
158
159     p2[X] = p1[X] + vec[X];
160     p2[Y] = p1[Y] + vec[Y];
161     p2[Z] = p1[Z] + vec[Z];
162
163     FindSubFSeg(home, p1, p2, burg, oldfp1, oldfp2, newPos, f0Seg1,
164                f1Seg1, f0Seg2, f1Seg2);
165
166     return;
167 }
```

```
187 void EstCoarsenForces(Home_t *home, Node_t *node1, Node_t *node2,
188                      Node_t *node3, real8 f0Seg[3], real8 f1Seg[3])
189 {
190     int    i, arm21, arm23, arm12, arm32;
191     real8  burg1[3], burg2[3];
192     real8  p1[3], p2[3], p3[3];
193     real8  fp0[3], fp1[3], fp2[3], fp3[3];
194     real8  len1[3], len2[3];
195     Param_t *param;
196
197     param = home->param;
```

```

198
199     arm21 = GetArmID(node2, node1);
200     arm23 = GetArmID(node2, node3);
201     arm12 = GetArmID(node1, node2);
202     arm32 = GetArmID(node3, node2);
203
204     p1[X] = node1->x; p2[X] = node2->x; p3[X] = node3->x;
205     p1[Y] = node1->y; p2[Y] = node2->y; p3[Y] = node3->y;
206     p1[Z] = node1->z; p2[Z] = node2->z; p3[Z] = node3->z;
207
208 /*
209  *   If periodic boundaries are enabled, the nodes may
210  *   be on opposite side of the problem space, so adjust
211  *   the positions accordingly.
212  */
213     PBCPOSITION(param, p1[X], p1[Y], p1[Z], &p2[X], &p2[Y], &p2[Z]);
214     PBCPOSITION(param, p1[X], p1[Y], p1[Z], &p3[X], &p3[Y], &p3[Z]);
215
216     burg1[X] = node1->burgX[arm12];
217     burg1[Y] = node1->burgY[arm12];
218     burg1[Z] = node1->burgZ[arm12];
219
220     burg2[X] = node2->burgX[arm23];
221     burg2[Y] = node2->burgY[arm23];
222     burg2[Z] = node2->burgZ[arm23];
223
224     fp0[X] = node1->armfx[arm12];
225     fp0[Y] = node1->armfy[arm12];
226     fp0[Z] = node1->armfz[arm12];
227
228     fp1[X] = node2->armfx[arm21];
229     fp1[Y] = node2->armfy[arm21];
230     fp1[Z] = node2->armfz[arm21];
231
232     fp2[X] = node2->armfx[arm23];
233     fp2[Y] = node2->armfy[arm23];
234     fp2[Z] = node2->armfz[arm23];
235
236     fp3[X] = node3->armfx[arm32];
237     fp3[Y] = node3->armfy[arm32];
238     fp3[Z] = node3->armfz[arm32];
239
240 /*
241  *   It is possible that we'll have to deal with extremely short
242  *   segments here (zero length segments created during collisions and
243  *   short segments being coarsened out). Since the FindFSegComb()
244  *   function does not handle these cases well, we do special handling
245  *   of them here.
246  *
247  *   If we find a zero-length segment, the force on a segment between
248  *   nodes 1 and 3 would equal the force on the non-zero length segment
249  *   attached to node 2. Otherwise, use FindFSegComb() to get the force.
250  */
251     for (i = 0; i < 3; i++) {
252         len1[i] = p1[i] - p2[i];
253         len2[i] = p2[i] - p3[i];
254     }
255
256 /*

```

```

257 *      Segments less than .01b in length are treated as zero length
258 */
259     if ((len1[X]*len1[X] + len1[Y]*len1[Y] + len1[Z]*len1[Z]) < 1.0e-4) {
260         f0Seg[X] = node2->armfx[arm23];
261         f0Seg[Y] = node2->armfy[arm23];
262         f0Seg[Z] = node2->armfz[arm23];
263         f1Seg[X] = node3->armfx[arm32];
264         f1Seg[Y] = node3->armfy[arm32];
265         f1Seg[Z] = node3->armfz[arm32];
266     } else if ((len2[X]*len2[X]+len2[Y]*len2[Y]+len2[Z]*len2[Z])<1.0e-4) {
267         f0Seg[X] = node1->armfx[arm12];
268         f0Seg[Y] = node1->armfy[arm12];
269         f0Seg[Z] = node1->armfz[arm12];
270         f1Seg[X] = node2->armfx[arm21];
271         f1Seg[Y] = node2->armfy[arm21];
272         f1Seg[Z] = node2->armfz[arm21];
273     } else {
274         FindFSegComb(home, p1, p2, p3, burg1, burg2,
275                     fp0, fp1, fp2, fp3, f0Seg, f1Seg);
276     }
277
278     return;
279 }
280
281
282 #ifdef _ARLFEM
283 /*-----
284 *
285 *      Function:      CutSurfaceSegments
286 *      Description:   Loop through all the local nodes looking for
287 *                     surface nodes with connections to other nodes
288 *                     on the same surface.
289 *
290 *-----*/

```

Line-by-line explanation.

Lines 132–167

`EstRefinementForces()` finds the existing arm indices, copies Burgers data and endpoint arm forces, represents node 2 in the same periodic image as node 1, then calls `FindSubFSeg()` to estimate four endpoint forces on the two child segments.

Lines 187–224

`EstCoarsenForces()` finds both old arm IDs and makes all three nodes periodic-image consistent. It copies Burgers and force data needed for a combined future segment.

Lines 225–290

Very short/zero arms are treated specially because `FindFSegComb()` is not stable for them. Otherwise it estimates endpoint forces for the one segment that replaces two old segments.

These are estimates, not a substitute for full force calculation. Later cleanup calls `MarkNodeForceObsolete()` so accurate forces are calculated at the next appropriate force-update point.

7.3 Mesh coarsening: selecting a removal

```

378 void MeshCoarsen(Home_t *home)

```

```

379 {
380 #ifdef DEBUG_TOPOLOGY_DOMAIN
381     dbgDom = DEBUG_TOPOLOGY_DOMAIN;
382 #else
383     dbgDom = -1;
384 #endif
385
386     int      thisDomain = home->myDomain;
387     Param_t *param      = home->param;
388
389     int      allowFuzzyPlanes = 0;
390
391     real8 cellLength = home->param->Lx / home->param->nXcells;
392     cellLength = MIN(cellLength, home->param->Ly / home->param->nYcells);
393     cellLength = MIN(cellLength, home->param->Lz / home->param->nZcells);
394
395     real8 cutoffLength1 = param->maxSeg;
396     real8 cutoffLength2 = MIN(cutoffLength1, 0.45 * cellLength);
397
398     real8 areaMin      = param->remeshAreaMin;
399     real8 areaMin2     = areaMin * areaMin;
400     real8 delta        = 1.0e-16;
401
402     int    localCoarsenCnt=0;
403
404     /*
405     *   Allocate a block of structures to hold info about rediscretization
406     *   events. We allocate enough structures to refine coarsen twice as
407     *   many segments as there are nodes which is overkill, but it means we
408     *   shouldn't run out of entries and have to reallocate the block,
409     *   plus minimizes the number of operations we'll have to do within
410     *   the thread-critical sections of code.
411     */
412     int      coarsenListSize = 2 * home->newNodeKeyPtr;
413     int      coarsenListEnts = 0;
414     CoarsenList_t *coarsenList = (CoarsenList_t *)NULL;
415
416     if (coarsenListSize > 0) {
417         coarsenList = (CoarsenList_t *)malloc(sizeof(CoarsenList_t) *
418                                             coarsenListSize);
419     }
420
421     /*
422     *   Check for various conditions that will exempt a node from
423     *   removal:
424     *
425     *   1) does not have exactly 2 arms
426     *   2) node is a 'pinned' node
427     *   3) node is flagged as exempt from coarsen operations
428     *   4) current domain does not 'own' at least one of the segments
429     *   5) If the node's arms are on different glide planes we might
430     *       not allow the node to be removed.
431     *
432     *   NOTE: For now, a node that is pinned in ANY dimension is
433     *           treated here as if it is pinned in ALL dimensions.
434     *           This may change in the future
435     */
436     if ((node->numNbrs != 2)

```

```

||

```

```

476         HAS_ANY_OF_CONSTRAINTS(node->constraint, PINNED_NODE) ||
477         (node->flags & NO_MESH_COARSEN)) {
478             continue;
479         }
480
481         nbr1 = GetNeighborNode(home, node, 0);
482         nbr2 = GetNeighborNode(home, node, 1);
483
484         if ((nbr1 == (Node_t *)NULL) || (nbr2 == (Node_t *)NULL)) {
485             printf("WARNING: Neighbor not found at %s line %d\n",
486                 __FILE__, __LINE__);
487             continue;
488         }
489
490         domOwnsSeg1 = DomainOwnsSeg(home, OPCLASS_REMESH, thisDomain, &nbr1->myTag);
491         domOwnsSeg2 = DomainOwnsSeg(home, OPCLASS_REMESH, thisDomain, &nbr2->myTag);
492
493         if ((!domOwnsSeg1) && (!domOwnsSeg2)) {
494             continue;
495         }
496
497         nbr1IsRemote = (node->myTag.domainID != nbr1->myTag.domainID);
498         nbr2IsRemote = (node->myTag.domainID != nbr2->myTag.domainID);
499
500         hasRemoteNbr = nbr1IsRemote | nbr2IsRemote;
501
502         /*
503          *
504          * Calculate the lengths of the node's 2 arms plus
505          * the distance between the two neighbor nodes.
506          *
507          * If periodic boundaries are enabled, the nodes may
508          * be on opposite side of the problem space, so adjust
509          * the lengths/distances accordingly.
510          */
511
512         vec1x = nbr1->x - node->x;
513         vec1y = nbr1->y - node->y;
514         vec1z = nbr1->z - node->z;
515
516         vec2x = nbr2->x - node->x;
517         vec2y = nbr2->y - node->y;
518         vec2z = nbr2->z - node->z;
519
520         vec3x = vec2x - vec1x;
521         vec3y = vec2y - vec1y;
522         vec3z = vec2z - vec1z;
523
524         ZImage(param, &vec1x, &vec1y, &vec1z);
525         ZImage(param, &vec2x, &vec2y, &vec2z);
526         ZImage(param, &vec3x, &vec3y, &vec3z);
527
528         r1 = sqrt(vec1x*vec1x + vec1y*vec1y + vec1z*vec1z);
529         r2 = sqrt(vec2x*vec2x + vec2y*vec2y + vec2z*vec2z);
530         r3 = sqrt(vec3x*vec3x + vec3y*vec3y + vec3z*vec3z);
531
532         /*
533          * Preserve the physical corner created by a three-arm binary
534          * junction split. As in OpenDiS, permit coarsening only when
535          * an attached segment is very short so pathological segments
536          * can still be removed.

```

```

535 */
536         if ((HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
537             HAS_ANY_OF_CONSTRAINTS(nbr1->constraint, CORNER_NODE)) &&
538             (r1 > (2.0 * param->rann)) &&
539             (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
540             HAS_ANY_OF_CONSTRAINTS(nbr2->constraint, CORNER_NODE)) &&
541             (r2 > (2.0 * param->rann))) {
542             continue;
543         }
544
545 /*
546  *
547  *   If we are enforcing the use of segment glide planes, we do
548  *   not want to coarsen out a node whose arms are on different
549  *   glide planes.
550  *
551  *   However, when glide planes constraints are being enforced, we
552  *   tend to get a lot of debris (i.e. tiny triangles or quadrangles,
553  *   really short segments, etc) that oscillate quickly and adversely
554  *   affect the timestep. So, under the following circumstances,
555  *   we'll allow the code to violate glide plane constraints.
556  *
557  *   1) If the node has an attached segment less than 1b in length
558  *   2) If the node has two segments less than 20% of the minimum
559  *       segment length AND both of the node's neighbors are connected
560  *       to each other (i.e. 3 nodes form a triangle)
561  *   3) If the glide planes are allowed to be 'fuzzy' add
562  *       a few extra exceptions (see below)
563  */
564
565         if (param->enforceGlidePlanes) {
566             int connectionID, violateGlidePlanesOK = 0;
567
568             if ((r1 < 1.0) ||
569                 (r2 < 1.0) ||
570                 ((MAX(r1, r2) < MAX(1.0, 0.20 * param->minSeg) &&
571                  (Connected(nbr1, nbr2, &connectionID)))) {
572                 violateGlidePlanesOK = 1;
573             }
574
575             gp0[0] = node->nx[0];
576             gp0[1] = node->ny[0];
577             gp0[2] = node->nz[0];
578
579             gp1[0] = node->nx[1];
580             gp1[1] = node->ny[1];
581             gp1[2] = node->nz[1];
582
583 /*
584  *
585  *   Glide planes are being used, but if we are allowing
586  *   some 'fuzziness' in the planes there are a couple
587  *   situations in which we allow glide plane constraints
588  *   to be violated.
589  *
590  *   1) the two segment glide planes are within a small
591  *       number of degrees
592  *   2) If either segment is shorter than the annihilation
593  *       distance
594  *   3) when each segment is mapped to the closest
595  *       precise glide plane, if the two precise planes are
596  *       the same

```

```

594  */
595      if (param->allowFuzzyGlidePlanes) {
596
597          if (fabs(DotProduct(gp0, gp1)) > 0.9555) {
598              violateGlidePlanesOK = 1;
599          } else if ((r1 < param->rann) || (r2 < param->rann)) {
600              violateGlidePlanesOK = 1;
601          } else {
602              real8 burg1[3], burg2[3];
603              real8 lineDir1[3], lineDir2[3];
604              real8 testPlane1[3], testPlane2[3];
605
606              burg1[X] = node->burgX[0];
607              burg1[Y] = node->burgY[0];
608              burg1[Z] = node->burgZ[0];
609
610              burg2[X] = node->burgX[1];
611              burg2[Y] = node->burgY[1];
612              burg2[Z] = node->burgZ[1];
613
614              lineDir1[X] = vec1x;
615              lineDir1[Y] = vec1y;
616              lineDir1[Z] = vec1z;
617
618              lineDir2[X] = vec2x;
619              lineDir2[Y] = vec2y;
620              lineDir2[Z] = vec2z;
621
622          /*
623           * FindPreciseGlidePlanes() just uses l cross b if
624           * fuzzy planes are allowed, in this case we've
625           * set 'allowFuzzyPlanes' to false to explicitly
626           * disallow the use of 'fuzzy' planes so we truly
627           * find the closest precise plane.
628           */
629
630              FindPreciseGlidePlane(home, burg1, lineDir1,
631                                  testPlane1, allowFuzzyPlanes);
632              FindPreciseGlidePlane(home, burg2, lineDir2,
633                                  testPlane2, allowFuzzyPlanes);
634
635              if (fabs(DotProduct(testPlane1, testPlane2)) > 0.99) {
636                  violateGlidePlanesOK = 1;
637              }
638          }
639      } /* end if (param->allowFuzzyGlidePlanes) */
640
641      if (!violateGlidePlanesOK) {
642          real8 tmp3[3];
643
644          cross(gp0, gp1, tmp3);
645
646          if (fabs(DotProduct(tmp3, tmp3)) > 1.0e-3) {
647              continue;
648          }
649      }
650
651      } /* end if (param->enforceGlidePlanes) */
652

```

```

653  /*
654  *      If coarsening out a node would leave a segment longer
655  *      than a defined length, the node should not be removed.
656  *      This 'cutoff length' is the maximum segment length
657  *      if all involved nodes are within the same domain, but
658  *      if a remote node is involved, we need to set the
659  *      cutoff length to (at most) 1/2 the cell length. This
660  *      is needed because the remote node may potentially be involved
661  *      in a simultaneous mesh coarsening in the remote domain,
662  *      and although the node would not be removed, it could
663  *      be repositioned resulting in a segment that spanned
664  *      more than 2 cells... this is a bad thing.
665  */
666
667  if (((hasRemoteNbr == 0) && (r3 > cutoffLength1)) ||
668      ((hasRemoteNbr == 1) && (r3 > cutoffLength2))) {
669      continue;
670  }
671
672  /*
673  *      Check if the area of the triangle defined by node
674  *      and its two neighbors, plus determine if that area
675  *      is increasing or decreasing.
676  */
677
678  s = 0.5 * (r1 + r2 + r3);
679  area2 = (s * (s-r1) * (s-r2) * (s-r3));
680
681  dvec1xdt = nbr1->vX - node->vX;
682  dvec1ydt = nbr1->vY - node->vY;
683  dvec1zdt = nbr1->vZ - node->vZ;
684
685  dvec2xdt = nbr2->vX - node->vX;
686  dvec2ydt = nbr2->vY - node->vY;
687  dvec2zdt = nbr2->vZ - node->vZ;
688
689  dvec3xdt = dvec2xdt - dvec1xdt;
690  dvec3ydt = dvec2ydt - dvec1ydt;
691  dvec3zdt = dvec2zdt - dvec1zdt;
692
693  dr1dt = ((vec1x * dvec1xdt) + (vec1y * dvec1ydt) +
694           (vec1z * dvec1zdt)) / (r1 + delta);
695
696  dr2dt = ((vec2x * dvec2xdt) + (vec2y * dvec2ydt) +
697           (vec2z * dvec2zdt)) / (r2 + delta);
698
699  dr3dt = ((vec3x * dvec3xdt) + (vec3y * dvec3ydt) +
700           (vec3z * dvec3zdt)) / (r3 + delta);
701
702  dsdt = 0.5 * (dr1dt + dr2dt + dr3dt);
703
704  darea2dt = (dsdt * (s-r1) * (s-r2) * (s-r3));
705  darea2dt += s * (dsdt-dr1dt) * (s-r2) * (s-r3);
706  darea2dt += s * (s-r1) * (dsdt-dr2dt) * (s-r3);
707  darea2dt += s * (s-r1) * (s-r2) * (dsdt-dr3dt);
708
709  /*
710  *      If the area is less than the specified minimum and shrinking,
711  *      or one of the arms is less than the minimum segment length, the
712  *      node should be removed, so add the node tag (and its neighbor

```



```

712 *           tags to the 'coarsen' list.
713 */
714 if (((area2 < areaMin2) && (darea2dt < 0.0)) ||
715     ((r1 < param->minSeg) || (r2 < param->minSeg))) {
716 /*
717 *           If the node to be removed has a segment owned by a remote
718 *           node, we can't coarsen it out. Reason is:
719 *
720 *           Given nodes A, B, C, D connected as A--B--C--D where
721 *           A and B are in domain 1 and C and D are in domain 2 with
722 *           segment B--C owned by domain 1. Domain 1 owns B and
723 *           segment B--C so it removes node B. Domain 2 cannot
724 *           remove node C because it is connected to B via a segment
725 *           owned by domain 1. However, MergeNode() would permit
726 *           nodes C and D to be merged by getting rid of D and
727 *           moving C to D's original location. This breaks no
728 *           ownership rules, but the resulting single segment spans
729 *           the original positions A to D. If the original segments
730 *           were of sufficient length, the resulting segment would
731 *           have a length exceeding <maxSeg> which can cause
732 *           problems...
733 */
734     if (((domOwnsSeg1 == 0) && (nbr1IsRemote)) ||
735         ((domOwnsSeg2 == 0) && (nbr2IsRemote))) {
736         continue;
737     }
738
739 #ifndef _OPENMP
740 #pragma omp critical (CRIT_COARSEN_A)
741 #endif
742     {
743 /*
744 *           Just a quick sanity check, but make sure there is
745 *           still space on the list...
746 */
747         if (coarsenListEnts < coarsenListSize) {
748
749             coarsenList[coarsenListEnts].nodeTag = node->myTag;
750             coarsenList[coarsenListEnts].nbr1Tag = nbr1->myTag;
751             coarsenList[coarsenListEnts].nbr2Tag = nbr2->myTag;
752             coarsenList[coarsenListEnts].status = REMESH_PENDING;
753
754             coarsenListEnts += 1;
755         }
756     } /* end "omp critical" section */
757 }
758

```

Line-by-line explanation of candidate selection.

Lines 378–419

`MeshCoarsen()` derives local/remote maximum-length cutoffs, squared area threshold, and allocates candidate storage. The list allows the initial geometry scan to be parallel while graph changes are serialized safely.

Lines 460–495

A candidate must be degree two, unpinned, not already exempt, have two resolvable neighbors, and

own at least one incident segment.

Lines 497–528

The two arms and future neighbor-to-neighbor chord are computed in minimum-image coordinates as r_1, r_2, r_3 .

Lines 530–543

CORNER_NODE protects a physical junction corner. It can be removed only across a sufficiently short attached segment, avoiding loss of a real kink merely because it is degree two.

Lines 545–651

When glide planes are enforced, the arms normally must share a plane. Exceptions allow debris cleanup for tiny segments, small triangles, fuzzy/close planes, and matching nearest precise planes.

Lines 653–668

The future direct segment cannot exceed `maxSeg`. With a remote neighbor, the stricter half-cell cutoff prevents two independent domain changes creating a segment spanning too many cells.

Lines 671–715

Heron's formula gives `area2`, which is actually A^2 , not twice the area. Its time derivative determines shrinking. A candidate is listed when $A^2 < A_{\min}^2$ and shrinking, or either arm is shorter than `minSeg`.

Lines 717–758

Unsafe remote-ownership cases are rejected. Accepted candidates are appended under a critical section as `REMESH_PENDING`.

7.4 Mesh coarsening: committing nonoverlapping removals

```
764  /*
765  *      Now loop through the list of nodes to be coarsened and remove
766  *      them (if possible)
767  */
768  #ifdef _OPENMP
769  #pragma omp parallel for if (coarsenListEnts > 1)
770  #endif
771      for (int i=0; i < coarsenListEnts; i++)
772      {
773          int    q, proceed;
774          int    mergeNbr1Allowed, mergeNbr2Allowed;
775          int    mergeDone, mergeStatus, globalOp;
776          real8  newPos[3], f0seg1[3], f1seg1[3];
777          Tag_t  nbr1Tag, nbr2Tag;
778          Node_t *node, *nbr1, *nbr2, *mergedNode;
779  #ifdef DEBUG_TOPOLOGY_CHANGES
780          Tag_t  oldTag1, oldTag2, oldTag3;
781  #endif
782
783  /*
784  *      Verify that the neither the node nor its neighbors have
785  *      been involved in a "committed" coarsen event in any other
786  *      thread. This is needed because coarsening a node modifies
787  *      the node structures of both neighboring nodes plus various
788  *      global data structures. We can't risk different threads
789  *      trying to access these data structures simultaneously,
790  *      so we don't allow threads to work on overlapping sets
791  *      of nodes.
792  */
793  #ifdef _OPENMP
```

```

794 #pragma omp critical (CRIT_COARSEN_B)
795 #endif
796 {
797     proceed = ProceedWithCoarsen(coarsenList, coarsenListEnts,
798                                 &coarsenList[i].nodeTag,
799                                 &coarsenList[i].nbr1Tag,
800                                 &coarsenList[i].nbr2Tag);
801     if (proceed) {
802         coarsenList[i].status = REMESH_COMMITTED;
803     } else {
804         coarsenList[i].status = REMESH_REJECTED;
805     }
806
807 } /* end "omp critical" section */
808
809 /*
810 *      Not allowed to jump out of a 'critical' section, so had to
811 *      wait until we were out of the section before checking if we
812 *      need to cycle to the next loop iteration.
813 */
814 if (!proceed) {
815     continue;
816 }
817
818 /*
819 *      Get the node and its neighbors from the 'coarsen' list
820 */
821 node = GetNodeFromTag(home, coarsenList[i].nodeTag);
822 nbr1 = GetNodeFromTag(home, coarsenList[i].nbr1Tag);
823 nbr2 = GetNodeFromTag(home, coarsenList[i].nbr2Tag);
824
825 /*
826 *      It is possible that another coarsen event has resulted
827 *      in the removal of the current node or one of its neighbors
828 *      (due to annihilation). Check for this before attempting
829 *      to remove the current node.
830 *
831 *      Given that the call to ProceedWithCoarsen() succeeded, this
832 *      check *may* not be required.
833 */
834 if ((node == (Node_t *)NULL) ||
835     (nbr1 == (Node_t *)NULL) ||
836     (nbr2 == (Node_t *)NULL)) {
837     continue;
838 }
839
840 /*
841 *      OpenDiS permits a corner node to merge only across a segment no
842 *      longer than twice the annihilation distance. The coarsen list
843 *      contains both neighbors, so gate each merge independently; an
844 *      arbitrary arm ordering must not collapse the longer segment.
845 */
846 mergeNbr1Allowed = 1;
847 mergeNbr2Allowed = 1;
848
849 if (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
850     HAS_ANY_OF_CONSTRAINTS(nbr1->constraint, CORNER_NODE)) {
851     real8 dx1, dy1, dz1;
852     real8 cornerMergeDist2;

```

```

853         dx1 = nbr1->x - node->x;
854         dy1 = nbr1->y - node->y;
855         dz1 = nbr1->z - node->z;
856
857         ZImage(param, &dx1, &dy1, &dz1);
858
859         cornerMergeDist2 = 4.0 * param->rann * param->rann;
860
861         mergeNbr1Allowed = ((dx1*dx1 + dy1*dy1 + dz1*dz1) <=
862                             cornerMergeDist2);
863     }
864
865     if (HAS_ANY_OF_CONSTRAINTS(node->constraint, CORNER_NODE) ||
866         HAS_ANY_OF_CONSTRAINTS(nbr2->constraint, CORNER_NODE)) {
867         real8 dx2, dy2, dz2;
868         real8 cornerMergeDist2;
869
870         dx2 = nbr2->x - node->x;
871         dy2 = nbr2->y - node->y;
872         dz2 = nbr2->z - node->z;
873
874         ZImage(param, &dx2, &dy2, &dz2);
875
876         cornerMergeDist2 = 4.0 * param->rann * param->rann;
877
878         mergeNbr2Allowed = ((dx2*dx2 + dy2*dy2 + dz2*dz2) <=
879                             cornerMergeDist2);
880     }
881
882     if ((!mergeNbr1Allowed) && (!mergeNbr2Allowed)) {
883         continue;
884     }
885
886     EstCoarsenForces(home, nbr1, node, nbr2, f0seg1, f1seg1);
887
888     mergeDone = 0;
889
890     nbr1Tag = nbr1->myTag;
891     nbr2Tag = nbr2->myTag;
892
893     /*
894     *
895     * If either of the neighbor nodes (or any of their neighbors)
896     * is in a remote domain, the operation must be treated as global.
897     * This is necessary to prevent an inconsistent linkage problem.
898     * For example, given the segments A--B--C--D where nodes A, B
899     * and C are in domain 1, D is in domain 2, and segment C--D is
900     * owned by domain 1: domain 1 could coarsen node B into A, then
901     * node C into D. If the first operation was not communicated
902     * to the remote domain, an inconsistency would arise.
903     *
904     * NOTE: It is safe to not distribute the purely local coarsen
905     * operations so long as no other topological operations
906     * are done after Remesh() but before the ghost node
907     * are redistributed.
908     */
909     globalOp = ((nbr1->myTag.domainID != thisDomain) ||
910                (nbr2->myTag.domainID != thisDomain));
911

```

```

912         for (q = 0; q < nbr1->numNbrs; q++) {
913             globalOp |= (nbr1->nbrTag[q].domainID != thisDomain);
914         }
915
916         for (q = 0; q < nbr2->numNbrs; q++) {
917             globalOp |= (nbr2->nbrTag[q].domainID != thisDomain);
918         }
919
920 #ifndef DEBUG_TOPOLOGY_CHANGES
921         oldTag1 = nbr1->myTag;
922         oldTag2 = node->myTag;
923         oldTag3 = nbr2->myTag;
924 #endif
925
926 /*
927  *       The next block of code results in modification to
928  *       multiple node structures and global data structures.
929  *       Access to those data structures is not yet thread-safe,
930  *       so for now we need to insure that only 1 thread at a time
931  *       will be modifying those structures. Note: this will
932  *       result in deletion of 1 or more nodes, but other checks
933  *       above should guarantee that no other thread will be attempting
934  *       to access the same nodes while this is being done.
935  */
936 #ifdef _OPENMP
937 #pragma omp critical (CRIT_COARSEN_C)
938 #endif
939     {
940 /*
941  *       If the first neighbor is not exempt from a coarsen
942  *       operation, attempt to merge the nodes.
943  */
944         if (mergeNbr1Allowed &&
945             ((nbr1->flags & NO_MESH_COARSEN) == 0)) {
946
947             newPos[X] = nbr1->x;
948             newPos[Y] = nbr1->y;
949             newPos[Z] = nbr1->z;
950
951             MergeNode(home, OPCLASS_REMESH, node, nbr1, newPos,
952                     &mergedNode, &mergeStatus, globalOp);
953
954             mergeDone = mergeStatus & MERGE_SUCCESS;
955
956             if (mergeDone) {
957                 localCoarsenCnt++;
958             }
959         }
960
961 /*
962  *       If the merge could not be done, try using
963  *       the other neighbor.
964  */
965         if (mergeDone == 0) {
966             if (mergeNbr2Allowed &&
967                 ((nbr2->flags & NO_MESH_COARSEN) == 0)) {
968                 newPos[X] = nbr2->x;
969                 newPos[Y] = nbr2->y;
970                 newPos[Z] = nbr2->z;

```

```

971         MergeNode(home, OPCLASS_REMESH, node, nbr2, newPos,
972                   &mergedNode, &mergeStatus, globalOp);
973
974
975         mergeDone = mergeStatus & MERGE_SUCCESS;
976
977         if (mergeDone) {
978             localCoarsenCnt++;
979         }
980     }
981 }
982
983 } /* end "omp critical" section */
984
985 /*
986  *      If the merge was successful, update the forces
987  *      on the remaining nodes.  Otherwise go back and
988  *      continue looking for more nodes to coarsen out.
989  */
990     if (mergeDone == 0) continue;
991
992 #ifdef DEBUG_TOPOLOGY_CHANGES
993     if ((dbgDom < 0) || (dbgDom == home->myDomain)) {
994         printf("Coarsen: (%d,%d)--(%d,%d)--(%d,%d)\n",
995               oldTag1.domainID, oldTag1.index,
996               oldTag2.domainID, oldTag2.index,
997               oldTag3.domainID, oldTag3.index);
998     }
999 #endif
1000
1001 /*
1002  *      It is possible coarsening out the node resulted in the
1003  *      annihilation of one or more of the neighboring nodes.
1004  *      if that's the case, skip any more operations on the
1005  *      nodes involved.
1006  */
1007     nbr1 = GetNodeFromTag(home, nbr1Tag);
1008     nbr2 = GetNodeFromTag(home, nbr2Tag);
1009
1010     if ((nbr1 == (Node_t *)NULL) && (nbr2 == (Node_t *)NULL)) {
1011         continue;
1012     }
1013
1014 /*
1015  *      The merge will have placed the resultant node at the
1016  *      location of either nbr1 or nbr2, but the merge function
1017  *      determines which of the two specified nodes is deleted
1018  *      and which survives, which means that nbr1 or nbr2 may
1019  *      have been the deleted and the <node> repositioned to
1020  *      the correct location... so if one of the nbr nodes does
1021  *      not exist anymore, <mergedNode> (if it exists) should
1022  *      be the node that replaced the nbr.
1023  */
1024     if (nbr1 == (Node_t *)NULL) {
1025         nbr1 = mergedNode;
1026     } else if (nbr2 == (Node_t *)NULL) {
1027         nbr2 = mergedNode;
1028     }
1029

```

```

1030 /*
1031 *      At this point, if we don't have a node at the location
1032 *      of at least one of the original nbr nodes, looks like
1033 *      some nodes were orphaned and deleted, so the force
1034 *      estimates we made are not applicable.
1035 */
1036     if ((nbr1 == (Node_t *)NULL) || (nbr2 == (Node_t *)NULL)) {
1037         continue;
1038     }
1039
1040     mergedNode->flags |= NO_MESH_COARSEN;
1041
1042 /*
1043 *      Reset force/velocity for the two remaining nodes
1044 */
1045     ResetSegForces(home, nbr1, &nbr2->myTag, f0seg1[X],
1046                   f0seg1[Y], f0seg1[Z], 1);
1047
1048     ResetSegForces(home, nbr2, &nbr1->myTag, f1seg1[X],
1049                   f1seg1[Y], f1seg1[Z], 1);
1050
1051 } /* end "omp parallel for" section */

```

```

1053 /*
1054 *      Last phase of mesh coarsening is loop once more through
1055 *      the coarsen list and for each 'committed' coarsen, mark
1056 *      the forces for the nodes (and their neighbors as well) at
1057 *      the ends of the newly formed segment as obsolete. This
1058 *      is done because the force estimates above are not accurate
1059 *      enough so we need to recalculate more exact forces for
1060 *      these nodes before (or at the beginning of) the next timestep.
1061 *
1062 *      This is not done in the loop above because we need to
1063 *      access all the neighbors of the nodes forming the new
1064 *      segment, and in the loop above, those other nodes may
1065 *      be involved in a coarsen event in another thread.
1066 *      We didn't want to loop over them until all coarsened nodes
1067 *      have been completely removed and the topology is stable again.
1068 */
1069 #ifndef _OPENMP
1070 #pragma omp parallel for if (coarsenListEnts > 1)
1071 #endif
1072     for (int i=0; i < coarsenListEnts; i++) {
1073         int q;
1074         Node_t *nbr1, *nbr2, *nbr;
1075
1076 /*
1077 *      Only need to do the updates for nodes which were
1078 *      involved in a coarsen event that was committed.
1079 */
1080         if (coarsenList[i].status != REMESH_COMMITTED) {
1081             continue;
1082         }
1083
1084         nbr1 = GetNodeFromTag(home, coarsenList[i].nbr1Tag);
1085         nbr2 = GetNodeFromTag(home, coarsenList[i].nbr2Tag);
1086
1087 /*

```

```

1088 *      Usually, the segment endpoints will be the neighbors of
1089 *      the original node, however, it is possible that the
1090 *      original node to be coarsened out was NOT removed, but
1091 *      repositioned to one of the neighbors' positions and the
1092 *      neighbor removed. In that case, the segment is defined
1093 *      by the original node and one of its neighbors rather than
1094 *      the two original neighbors. If one of the original neighbors
1095 *      is gone, use the original node instead.
1096 */
1097     if (nbr1 == (Node_t *)NULL) {
1098         nbr1 = GetNodeFromTag(home, coarsenList[i].nodeTag);
1099     } else if (nbr2 == (Node_t *)NULL) {
1100         nbr2 = GetNodeFromTag(home, coarsenList[i].nodeTag);
1101     }
1102
1103 /*
1104 *      One last safety check... it is possible that coarsening
1105 *      out the node led to annihilation of the node and the
1106 *      neighboring nodes (i.e. a triangle collapses in on itself).
1107 *      If that happens, we don't need to do any more.
1108 */
1109     if ((nbr1 == (Node_t *)NULL) || (nbr2 == (Node_t *)NULL)) {
1110         continue;
1111     }
1112
1113     MarkNodeForceObsolete(home, nbr1);
1114     MarkNodeForceObsolete(home, nbr2);
1115
1116     for (q = 0; q < nbr1->numNbrs; q++) {
1117         nbr = GetNodeFromTag(home, nbr1->nbrTag[q]);
1118         if (nbr == (Node_t *)NULL) continue;
1119         MarkNodeForceObsolete(home, nbr);
1120     }
1121
1122     for (q = 0; q < nbr2->numNbrs; q++) {
1123         nbr = GetNodeFromTag(home, nbr2->nbrTag[q]);
1124         if (nbr == (Node_t *)NULL) continue;
1125         MarkNodeForceObsolete(home, nbr);
1126     }
1127
1128 /*
1129 *      If we are enforcing glide planes but allowing them to be
1130 *      slightly fuzzy, we need to recalculate the glide plane for
1131 *      the new segment.
1132 */
1133     if (param->enforceGlidePlanes &&
1134         param->allowFuzzyGlidePlanes) {
1135
1136         RecalcSegGlidePlane(home, nbr1, nbr2, 1);
1137     }
1138
1139 } /* end "omp parallel for" section */
1140
1141 #ifdef DEBUG_LOG_MESH_COARSEN
1142 #ifdef PARALLEL
1143     int globalCoarsenCnt = 0;
1144     MPI_Reduce(&localCoarsenCnt, &globalCoarsenCnt, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
1145 #else
1146     int globalCoarsenCnt = localCoarsenCnt;

```



```

1147 #endif
1148     if (home->myDomain == 0) {
1149         printf("  Remesh: coarsen count = %d\n", globalCoarsenCnt);
1150     }
1151 #endif
1152
1153 /*
1154  *   Free the coarsen list now that we no longer need it.
1155  */
1156     if (coarsenList != (CoarsenList_t *)NULL) {
1157         free(coarsenList);
1158     }
1159
1160     return;

```

Line-by-line explanation of commit and repair.

Lines 764–816

Candidates are revisited in parallel, but `ProceedWithCoarsen()` runs in a critical section. It prevents two events sharing a node or neighbor from both committing.

Lines 818–838

Current nodes are resolved from saved tags because earlier events can delete one. Missing data safely cancels the event.

Lines 840–885

Corner-protected endpoints may only merge over a link no longer than $2r_{ann}$. The code tests both potential target neighbors.

Lines 887–918

Segment forces are estimated before mutation. An event is global if either neighbor, or any arm connected to either neighbor, reaches another domain.

Lines 926–983

Actual mutation is in a critical section. Code first attempts `MergeNode(node, nbr1)` at `nbr1`'s location, then tries the other neighbor if the first merge is illegal.

Lines 985–1051

Surviving endpoints are resolved again, estimated forces are installed on the new segment, and the result becomes coarsen-exempt during this pass.

Lines 1053–1139

A stable-topology pass marks endpoint and neighbor forces obsolete. When fuzzy planes are enabled, new segment plane is recalculated.

Lines 1141–1160

Optional diagnostics reduce counts across ranks, then temporary candidate storage is freed.

7.5 Rule 2 refinement: midpoint insertion

```

35 static void MeshRefine(Home_t *home)
36 {
37
38     /*
39      *   Initialize some of the constants we'll need during this call.
40      */
41     int      thisDomain = home->myDomain;
42     Param_t *param      = home->param;

```

```

43
44     real8    areaMax  = param->remeshAreaMax;
45     real8    areaMax2 = areaMax * areaMax;
46     real8    maxSeg2  = param->maxSeg * param->maxSeg;
47     real8    delta    = 1.0e-16;
48
49     int localRefineCnt = 0;
50
51     /*
52     *   Allocate a block of structures to hold info about rediscretization
53     *   events. We allocate enough structures to refine twice as many segments
54     *   as there are nodes which is probably overkill, but it means we
55     *   shouldn't run out of entries and have to reallocate the block,
56     *   plus minimizes the number of operations we'll have to do within
57     *   the thread-critical sections of code.
58     */
59     int        refineListSize = 2 * home->newNodeKeyPtr;
60     int        refineListEnts = 0;
61     RefineList_t *refineList    = (RefineList_t *)NULL;
62
63     if (refineListSize > 0) {
64         refineList = (RefineList_t *)malloc(sizeof(RefineList_t) * refineListSize);
65     }

```

```

95         if ((node = home->nodeKeys[nIndex]) == (Node_t *)NULL) {
96             continue;
97         }
98
99     /*
100    *   It can happen that a node is pinned and has two neighbors.
101    *   Prevent remesh refinement of such a node.
102    */
103     if (HAS_ANY_OF_CONSTRAINTS(node->constraint, PINNED_NODE))
104         continue;
105
106     /*
107    *   If the node has only two arms, use both arm lengths and
108    *   the area in the triangle defined by the node and its
109    *   neighbors to decide whether one of its segments should
110    *   be refined.
111    */
112     if (node->numNbrs == 2) {
113
114         /*
115        *   Calculate the lengths of the node's 2 arms plus
116        *   the distance between the two neighbor nodes.
117        */
118
119         nbr1 = GetNeighborNode(home, node, 0);
120         nbr2 = GetNeighborNode(home, node, 1);
121
122         if ((nbr1 == (Node_t *)NULL) || (nbr2 == (Node_t *)NULL)) {
123             printf("WARNING: Neighbor not found at %s line %d\n",
124                 __FILE__, __LINE__);
125             continue;
126         }
127
128         vec1[X] = nbr1->x - node->x;
129         vec1[Y] = nbr1->y - node->y;
130         vec1[Z] = nbr1->z - node->z;

```

```

129      vec2[X] = nbr2->x - node->x;
130      vec2[Y] = nbr2->y - node->y;
131      vec2[Z] = nbr2->z - node->z;
132
133
134      vec3[X] = vec2[X] - vec1[X];
135      vec3[Y] = vec2[Y] - vec1[Y];
136      vec3[Z] = vec2[Z] - vec1[Z];
137
138      ZImage(param, &vec1[X], &vec1[Y], &vec1[Z]);
139      ZImage(param, &vec2[X], &vec2[Y], &vec2[Z]);
140      ZImage(param, &vec3[X], &vec3[Y], &vec3[Z]);
141
142      r1 = sqrt(DotProduct(vec1, vec1));
143      r2 = sqrt(DotProduct(vec2, vec2));
144      r3 = sqrt(DotProduct(vec3, vec3));
145
146      s = 0.5 * (r1 + r2 + r3);
147      area2 = (s * (s - r1) * (s - r2) * (s - r3));
148
149  /*
150   * Determine if the area of the triangle defined by the node
151   * and its two neighbors is increasing or decreasing.
152   */
153
154      dvec1xdt = nbr1->vX - node->vX;
155      dvec1ydt = nbr1->vY - node->vY;
156      dvec1zdt = nbr1->vZ - node->vZ;
157
158      dvec2xdt = nbr2->vX - node->vX;
159      dvec2ydt = nbr2->vY - node->vY;
160      dvec2zdt = nbr2->vZ - node->vZ;
161
162      dvec3xdt = dvec2xdt - dvec1xdt;
163      dvec3ydt = dvec2ydt - dvec1ydt;
164      dvec3zdt = dvec2zdt - dvec1zdt;
165
166      dr1dt = ((vec1[X] * dvec1xdt) + (vec1[Y] * dvec1ydt) +
167              (vec1[Z] * dvec1zdt)) / (r1 + delta);
168
169      dr2dt = ((vec2[X] * dvec2xdt) + (vec2[Y] * dvec2ydt) +
170              (vec2[Z] * dvec2zdt)) / (r2 + delta);
171
172      dr3dt = ((vec3[X] * dvec3xdt) + (vec3[Y] * dvec3ydt) +
173              (vec3[Z] * dvec3zdt)) / (r3 + delta);
174
175      dsdt = 0.5 * (dr1dt + dr2dt + dr3dt);
176
177      darea2dt = (dsdt * (s-r1) * (s-r2) * (s-r3));
178      darea2dt += s * (dsdt-dr1dt) * (s-r2) * (s-r3);
179      darea2dt += s * (s-r1) * (dsdt-dr2dt) * (s-r3);
180      darea2dt += s * (s-r1) * (s-r2) * (dsdt-dr3dt);
181
182  /*
183   * Start with the assumption that both segments are
184   * permitted to be refined, then do some checks to see
185   * if indeded we are permitted to refine the segments.
186   */
187
188      splitOK[0] = 1;
189      splitOK[1] = 1;

```

```

188
189 /*
190 *      If both nodes of the segment are flagged to have forces
191 *      updated, forces and velocities on the node may not be
192 *      good enough to accurately position the new node, so
193 *      don't refine this segment this cycle.
194 */
195
196     if (((node->flags & NODE_RESET_FORCES) != 0) &&
197         ((nbr1->flags & NODE_RESET_FORCES) != 0)) {
198         splitOK[0] = 0;
199     }
200
201     if (((node->flags & NODE_RESET_FORCES) != 0) &&
202         ((nbr2->flags & NODE_RESET_FORCES) != 0)) {
203         splitOK[1] = 0;
204     }
205
206 /*
207 *      Check if the current domain owns the segments.
208 *      It may only refine a segment it owns...
209 */
210
211     if (splitOK[0]) {
212         splitOK[0] = DomainOwnsSeg(home, OPCLASS_REMESH,
213                                   thisDomain, &nbr1->myTag);
214     }
215
216     if (splitOK[1]) {
217         splitOK[1] = DomainOwnsSeg(home, OPCLASS_REMESH,
218                                   thisDomain, &nbr2->myTag);
219     }
220
221     splitSegList[0] = 1;
222     splitSegList[1] = 2;
223
224 /*
225 *      Check both segments (in the preferred order) and
226 *      determine if we should bisect one.
227 */
228
229     for (segID = 0; segID < 2; segID++) {
230         int segLen;
231         real8 *vec;
232
233         if (splitSegList[segID] == 1) {
234             if (splitOK[0] == 0) {
235                 continue;
236             }
237             nbr = nbr1;
238             vec = vec1;
239             segLen = r1;
240         } else {
241             if (splitOK[1] == 0) {
242                 continue;
243             }
244             nbr = nbr2;
245             vec = vec2;
246             segLen = r2;

```

```

247  /*
248  *          If the segment length is less than the maxseg length,
249  *          AND (the area of the triangle defined by the node and
250  *          its two neighbors is less than the limit OR the area
251  *          is decreasing in size OR the segment is too small to
252  *          bisect) we don't refine it.
253  */
254
255      if ((segLen < param->maxSeg) &&
256          ((area2 < areaMax2) ||
257           (segLen < param->minSeg * 2.0) ||
258           (darea2dt < 0.0))) {
259          continue;
260      }
261
262  /*
263  *          Add the segment to list of segments to be refined.
264  */
265  #ifdef _OPENMP
266  #pragma omp critical (CRIT_REFINE_LIST)
267  #endif
268  {
269      int i, duplicateEntry = 0;
270
271  /*
272  *          Just a quick sanity check, but make sure there
273  *          is still space on the list...
274  */
275
276      if (refineListEnts < refineListSize) {
277
278          /*
279          *          We're not requiring the segment to be owned by
280          *          the current node, so we need to verify the
281          *          segment has not already been placed on the
282          *          list by another thread.
283          */
284
285          for (i = 0; i < refineListEnts; i++) {
286              RefineList_t *entry;
287
288              entry = &refineList[i];
289
290              if ((entry->node1Tag.domainID ==
291                  nbr->myTag.domainID) &&
292                  (entry->node1Tag.index ==
293                  nbr->myTag.index) &&
294                  (entry->node2Tag.domainID ==
295                  node->myTag.domainID) &&
296                  (entry->node2Tag.index ==
297                  node->myTag.index)) {
298                  duplicateEntry = 1;
299                  break;
300              }
301          }
302
303          if (!duplicateEntry) {
304
305              refineList[refineListEnts].node1Tag =
306                  node->myTag;
307              refineList[refineListEnts].node2Tag =
308                  nbr->myTag;
309              refineList[refineListEnts].status =

```

```

306             REMESH_PENDING;
307             VECTOR_COPY(
308                 refineList[refineListEnts].vector,
309                 vec);
310
311             refineListEnts += 1;
312         }
313
314         } /* end if (refineListEnts < refineListSize) */
315
316     } /* end "omp critical" section */
317
318 } /* for (seg = 0; seg < 2; ...) */
319
320 } else {
321     int nbrIndex;
322     /*
323     *
324     * For nodes with other than exactly two arms, we
325     * just refine any arm exceeding the max allowable
326     * segment length, but only do the refinement from the
327     * domain "owning" the segment.
328     *
329     * A "split" is only considered a global operation
330     * to be sent to remote domains if the segment being
331     * split spans domains.
332     */
333     for (nbrIndex = 0; nbrIndex < node->numNbrs; nbrIndex++) {
334
335         nbr1 = GetNeighborNode(home, node, nbrIndex);
336
337         if (nbr1 == (Node_t *)NULL) {
338             printf("WARNING: Neighbor not found at %s line %d\n",
339                 __FILE__, __LINE__);
340             continue;
341         }
342
343         if (OrderNodes(node, nbr1) >= 0) {
344             continue;
345         }
346
347         if (!DomainOwnsSeg(home, OPCLASS_REMESH,
348             thisDomain, &nbr1->myTag)) {
349             continue;
350         }
351
352         vec1[X] = nbr1->x - node->x;
353         vec1[Y] = nbr1->y - node->y;
354         vec1[Z] = nbr1->z - node->z;
355
356         ZImage(param, &vec1[X], &vec1[Y], &vec1[Z]);
357
358         r1 = DotProduct(vec1, vec1);
359
360         /*
361         *
362         * If the segment is too long, add it to the list of
363         * segments to be refined
364         */
365         if (r1 > maxSeg2) {

```

```

365 *           If there is no more room on the list, reallocate
366 *           the buffer with some more entries.
367 */
368 #ifndef _OPENMP
369 #pragma omp critical (CRIT_REFINE_LIST)
370 #endif
371 {
372     int i, duplicateEntry = 0;
373     /*
374     *           Just a quick sanity check, but make sure there
375     *           is still space on the list...
376     */
377     if (refineListEnts < refineListSize) {
378         /*
379         *           We're not requiring the segment to be
380         *           owned by the current node, so we need to
381         *           verify the segment has not already been
382         *           placed on the list by another thread.
383         */
384         for (i = 0; i < refineListEnts; i++) {
385             RefineList_t *entry;
386
387             entry = &refineList[i];
388
389             if ((entry->node1Tag.domainID ==
390                 nbr1->myTag.domainID)      &&
391                 (entry->node1Tag.index ==
392                 nbr1->myTag.index)          &&
393                 (entry->node2Tag.domainID ==
394                 node->myTag.domainID)      &&
395                 (entry->node2Tag.index ==
396                 node->myTag.index)) {
397                 duplicateEntry = 1;
398                 break;
399             }
400         }
401
402         if (!duplicateEntry) {
403
404             refineList[refineListEnts].node1Tag =
405                 node->myTag;
406             refineList[refineListEnts].node2Tag =
407                 nbr1->myTag;
408             refineList[refineListEnts].status =
409                 REMESH_PENDING;
410
411             VECTOR_COPY(refineList[refineListEnts].vector, vec1);
412
413             refineListEnts += 1;
414         }
415     }
416 }
417 } /* end "omp critical" section */
418 }
419 }
420 }

```

Line-by-line explanation of selection.

Lines 35–65

The function derives squared maximum length/area thresholds and allocates a refine list. The leading source comment calls this version 3, but dispatcher lines in `Remesh.cc` show that the file implements rule 2; this is a source-comment typo.

Lines 95–104

Pinned nodes are never used as degree-two refinement anchors.

Lines 106–179

A degree-two node uses its two arm lengths and local triangle geometry. Rule 2 area derivative includes all endpoint velocities.

Lines 181–217

Each arm starts eligible, then is rejected if forces at both endpoints are stale or this domain does not own the segment.

Lines 219–318

A segment is selected if it exceeds `maxSeg`, or if it is sufficiently curved: it is not too short to bisect, local area exceeds `remeshAreaMax`, and local area is not shrinking. Duplicate checks ensure only one list entry represents the physical segment.

Lines 320–420

Nodes whose degree is not two use a simpler rule: the stable lower tag endpoint considers each owned arm once and lists it only when its squared length exceeds `maxSeg2`.

```
426 /*
427 *      Next phase is to loop through the segments selected for
428 *      refinement and bisect them.
429 *
430 *      Note: The restrictions implemented when adding segments to the list
431 *            should be sufficient that we can refine every segment that
432 *            made it onto the list.
433 */
434 #ifndef _OPENMP
435 #pragma omp parallel for if (refineListEnts > 1)
436 #endif
437     for (int segIndex=0; segIndex < refineListEnts; segIndex++) {
438         int    splitStatus=0, globalOp;
439         real8   vec1[3];
440         real8   newPos[3], newVel[3];
441         real8   nodePos[3], nodeVel[3];
442         real8   f0Seg1[3], f1Seg1[3], f0Seg2[3], f1Seg2[3];
443         Node_t *node1, *node2, *splitNode1, *splitNode2;
444
445         node1 = GetNodeFromTag(home, refineList[segIndex].node1Tag);
446         node2 = GetNodeFromTag(home, refineList[segIndex].node2Tag);
447         VECTOR_COPY(vec1, refineList[segIndex].vector);
448
449 /*
450 *      Splitting a segment screws with the associated nodes'
451 *      segment lists, velocities, etc.  In order to prevent multiple
452 *      threads from messing with the same nodes simultaneously
453 *      lock the two nodes.
454 */
455         LOCK(&node1->nodeLock);
456         LOCK(&node2->nodeLock);
457
458 /*
```



```

459 *           Get an estimate of forces that would change due to refining
460 *           the segment.
461 */
462     newPos[X] = node1->x + (vec1[X] * 0.5);
463     newPos[Y] = node1->y + (vec1[Y] * 0.5);
464     newPos[Z] = node1->z + (vec1[Z] * 0.5);
465
466     newVel[X] = (node1->vX + node2->vX) * 0.5;
467     newVel[Y] = (node1->vY + node2->vY) * 0.5;
468     newVel[Z] = (node1->vZ + node2->vZ) * 0.5;
469
470 /*
471 *           Because the EstRefinementForces() call looks at the segment
472 *           lists of <node1> and <node2> to find the connections between
473 *           them, this call must be within the critical section or we
474 *           risk another thread bisecting a segment attached to one of
475 *           the nodes and modifying the node's segment list while
476 *           EstRefinementForces is looking at the list.
477 */
478     EstRefinementForces(home, node1, node2, newPos, vec1,
479                         f0Seg1, f1Seg1, f0Seg2, f1Seg2);
480
481     FoldBox(param, &newPos[X], &newPos[Y], &newPos[Z]);
482
483 /*
484 *           This should be a global operation distributed out to remote
485 *           domains only if the segment spans domains.
486 */
487     globalOp = (node2->myTag.domainID != node1->myTag.domainID);
488
489     nodePos[X] = node1->x;
490     nodePos[Y] = node1->y;
491     nodePos[Z] = node1->z;
492
493     nodeVel[X] = node1->vX;
494     nodeVel[Y] = node1->vY;
495     nodeVel[Z] = node1->vZ;
496
497 /*
498 *           Need a critical section now because we'll be inserting a
499 *           node and changing connections between nodes...
500 */
501 #ifdef _OPENMP
502 #pragma omp critical (CRIT_REFINE_SEG)
503 #endif
504     {
505 /*
506 *           When refining a segment, we always move exactly one arm
507 *           from the original node to the node being created...
508 */
509         int armIndex = GetArmID(node1, node2);
510         int armCount = 1;
511
512         if (armIndex >= 0) {
513
514             splitStatus = SplitNode(home, OPCLASS_REMESH,
515                                     node1, nodePos, newPos,
516                                     nodeVel, newVel,
517                                     armCount, &armIndex,

```

```

518             globalOp, &splitNode1,
519             &splitNode2, 0);
520
521             if (splitStatus == SPLIT_SUCCESS) {
522                 localRefineCnt++;
523             }
524         }
525
526     } /* end "omp critical" section */
527
528     /*
529     *       It's safe to allow another thread to mess with the two
530     *       nodes again.
531     */
532     UNLOCK(&node1->nodeLock);
533     UNLOCK(&node2->nodeLock);
534
535     /*
536     *       We refined the segment, so we need to save some more info related
537     *       to this refine event. Don't need to lock anything here since
538     *       the refine list will not be reallocated here nor will any other
539     *       thread be updating this entry.
540     */
541     if (splitStatus == SPLIT_SUCCESS) {
542         refineList[segIndex].status = REMESH_COMMITTED;
543         refineList[segIndex].newNodeTag = splitNode2->myTag;
544         VECTOR_COPY(refineList[segIndex].f0Seg1, f0Seg1);
545         VECTOR_COPY(refineList[segIndex].f1Seg1, f1Seg1);
546         VECTOR_COPY(refineList[segIndex].f0Seg2, f0Seg2);
547         VECTOR_COPY(refineList[segIndex].f1Seg2, f1Seg2);
548     }
549 }
550
551 /*
552 *       Last step is to update nodal forces and velocities for
553 *       nodes involved in the the refine events
554 */
555 #ifdef _OPENMP
556 #pragma omp parallel for if (refineListEnts > 1)
557 #endif
558 for (int segIndex=0; segIndex < refineListEnts; segIndex++)
559 {
560     MobArgs_t mobArgs;
561
562     /*
563     *       I don't think any of the refine events should have failed
564     *       in the loop above, but just in case...
565     */
566     if (refineList[segIndex].status != REMESH_COMMITTED) {
567         continue;
568     }
569
570     Node_t *node1 = GetNodeFromTag(home, refineList[segIndex].node1Tag );
571     Node_t *node2 = GetNodeFromTag(home, refineList[segIndex].node2Tag );
572     Node_t *newNode = GetNodeFromTag(home, refineList[segIndex].newNodeTag);
573
574     /*
575     *       Mark the force and velocity data for some nodes as obsolete so
576     *       that more accurate forces will be recalculated at the beginning

```

```

577 *           of the next timestep.
578 */
579     MarkNodeForceObsolete(home, node1 );
580     MarkNodeForceObsolete(home, node2 );
581     MarkNodeForceObsolete(home, newNode);
582
583 /*
584 *           Using the force estimates calculated in the previous loop,
585 *           reset nodal forces on all nodes involved in the refine event.
586 *           Need to lock each node as we update it so no other thread
587 *           updates the same node simultaneously
588 */
589     real8  f0Seg1[3]; VECTOR_COPY(f0Seg1, refineList[segIndex].f0Seg1);
590     real8  f1Seg1[3]; VECTOR_COPY(f1Seg1, refineList[segIndex].f1Seg1);
591     real8  f0Seg2[3]; VECTOR_COPY(f0Seg2, refineList[segIndex].f0Seg2);
592     real8  f1Seg2[3]; VECTOR_COPY(f1Seg2, refineList[segIndex].f1Seg2);
593
594 /*
595 *           Node 1
596 */
597     LOCK(&node1->nodeLock);
598     ResetSegForces(home, node1, &newNode->myTag, f0Seg1[X], f0Seg1[Y], f0Seg1[Z], 1);
599     (void)EvaluateMobility(home, node1, &mobArgs);
600     UNLOCK(&node1->nodeLock);
601
602 /*
603 *           New node
604 */
605     LOCK(&newNode->nodeLock);
606     ResetSegForces(home, newNode, &node1->myTag, f1Seg1[X], f1Seg1[Y], f1Seg1[Z], 1);
607     ResetSegForces(home, newNode, &node2->myTag, f0Seg2[X], f0Seg2[Y], f0Seg2[Z], 1);
608     (void)EvaluateMobility(home, newNode, &mobArgs);
609     UNLOCK(&newNode->nodeLock);
610
611 /*
612 *           Node 2
613 */
614     LOCK(&node2->nodeLock);
615     ResetSegForces(home, node2, &newNode->myTag, f1Seg2[X], f1Seg2[Y], f1Seg2[Z], 1);
616     (void)EvaluateMobility(home, node2, &mobArgs);
617     UNLOCK(&node2->nodeLock);
618
619 /*
620 *           When debugging, dump some info on refine events taking
621 *           place and the nodes involved
622 */
623 #ifdef DEBUG_TOPOLOGY_CHANGES
624
625     if ((dbgDom < 0) || (dbgDom == home->myDomain))
626     {
627         printf("Remesh/refine:  (%d,%d)--(%d,%d) ==> "
628             "(%d,%d)--(%d,%d)--(%d,%d)\n",
629             node1->myTag.domainID, node1->myTag.index,
630             node2->myTag.domainID, node2->myTag.index,
631             node1->myTag.domainID, node1->myTag.index,
632             newNode->myTag.domainID, newNode->myTag.index,
633             node2->myTag.domainID, node2->myTag.index);
634         PrintNode(node1);
635         PrintNode(newNode);

```

```

636         PrintNode(node2);
637     }
638 #endif
639 }
640
641
642 #ifdef DEBUG_LOG_MESH_REFINE
643 #ifdef PARALLEL
644     int globalRefineCnt = 0;
645     MPI_Reduce(&localRefineCnt, &globalRefineCnt, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
646 #else
647     int globalRefineCnt = localRefineCnt;
648 #endif
649     if (home->myDomain == 0) {
650         printf(" Remesh: refine count = %d\n", globalRefineCnt);
651     }
652 #endif
653
654 /*
655  * Free the refine list now that we no longer need it.
656  */
657     if (refineList != (RefineList_t *)NULL) {
658         free(refineList);
659     }
660
661     return;

```

Line-by-line explanation of Rule 2 commit.

Lines 426–479

Saved endpoint tags are resolved and both nodes lock. Stored segment vector puts the point at the midpoint; endpoint velocities are averaged; replacement-segment forces are estimated before arm arrays change.

Lines 481–526

Insertion point is folded into the periodic box. A split is global only across domains. Under a critical section, `SplitNode()` moves exactly one arm from node 1 to a new node, replacing $n_1 - -n_2$ with $n_1 - -n_{new} - -n_2$.

Lines 528–548

Locks are released. A successful entry saves new tag and four estimated endpoint-force vectors.

Lines 551–618

Each committed event marks three nodes force-stale, installs estimates on the two new segments, and reevaluates mobility under the relevant node lock.

Lines 642–661

Optional logging prints transformation and list storage is released.

7.6 Rule 3 refinement: fitted-arc insertion

```

123     if (node->numNbrs == 2) {
124         real8 p1, p2, p3;
125         real8 mid1[3], mid2[3];
126         real8 mv3[3], rhs[3];
127         real8 mat[3][3], invMat[3][3];
128

```

```

129  /*
130  *      Calculate the lengths of the node's 2 arms plus
131  *      the distance between the two neighbor nodes.
132  */
133      nbr1 = GetNeighborNode(home, node, 0);
134      nbr2 = GetNeighborNode(home, node, 1);
135
136      if ((nbr1 == (Node_t *)NULL) || (nbr2 == (Node_t *)NULL)) {
137          printf("WARNING: Neighbor not found at %s line %d\n",
138              __FILE__, __LINE__);
139          continue;
140      }
141
142      vec1[X] = nbr1->x - node->x;
143      vec1[Y] = nbr1->y - node->y;
144      vec1[Z] = nbr1->z - node->z;
145
146      vec2[X] = nbr2->x - node->x;
147      vec2[Y] = nbr2->y - node->y;
148      vec2[Z] = nbr2->z - node->z;
149
150      vec3[X] = vec2[X] - vec1[X];
151      vec3[Y] = vec2[Y] - vec1[Y];
152      vec3[Z] = vec2[Z] - vec1[Z];
153
154      ZImage(param, &vec1[X], &vec1[Y], &vec1[Z]);
155      ZImage(param, &vec2[X], &vec2[Y], &vec2[Z]);
156      ZImage(param, &vec3[X], &vec3[Y], &vec3[Z]);
157
158      r1 = sqrt(DotProduct(vec1, vec1));
159      r2 = sqrt(DotProduct(vec2, vec2));
160      r3 = sqrt(DotProduct(vec3, vec3));
161
162      s = 0.5 * (r1 + r2 + r3);
163      area2 = (s * (s - r1) * (s - r2) * (s - r3));
164
165  /*
166  *      Determine if the area of the triangle defined by the node
167  *      and its two neighbors is increasing or decreasing.
168  *      However, only calculate the effects of the velocity of
169  *      the primary node on that area (i.e. treat the neighboring
170  *      nodes as if they had zero velocity). If we don't do
171  *      this, there are situations where although the area is
172  *      increasing the primary node is moving *toward* the
173  *      other two nodes and placing new nodes on an arc
174  *      containing the 3 nodes turns out to be the incorrect
175  *      placement.
176  */
177      dr1dt = ((vec1[X] * -node->vX) + (vec1[Y] * -node->vY) +
178              (vec1[Z] * -node->vZ)) / (r1 + delta);
179
180      dr2dt = ((vec2[X] * -node->vX) + (vec2[Y] * -node->vY) +
181              (vec2[Z] * -node->vZ)) / (r2 + delta);
182
183      dr3dt = 0.0;
184
185      dsdt = 0.5 * (dr1dt + dr2dt + dr3dt);
186
187      darea2dt = (dsdt * (s-r1) * (s-r2) * (s-r3));

```

```

188         darea2dt += s * (dsdt-dr1dt) * (s-r2) * (s-r3);
189         darea2dt += s * (s-r1) * (dsdt-dr2dt) * (s-r3);
190         darea2dt += s * (s-r1) * (s-r2) * (dsdt-dr3dt);
191
192     /*
193     *
194     *
195     *
196     */
197     splitOK[0] = 1;
198     splitOK[1] = 1;
199
200     /*
201     *
202     *
203     *
204     *
205     */
206     if (((node->flags & NODE_RESET_FORCES) != 0) &&
207         ((nbr1->flags & NODE_RESET_FORCES) != 0)) {
208         splitOK[0] = 0;
209     }
210
211     if (((node->flags & NODE_RESET_FORCES) != 0) &&
212         ((nbr2->flags & NODE_RESET_FORCES) != 0)) {
213         splitOK[1] = 0;
214     }
215
216     /*
217     *
218     *
219     */
220     if (splitOK[0]) {
221         splitOK[0] = DomainOwnsSeg(home, OPCLASS_REMESH,
222                                   thisDomain, &nbr1->myTag);
223     }
224
225     if (splitOK[1]) {
226         splitOK[1] = DomainOwnsSeg(home, OPCLASS_REMESH,
227                                   thisDomain, &nbr2->myTag);
228     }
229
230     splitSegList[0] = 1;
231     splitSegList[1] = 2;
232
233     /*
234     *
235     *
236     *
237     */
238     mid1[X] = node->x + (vec1[X] * 0.5);
239     mid1[Y] = node->y + (vec1[Y] * 0.5);
240     mid1[Z] = node->z + (vec1[Z] * 0.5);
241
242     mid2[X] = node->x + (vec2[X] * 0.5);
243     mid2[Y] = node->y + (vec2[Y] * 0.5);
244     mid2[Z] = node->z + (vec2[Z] * 0.5);
245
246     p1 = (vec1[X]*mid1[X] + vec1[Y]*mid1[Y] + vec1[Z]*mid1[Z]);

```

```

247     p2 = (vec2[X]*mid2[X] + vec2[Y]*mid2[Y] + vec2[Z]*mid2[Z]);
248
249     cross(vec1, vec2, mv3);
250
251     p3 = (mv3[X]*node->x + mv3[Y]*node->y + mv3[Z]*node->z);
252
253     mat[0][X] = vec1[X];
254     mat[0][Y] = vec1[Y];
255     mat[0][Z] = vec1[Z];
256
257     mat[1][X] = vec2[X];
258     mat[1][Y] = vec2[Y];
259     mat[1][Z] = vec2[Z];
260
261     mat[2][X] = mv3[X];
262     mat[2][Y] = mv3[Y];
263     mat[2][Z] = mv3[Z];
264
265     rhs[X] = p1;
266     rhs[Y] = p2;
267     rhs[Z] = p3;
268
269     /*
270     *
271     *
272     */
273     for (segID = 0; segID < 2; segID++) {
274         int segLen;
275         int burgIndex;
276         real8 *vec, *mid;
277         real8 radius, norm, np, tmp;
278         real8 center[3], dir[3], burg[3], nplane[3];
279         real8 diff[3], newPos[3], tmp3[3];
280
281         if (splitSegList[segID] == 1) {
282             if (splitOK[0] == 0) {
283                 continue;
284             }
285             nbr = nbr1;
286             vec = vec1;
287             segLen = r1;
288             mid = mid1;
289             burgIndex = 0;
290         } else {
291             if (splitOK[1] == 0) {
292                 continue;
293             }
294             nbr = nbr2;
295             vec = vec2;
296             segLen = r2;
297             mid = mid2;
298             burgIndex = 1;
299         }
300     }
301     /*
302     *
303     *
304     *
305     *

```

Check both segments (in the preferred order) and determine if we should bisect one.

If the segment length is less than the maxseg length, AND (the area of the triangle defined by the node and its two neighbors is less than the limit OR the area is decreasing in size OR the segment is too small to

```

306 *          bisect) we don't refine it.
307 */
308         if ((segLen < param->maxSeg) &&
309             ((area2 < areaMaxSq) ||
310              (segLen < param->minSeg * 2.0) ||
311              (darea2dt < 0.0))) {
312             continue;
313         }
314
315 /*
316 *          Now need to calculate the position along the arc at
317 *          which the new node should be added.
318 */
319         if (area2 > areaMaxSq) {
320             if (Matrix33_Inverse(invMat, mat) < 0 )
321                 { Fatal("%s::%s(%d) : Cannot invert 3X3 matrix!", __FILE__, __func__
, __LINE__ ); }
322
323             Matrix33Vector3Multiply(invMat, rhs, center);
324
325             tmp3[X] = node->x - center[X];
326             tmp3[Y] = node->y - center[Y];
327             tmp3[Z] = node->z - center[Z];
328
329             radius = Normal(tmp3);
330
331             tmp3[X] = mid[X] - center[X];
332             tmp3[Y] = mid[Y] - center[Y];
333             tmp3[Z] = mid[Z] - center[Z];
334
335             norm = Normal(tmp3);
336
337             dir[X] = tmp3[X] / norm;
338             dir[Y] = tmp3[Y] / norm;
339             dir[Z] = tmp3[Z] / norm;
340
341             burg[X] = node->burgX[burgIndex];
342             burg[Y] = node->burgY[burgIndex];
343             burg[Z] = node->burgZ[burgIndex];
344
345             cross(burg, vec, nplane);
346             np = Normal(nplane);
347
348             if (np > eps) {
349                 nplane[X] /= np;
350                 nplane[Y] /= np;
351                 nplane[Z] /= np;
352             } else {
353                 nplane[X] = 0.0;
354                 nplane[Y] = 0.0;
355                 nplane[Z] = 0.0;
356             }
357
358             diff[X] = (radius*dir[X]) - (mid[X]-center[X]);
359             diff[Y] = (radius*dir[Y]) - (mid[Y]-center[Y]);
360             diff[Z] = (radius*dir[Z]) - (mid[Z]-center[Z]);
361
362             tmp = (diff[X]*nplane[X] +
363                  diff[Y]*nplane[Y] +

```



```

364         diff[Z]*nplane[Z]);
365
366         diff[X] = diff[X] - (tmp * nplane[X]);
367         diff[Y] = diff[Y] - (tmp * nplane[Y]);
368         diff[Z] = diff[Z] - (tmp * nplane[Z]);
369
370         newPos[X] = mid[X] + diff[X];
371         newPos[Y] = mid[Y] + diff[Y];
372         newPos[Z] = mid[Z] + diff[Z];
373     } else {
374         newPos[X] = node->x + (vec[X] * 0.5);
375         newPos[Y] = node->y + (vec[Y] * 0.5);
376         newPos[Z] = node->z + (vec[Z] * 0.5);
377     }

```

Line-by-line explanation of Rule 3 geometry.

Lines 123–190

Rule 3 starts with the same degree-two triangle, but computes area rate using only the central node velocity. Treating neighbors as stationary prevents choosing an arc direction merely because neighbors move away while the central node moves toward them.

Lines 192–231

Ownership and stale-force guards are unchanged from rule 2.

Lines 233–267

Two segment midplanes and normal $\mathbf{v}_1 \times \mathbf{v}_2$ define three equations for the circumcentre of the local three-node arc.

Lines 269–313

Rule 3 uses the same basic length/area decision as rule 2 before spending work on fitted geometry.

Lines 315–323

For sufficiently large area, the 3×3 matrix is inverted to calculate the circle centre. An inversion failure is fatal; the source intentionally does not fall back to a midpoint in this branch.

Lines 325–339

The radius and radial direction through the selected chord midpoint are calculated.

Lines 341–356

The segment glide-plane normal is calculated as $\mathbf{b} \times \mathbf{l}$. A screw segment has zero normal, which is handled without division by zero.

Lines 358–377

Midpoint is moved onto the circumcircle, then its normal-to-glide-plane displacement is removed. The final `newPos` preserves local curvature while respecting the plane. Below curvature threshold, ordinary midpoint insertion is retained.

```

419         refineList[refineListEnts].node1Tag =
420             node->myTag;
421         refineList[refineListEnts].node2Tag =
422             nbr->myTag;
423         refineList[refineListEnts].status =
424             REMESH_PENDING;
425         refineList[refineListEnts].haveNewPos = 1;
426
427         VECTOR_COPY(
428             refineList[refineListEnts].vector,
429             vec);
430         VECTOR_COPY(

```

```

431         refineList[refineListEnts].newPos,
432         newPos);
433
434         refineListEnts += 1;

```

```

560     for (int segIndex = 0; segIndex < refineListEnts; segIndex++) {
561         int splitStatus=0, globalOp;
562         real8 vec1[3];
563         real8 newPos[3], newVel[3];
564         real8 nodePos[3], nodeVel[3];
565         real8 f0Seg1[3], f1Seg1[3], f0Seg2[3], f1Seg2[3];
566         Node_t *node1, *node2, *splitNode1, *splitNode2;
567
568         node1 = GetNodeFromTag(home, refineList[segIndex].node1Tag);
569         node2 = GetNodeFromTag(home, refineList[segIndex].node2Tag);
570         VECTOR_COPY(vec1, refineList[segIndex].vector);
571
572         /*
573          * Splitting a segment screws with the associated nodes'
574          * segment lists, velocities, etc. In order to prevent multiple
575          * threads from messing with the same nodes simultaneously
576          * lock the two nodes.
577          */
578         LOCK(&node1->nodeLock);
579         LOCK(&node2->nodeLock);
580
581         /*
582          * For some of the segments we'll be placing the new node
583          * on the arc containing the original 3 nodes. In such cases
584          * we calculated the new position earlier, so use the known
585          * position. Otherwise calculate it here.
586          */
587         if (refineList[segIndex].haveNewPos) {
588             VECTOR_COPY(newPos, refineList[segIndex].newPos);
589         } else {
590             newPos[X] = node1->x + (vec1[X] * 0.5);
591             newPos[Y] = node1->y + (vec1[Y] * 0.5);
592             newPos[Z] = node1->z + (vec1[Z] * 0.5);
593         }
594
595         newVel[X] = (node1->vX + node2->vX) * 0.5;
596         newVel[Y] = (node1->vY + node2->vY) * 0.5;
597         newVel[Z] = (node1->vZ + node2->vZ) * 0.5;
598
599         /*
600          * Get an estimate of forces that would change due to refining
601          * the segment.
602          */
603         EstRefinementForces(home, node1, node2, newPos, vec1,
604                             f0Seg1, f1Seg1, f0Seg2, f1Seg2);
605
606         FoldBox(param, &newPos[X], &newPos[Y], &newPos[Z]);
607
608         /*
609          * This should be a global operation distributed out to remote
610          * domains only if the segment spans domains.
611          */
612         globalOp = (node2->myTag.domainID != node1->myTag.domainID);

```

```

613         nodePos[X] = node1->x;
614         nodePos[Y] = node1->y;
615         nodePos[Z] = node1->z;
616
617         nodeVel[X] = node1->vX;
618         nodeVel[Y] = node1->vY;
619         nodeVel[Z] = node1->vZ;
620
621     /*
622     *         Need a critical section now because we'll be inserting a
623     *         node and changing connections between nodes...
624     */
625     #ifdef _OPENMP
626     #pragma omp critical (CRIT_REFINE_SEG)
627     #endif
628     {
629     /*
630     *         When refining a segment, we always move exactly one arm
631     *         from the original node to the node being created...
632     */
633
634         int armIndex = GetArmID(node1, node2);
635         int armCount = 1;
636
637         if (armIndex >= 0) {
638
639             splitStatus = SplitNode(home, OPCLASS_REMESH,
640                                     node1, nodePos, newPos,
641                                     nodeVel, newVel,
642                                     armCount, &armIndex,
643                                     globalOp, &splitNode1,
644                                     &splitNode2, 0);
645
646             if (splitStatus == SPLIT_SUCCESS) {
647                 localRefineCnt++;
648             }
649         }
650
651     } /* end "omp critical" section */
652
653     /*
654     *         It's safe to allow another thread to mess with the two
655     *         nodes again.
656     */
657
658     UNLOCK(&node1->nodeLock);
659     UNLOCK(&node2->nodeLock);
660
661     /*
662     *         We refined the segment, so we need to save some more info related
663     *         to this refine event. Don't need to lock anything here since
664     *         the refine list will not be reallocated here nor will any other
665     *         thread be updating this entry.
666     */
667
668     if (splitStatus == SPLIT_SUCCESS) {
669         refineList[segIndex].status = REMESH_COMMITTED;
670         refineList[segIndex].newNodeTag = splitNode2->myTag;
671         VECTOR_COPY(refineList[segIndex].f0Seg1, f0Seg1);
672         VECTOR_COPY(refineList[segIndex].f1Seg1, f1Seg1);
673         VECTOR_COPY(refineList[segIndex].f0Seg2, f0Seg2);

```

```

672         VECTOR_COPY(refineList[segIndex].f1Seg2, f1Seg2);
673     }
674 }
675
676 /*
677  *      Last step is to update nodal forces and velocities for
678  *      nodes involved in the the refine events
679  */
680 #ifdef _OPENMP
681 #pragma omp parallel for if (refineListEnts > 1)
682 #endif
683 for (int segIndex = 0; segIndex < refineListEnts; segIndex++)
684 {
685     MobArgs_t mobArgs;
686
687     /*
688     *      I don't think any of the refine events should have failed
689     *      in the loop above, but just in case...
690     */
691     if (refineList[segIndex].status != REMESH_COMMITTED) {
692         continue;
693     }
694
695     Node_t *node1 = GetNodeFromTag(home, refineList[segIndex].node1Tag );
696     Node_t *node2 = GetNodeFromTag(home, refineList[segIndex].node2Tag );
697     Node_t *newNode = GetNodeFromTag(home, refineList[segIndex].newNodeTag);
698
699     /*
700     *      Mark the force and velocity data for some nodes as obsolete so
701     *      that more accurate forces will be recalculated at the beginning
702     *      of the next timestep.
703     */
704     MarkNodeForceObsolete(home, node1);
705     MarkNodeForceObsolete(home, node2);
706     MarkNodeForceObsolete(home, newNode);
707
708     /*
709     *      Using the force estimates calculated in the previous loop,
710     *      reset nodal forces on all nodes involved in the refine event.
711     *      Need to lock each node as we update it so no other thread
712     *      updates the same node simultaneously
713     */
714     real8 f0Seg1[3]; VECTOR_COPY(f0Seg1, refineList[segIndex].f0Seg1);
715     real8 f1Seg1[3]; VECTOR_COPY(f1Seg1, refineList[segIndex].f1Seg1);
716     real8 f0Seg2[3]; VECTOR_COPY(f0Seg2, refineList[segIndex].f0Seg2);
717     real8 f1Seg2[3]; VECTOR_COPY(f1Seg2, refineList[segIndex].f1Seg2);
718
719     /*
720     *      Node 1
721     */
722     LOCK(&node1->nodeLock);
723     ResetSegForces(home, node1, &newNode->myTag, f0Seg1[X], f0Seg1[Y], f0Seg1[Z], 1);
724     (void)EvaluateMobility(home, node1, &mobArgs);
725     UNLOCK(&node1->nodeLock);
726
727     /*
728     *      New node
729     */
730     LOCK(&newNode->nodeLock);

```

```

731     ResetSegForces(home, newNode, &node1->myTag, f1Seg1[X], f1Seg1[Y], f1Seg1[Z], 1);
732     ResetSegForces(home, newNode, &node2->myTag, f0Seg2[X], f0Seg2[Y], f0Seg2[Z], 1);
733     (void)EvaluateMobility(home, newNode, &mobArgs);
734     UNLOCK(&newNode->nodeLock);
735
736     /*
737     *         Node 2
738     */
739     LOCK(&node2->nodeLock);
740     ResetSegForces(home, node2, &newNode->myTag, f1Seg2[X], f1Seg2[Y], f1Seg2[Z], 1);
741     (void)EvaluateMobility(home, node2, &mobArgs);
742     UNLOCK(&node2->nodeLock);
743
744     /*
745     *         When debugging, dump some info on refine events taking
746     *         place and the nodes involved
747     */
748     #ifndef DEBUG_TOPOLOGY_CHANGES
749
750         if ((dbgDom < 0) || (dbgDom == home->myDomain)) {
751             printf("Remesh/refine:  (%d,%d)--(%d,%d) ==> "
752                  " (%d,%d)--(%d,%d)--(%d,%d)\n",
753                  node1->myTag.domainID, node1->myTag.index,
754                  node2->myTag.domainID, node2->myTag.index,
755                  node1->myTag.domainID, node1->myTag.index,
756                  newNode->myTag.domainID, newNode->myTag.index,
757                  node2->myTag.domainID, node2->myTag.index);
758             PrintNode(node1);
759             PrintNode(newNode);
760             PrintNode(node2);
761         }
762     #endif
763 }
764
765
766 #ifdef DEBUG_LOG_MESH_REFINE
767 #ifdef PARALLEL
768     int globalRefineCnt=0;
769     MPI_Reduce(&localRefineCnt, &globalRefineCnt, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
770 #else
771     int globalRefineCnt = localRefineCnt;
772 #endif
773     if (home->myDomain == 0) {
774         printf("  Remesh: refine count = %d\n", globalRefineCnt);
775     }
776 #endif
777
778     /*
779     *         Free the refine list now that we no longer need it.
780     */
781     if (refineList != (RefineList_t *)NULL) {
782         free(refineList);
783     }
784
785     return;

```

Line-by-line explanation of Rule 3 commit.

Lines 419–434

The computed arc position is stored with `haveNewPos=1`. The record also retains the original segment vector for force estimation and fallback processing.

Lines 560–606

At commit, `haveNewPos` selects stored arc point or midpoint. Velocity interpolation and force estimation remain as in rule 2.

Lines 608–673

Global-operation decision, one-arm `SplitNode()` call, locking, and committed-record update match Rule 2 semantics.

Lines 676–785

Final force-obsolete, estimated-force, and mobility updates also match. The rules differ primarily in candidate geometry and insertion position, not graph mutation or MPI behavior.

7.7 Rule comparison

Rule	Selection trigger	Insertion placement
2	Long segment or a nonshrinking high-area degree-two triangle	Midpoint of selected segment.
3	Same basic trigger, with extra degree-two curved-geometry work	Fitted circumcircle/arc point projected to glide plane; midpoint fallback.

Both rules call `MeshCoarsen()` first, then refine. Both use the same operation records, locks, force estimates, MPI replay, duplicate-link cleanup, and orphan removal.

8 End-to-end transactions

8.1 Example: a long degree-two segment

1. `ParadisStep()` reaches `Remesh()` only after the physics-driven topology exchange completes.
2. `Remesh()` clears operation log, resets exemptions, and calls `RemeshRule_2()` or `RemeshRule_3()`.
3. `MeshRefine()` recognizes an owned segment whose length exceeds `maxSeg`. Rule 2 chooses a midpoint; rule 3 may choose an arc point for a curved degree-two configuration.
4. It locks endpoints, estimates forces for two replacements, then calls `SplitNode(OPCLASS_REMESH, ...)` with one selected arm.
5. `SplitNode()` creates a native node, relinks old neighbor, inserts the required connecting arm, and appends operations if the segment crosses domains.
6. Refinement installs estimates and marks nodes for accurate future force calculation. `CommSendRemesh()` and `FixRemesh()` make remote ghost topology match; cleanup removes duplicates and orphans.

8.2 Example: a short, collapsing degree-two node

1. `MeshCoarsen()` verifies degree two, unpinned status, ownership, glide-plane compatibility, maximum future length, and corner protections.
2. A shrinking small triangle or an arm shorter than `minSeg` adds a candidate. Overlap control chooses a nonoverlapping subset.
3. The code estimates future direct-segment force and calls `MergeNode(OPCLASS_REMESH, ...)` at an allowed neighbor position.

4. `MergeNode()` chooses legally deletable endpoint, transfers arms to survivor, inherits constraints, and invokes duplicate-link cleanup.
5. The new segment receives estimated forces; exact forces are marked obsolete. The second MPI exchange propagates deletion and relinking.

8.3 Example: a multi-node separation

1. The serial path enumerates arm partitions and makes each a temporary zero-length `SplitNode()` transaction. Parallel path calculates the same candidate force data across contributing domains.
2. Mobility, junction force, separation direction, and $f \cdot v$ power select the best admissible partition.
3. Every trial is rolled back. Only winning split runs with `globalOp=1`, writes operation records, installs saved forces, and marks participants collision-exempt.
4. Collision handling then runs and replays full log remotely. A later remesh pass can refine a new junction segment but does not normally coarsen a protected BCC physical corner.

9 Validation, limitations, and developer checklist

9.1 Initialization checks that protect remesh

```

690     }
691
692     if (param->minSeg > (0.5 * param->maxSeg)) {
693         Fatal("Minimum segment length must be < (0.5 * maxSeg)\n"
694             "    Current minSeg = %lf, maxSeg = %lf",
695             param->minSeg, param->maxSeg);
696     }
697
698     if (param->maxSeg <= param->minSeg) {
699         Fatal("Max segment length (%e) must be greater than the\n"
700             "    minimum segment length (%e)", param->maxSeg,
701             param->minSeg);
702     }
703
704     if (param->maxSeg > (minCellSize * 0.9)) {
705         Fatal("The maxSeg length must be less than the "
706             "minimum cell size * 0.9. Current values:\n"
707             "    maxSeg      = %lf\n    cellWidth = %lf",
708             param->maxSeg, minCellSize);
709     }
710
711     if (param->remeshAreaMin > (0.25 * param->remeshAreaMax)) {
712         Fatal("remeshAreaMin must be less than 0.25*remeshAreaMax\n"
713             "    Current remeshAreaMin = %lf, remeshAreaMax = %lf",
714             param->remeshAreaMin, param->remeshAreaMax);
715     }
716
717     // Now check for conditions that although not fatal, may result
718     // in undesired behaviour, and warn the user.
719
720     if (param->rc < 0.1) {
721         fprintf(stderr, "WARNING: specified rc value (%e) will yield a \nnegative core
energy\n", param->rc);
722     }
723

```

```

724         tmp = (param->maxSeg * param->maxSeg * param->maxSeg);
725
726         if (param->remeshAreaMax > (0.25 * sqrt(3) * tmp)) {
727             fprintf(stderr, "WARNING: Area criteria will be unused in remesh operations!\n")
728         ;
729             fprintf(stderr, "          rmeshAreaMax = %lf, maxSeg = %lf\n", param->
remeshAreaMax, param->maxSeg);
730         }
731
732         if (param->rann > (0.5 * param->rc + eps)) {
733             fprintf(stderr, "WARNING: Separation distance is larger than the core radius!\n"
734         );
735             fprintf(stderr, "          rann = %lf, rc = %lf\n", param->rann, param->rc);
736         }
737
738         if (param->rann > (2.0 * param->rTol)) {
739             fprintf(stderr, "WARNING: Collision distance is outside the positioning error
tolerance!\n");
740             fprintf(stderr, "          rann = %lf, rTol = %lf\n", param->rann, param->rTol);
741         }

```

Line-by-line explanation.

Lines 692–702

`minSeg` must be less than half `maxSeg`, and `maxSeg` must exceed `minSeg`. This assures that one bisection can produce acceptable child lengths.

Lines 704–709

`maxSeg` must remain under 90 percent of the smallest cell width, preserving force/cell spatial assumptions.

Lines 711–715

Derived area thresholds must remain mutually consistent; bad geometry tuning is stopped before stepping.

Lines 717–739

Nonfatal warnings expose poor core radius, area, and contact-distance choices that can destabilize topology decisions.

9.2 Implementation details to keep in mind

- In `RemeshRule_2.cc:227` and `RemeshRule_3.cc:274`, `segLen` is declared `int` but is assigned a real-valued arm length before comparisons. Consequently the degree-two length comparisons truncate fractional length. This guide records current behavior; maintainers may wish to audit whether `real8` was intended.
- Candidate arrays are initially sized at twice `home->newNodeKeyPtr`. If full, append code skips later candidates rather than reallocating. That is safe but can defer legitimate remesh work.
- Rule 3 treats a circumcentre-matrix inversion failure as fatal. This avoids silently inserting at invalid position, but it makes nearly degenerate curved cases worth testing.
- Cross slip is in topology phase because it changes segment planes and positions, but it normally preserves node/segment cardinality. Do not describe all cross-slip events as node creation or deletion.

9.3 Invariants for topology modifications

- **Ownership:** mutate a segment only when `DomainOwnsSeg()` grants operation-class ownership; mutate/delete a node only through legal primitive checks.
- **Burgers conservation:** `SplitNode()` creates a connector from moving-side Burgers sum. Outer driver calls `CheckBurgConservation()` after topology replay.
- **Both endpoints agree:** never edit one node's `nbrTag`, Burgers, plane, or force array by hand. Use connection/arm helpers.
- **No overlaps:** candidate detection may be parallel; commits must lock/serialize shared nodes and maintain `REMESH_COMMITTED/REMESH_REJECTED`.
- **Force validity:** respect `NODE_RESET_FORCES`, preserve estimates, and mark affected neighborhoods obsolete after mutation.
- **Periodic geometry:** calculate vectors with `ZImage()` or `PBCPOSITION()`, and fold stored coordinates with `FoldBox()`.
- **Distributed replay:** every remote-visible change needs an `AddOp*()` record. Replay it with `globalOp=0`.
- **Postconditions:** clean duplicate links after remote replay and remove local orphan nodes before ghost communication.

9.4 Useful breakpoints and logging

Location or symbol	What it reveals
<code>src/ParadisStep.cc:403,419,447,493</code>	High-level phase order for one cycle.
<code>src/Topology.cc:2465 SplitNode()</code>	Every arm-moving insertion, including remesh.
<code>src/Topology.cc:2976 MergeNode()</code>	Every coarsen/collision deletion decision.
<code>src/Remesh.cc:714</code> and <code>src/RemeshRule_2.cc:254</code>	Geometry predicates that nominate coarsen/refine candidates.
<code>src/RemoteOps.cc:670</code> and <code>src/FixRemesh.cc:92</code>	A split record being produced and replayed.
<code>DEBUG_TOPOLOGY_CHANGES</code>	Prints separation, coarsen, refine, and collision operations.
<code>DEBUG_LOG_MESH_COARSEN</code> , <code>DEBUG_LOG_MESH_REFINE</code>	Reduces and prints event counts across ranks.

9.5 Checklist for a new topology event

1. Decide whether event moves arms, merges nodes, changes a segment property, or needs a new primitive. Reuse `SplitNode()` and `MergeNode()` whenever semantics fit.
2. Define deterministic ownership, including a safe response when a candidate spans domains.
3. Screen candidates using force validity, constraints, periodic geometry, and per-cycle exemptions.
4. Make trial operations local/reversible; make final operation global only when remote state can observe it.
5. Record every remote-visible change through an `AddOp*()` operation, or add record type, producer, and `FixRemesh()` case together.
6. Restore/recompute per-arm forces and mobility; mark untrusted data obsolete.
7. Exercise serial, OpenMP, and MPI cases. Verify Burgers conservation, duplicate-link cleanup, and restart/ghost consistency.

Primary sources: the Section 1 code files; docs/ParaDiS_4.0_UserGuide.pdf; and docs/BCC_Binary_Junction_Node_Splitting_code.tex.